



UE23AM342BA3

Explainable AI

Unit II

Local and Global model agnostic Methods

Prof. Ayisha Noori VK, Dr. Siba Ram Baral
Dept. of CSE(AI&ML), PES University,
Bengaluru

UNIT II : Global and Local Model-Agnostic Methods

Understanding Model Behavior Through Partial Dependence Plots
and Accumulated Local Effects



Understanding Global Explanations

What Are Global Methods? Methods?

Global explanation methods reveal how a machine learning model behaves across the entire dataset, not just for individual predictions. They answer the critical question: "On average, how does this feature influence the model's predictions?"

Key Techniques

- **Partial Dependence Plots (PDP):** (PDP): Show average feature effects
- **Accumulated Local Effects (ALE):** (ALE): Handle correlated features features
- **SHAP:** Game-theoretic approach approach to feature importance importance



Partial Dependence Plots: The Foundation



Core Concept

PDPs visualize the average effect of one or two features on model predictions by marginalizing over all other features in the dataset.



Key Mechanism

The method plots model output versus feature values while holding other variables constant, revealing the marginal relationship.



Primary Use

Ideal for understanding how individual features drive predictions and communicating model behavior to stakeholders.

PDP Mathematical Definition

The partial dependence function quantifies the average effect of selected features on predictions:

$$f_{PDP}(x_S) = E_{x_C}[f(x_S, x_C)]$$

xS: Selected Features

The feature(s) of interest whose effect we want to understand and visualize in the plot.

xC: Complement Features

All other features in the model that are marginalized over during the computation.

Expectation Operator

The expectation is calculated over the empirical distribution of complement features in the training data.

PDP in Action: House Price Prediction

Real-World Example

Consider a house price prediction model trained on property features. When we plot the PDP for `size_sqft`, we observe a clear positive relationship: predicted prices rise steadily as house size

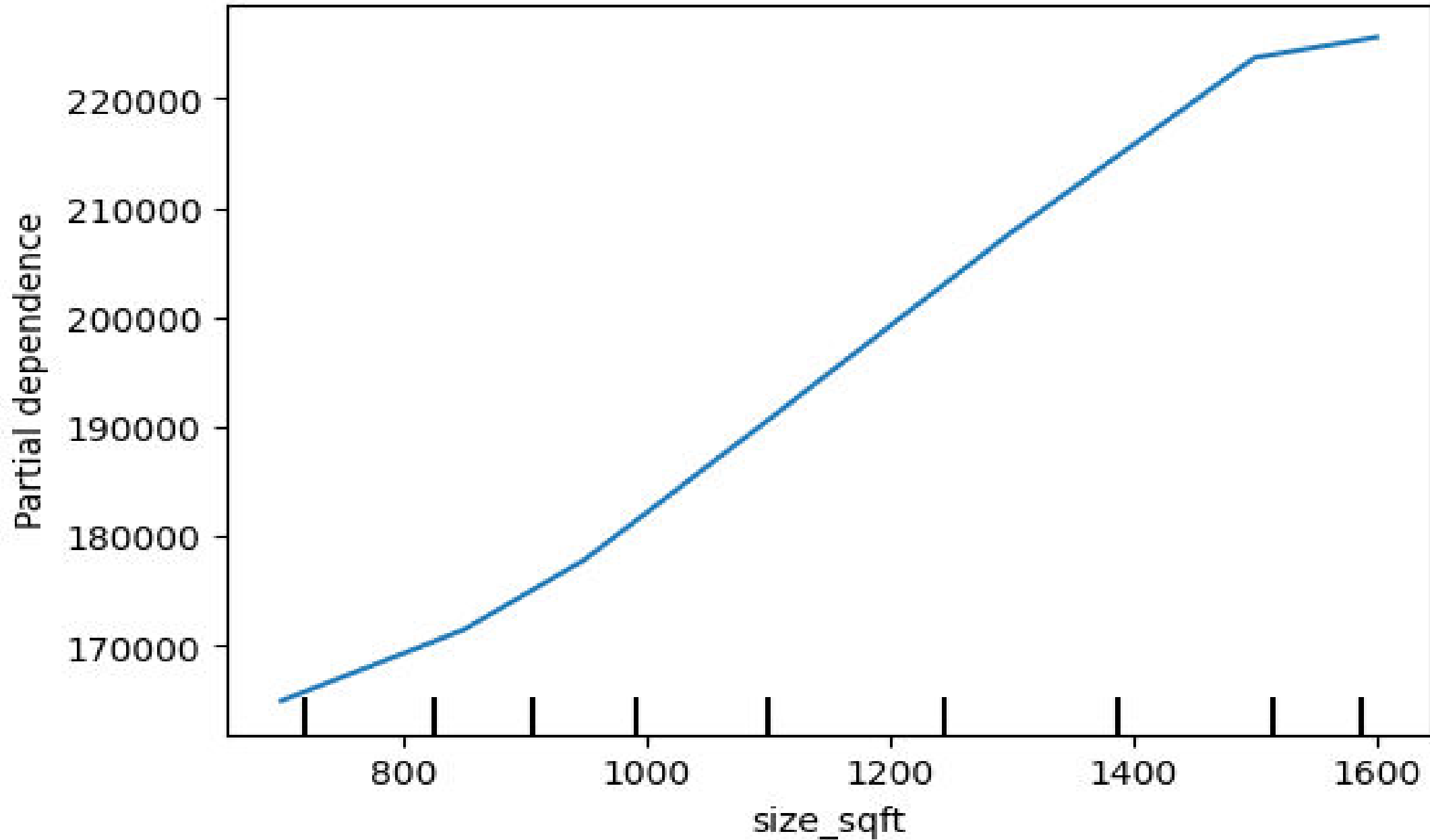
Key Interpretation

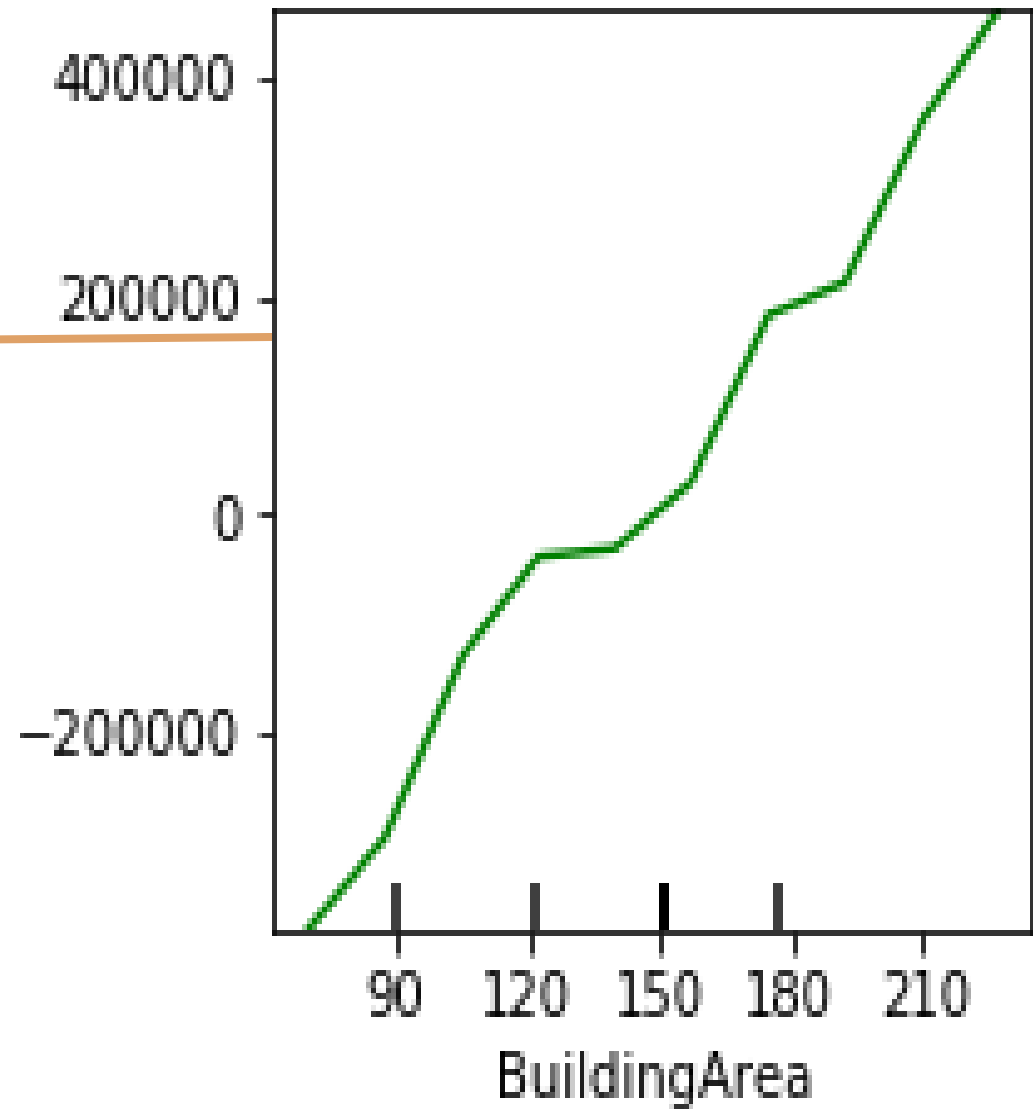
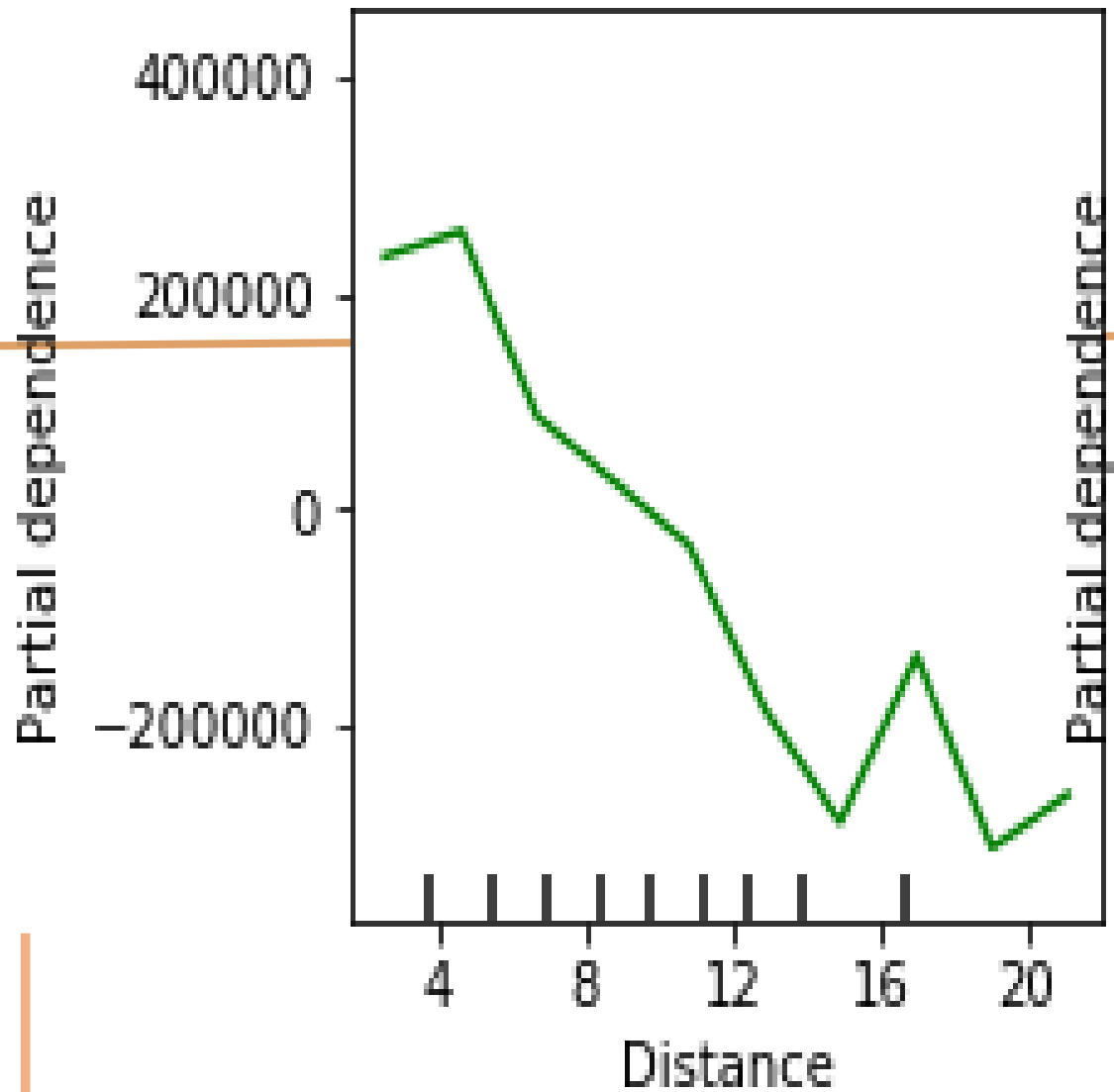
This visualization confirms our intuition—larger houses command higher prices in the market. The smooth curve indicates the model has learned this fundamental relationship consistently across different property types and locations.



Practice - [learning/machine_learning/santander_txn_prediction/04_txn_pred_rf_details.ipynb](#) at master · [harsh1kumar/learning](#)

Partial Dependence Plot (House Size vs Price)





Anatomy of a PDP Visualization

01

X-Axis: Feature Values

Displays the range of the feature being analyzed, such as house size in square feet or customer age in years.

03

The Curve: Marginal Effect

The plotted line reveals how the feature influences predictions—predictions—steep slopes indicate strong effects, flat regions suggest regions suggest minimal impact.

02

Y-Axis: Average Prediction

Shows the average predicted outcome (e.g., price, risk score) at each feature value after marginalizing over all other features.





PDP Strengths and Advantages

Simplicity and Intuition

PDPs are straightforward to understand and explain, making them ideal for communicating model behavior to non-technical stakeholders and decision-makers.

Model Agnostic

Works with any machine learning algorithm from linear regression to deep neural networks without requiring access to internal model parameters.

Feature Analysis

Excellent tool for analyzing individual feature contributions, helping identify which variables drive predictions and how they influence outcomes.

PDP Limitations to Consider

Independence Assumption

PDPs assume features are independent, which rarely holds in real-world data. This can lead to misleading interpretations when features are correlated.

Correlation Challenges

When features are highly correlated, PDPs may show unrealistic scenarios by creating feature combinations that don't exist in the actual data distribution.

Computational Cost

Computing PDPs requires many predictions—the model must be evaluated for each feature value across all data points, making it expensive for large datasets.

Important: Always check for feature correlations before relying solely on PDP interpretations. Consider using ALE plots for correlated features.

PDP Case Study: Disease Risk Prediction

Healthcare Application

In a disease risk prediction model, the PDP for patient age reveals an expected pattern: risk scores increase steadily with age, reflecting the biological reality of age-related health decline.

The Hidden Problem

However, this visualization tells an incomplete story. Age is highly correlated correlated with other risk factors like BMI and blood pressure. The PDP PDP doesn't account for these relationships, potentially showing an exaggerated age effect.

This demonstrates why PDPs should be complemented with correlation-aware methods like ALE plots for a complete understanding.



UNIT 2: Global Model-Agnostic Agnostic Methods

Accumulated Local Effects (ALE)

Accumulated Local Effects: A Robust Alternative



Core Innovation

ALE plots show average feature effects while explicitly accounting for correlations between features, providing more accurate interpretations.



Robustness Advantage

Unlike PDPs, ALE methods remain reliable when features are dependent, making them essential for complex real-world datasets.



Practical Approach

Use PDP for initial exploration and quick insights, then validate with ALE for correlated features to ensure features to ensure trustworthy interpretations.

"Understanding when to use PDP versus ALE is crucial for building trustworthy and interpretable machine learning systems."

Understanding Accumulated Local Effects

ALE plots reveal the average effect of a feature on model predictions while intelligently accounting for correlations. Unlike simpler methods, ALE adapts to the complex relationships in your data.

This makes ALE significantly more robust than Partial Dependence Plots when dealing with dependent features a common reality in real-world datasets.



The Mathematics Behind ALE

ALE builds its power through a calculus-based approach that captures local behavior:

$$ALE(x) = \int_{x_{min}}^x E \left[\frac{\partial f(x)}{\partial x} \right] dx$$

Local Derivatives

Uses the rate of change in model predictions at each point rather than global averages

Integration Process

Accumulates these local effects across the entire feature range to build the complete picture

Unbiased Estimation

Adjusts for correlations by focusing on changes within local neighborhoods

ALE : Disease Risk Prediction

01

Feature Selection

Analyzing age as the primary feature of interest

02

Correlation Adjustment

ALE automatically accounts for correlations with BMI and blood pressure

03

Risk Pattern

Results show disease risk increases with age, with adjustments for adjustments for confounding factors

This example demonstrates how ALE isolates the true effect of age of age by controlling for related health indicators that typically vary typically vary together.

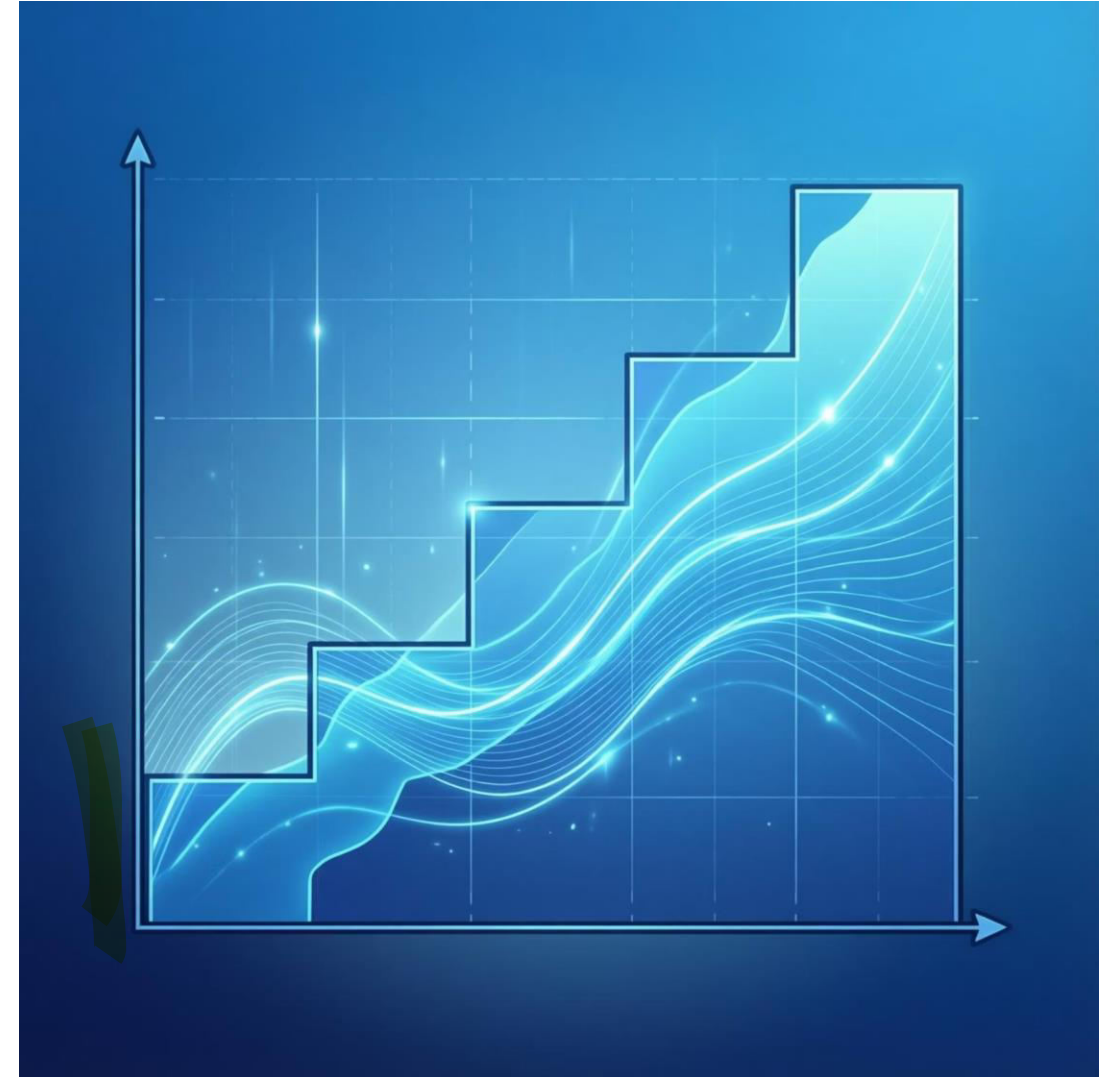


Interpreting ALE Visualizations

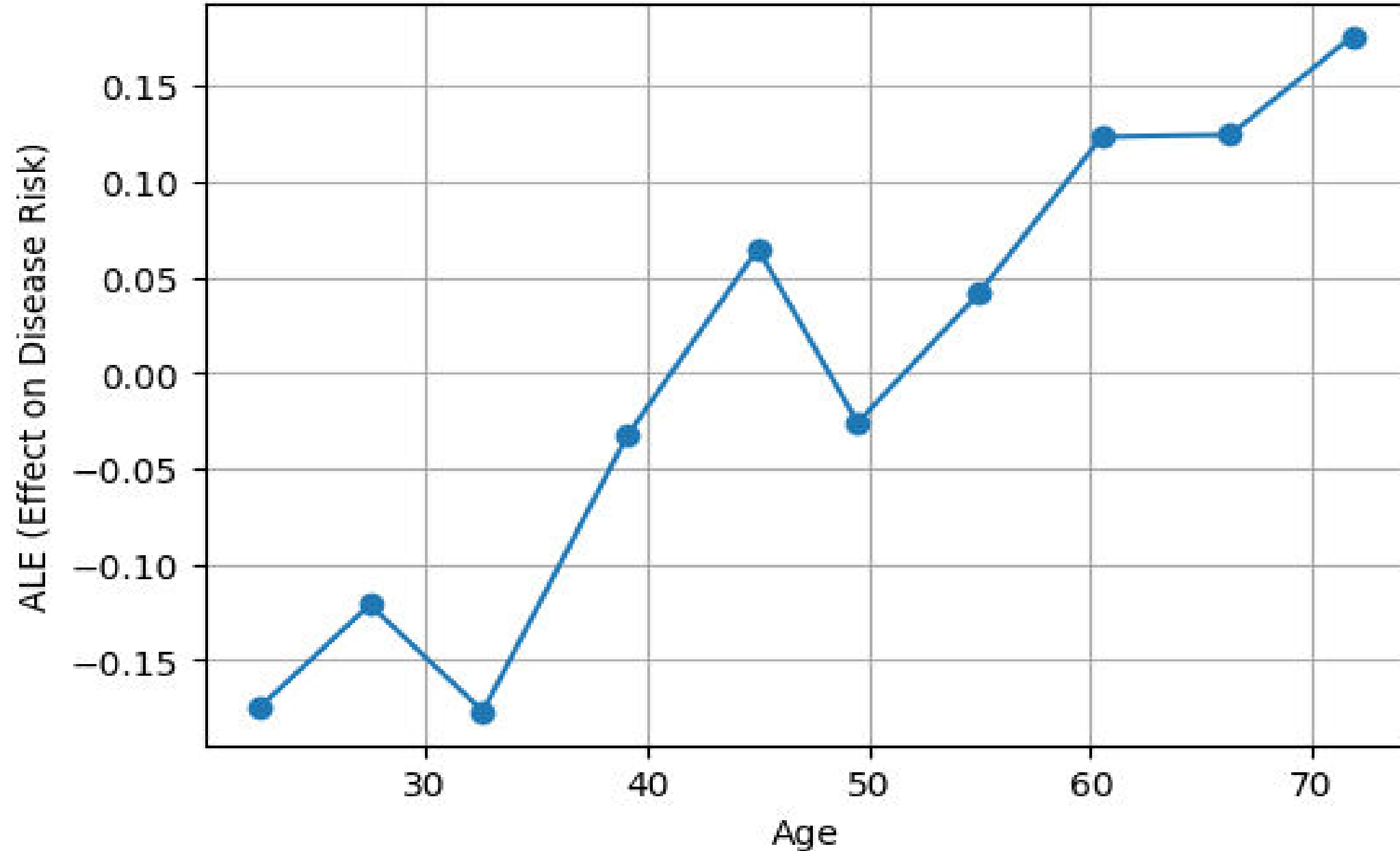
Key Components

- **X-axis:** Feature values (e.g., age from 20 to 80)
- **Y-axis:** Accumulated effect on prediction output
- **Plot style:** Step-like pattern showing local contributions contributions

The step-like appearance reflects how ALE accumulates effects effects across binned intervals of the feature range.



Accumulated Local Effects (Age vs Disease Risk)



Significance of ALE



Correlation Handling

Manages correlated features far far better than PDP by examining examining local changes rather rather than marginal distributions



Unbiased Estimates

Provides accurate feature effect estimates even when features interact or depend on each other



Complex Data Reliability

Performs exceptionally well on real-world datasets with intricate feature relationships

Challenges and Limitations

Computational Complexity

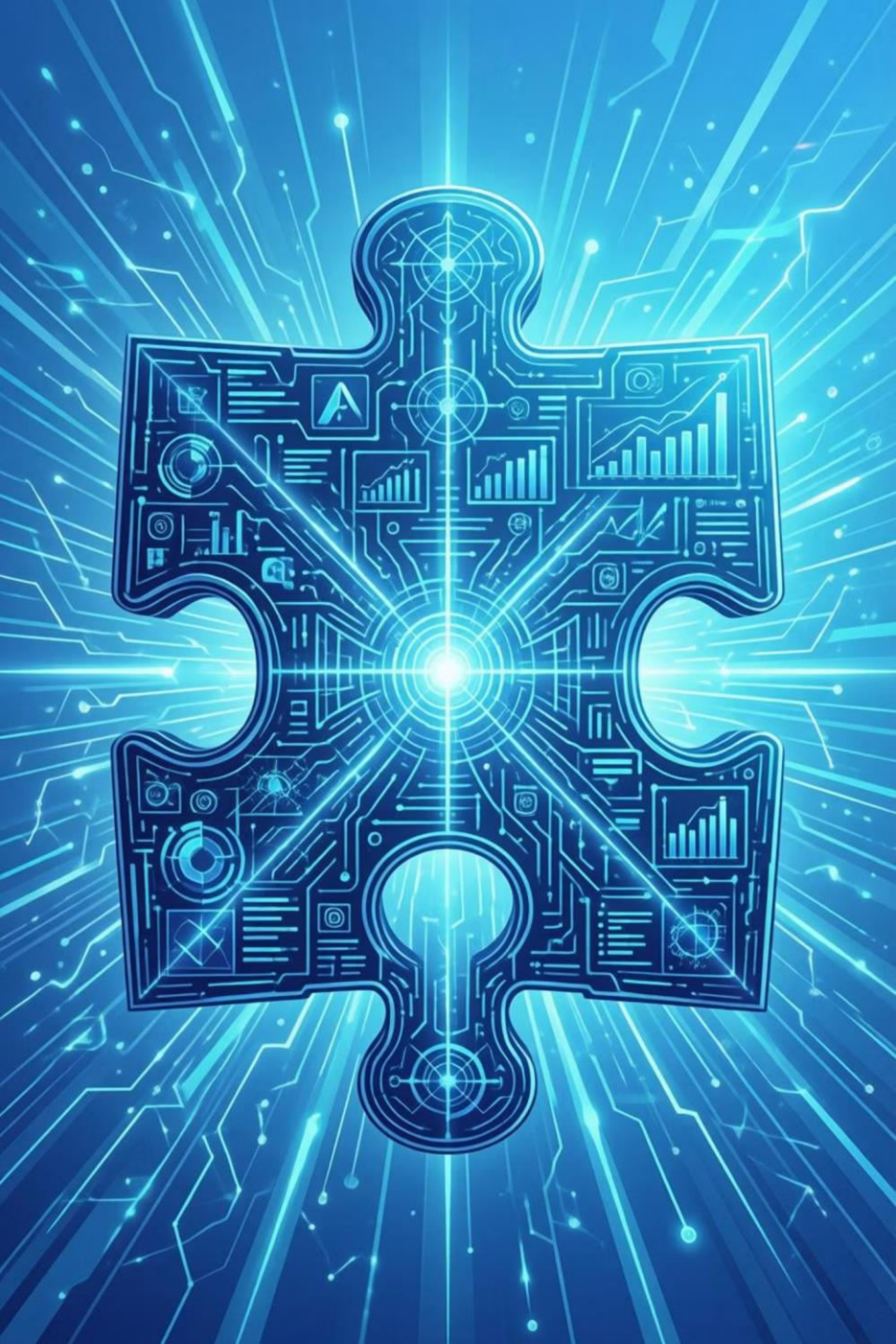
ALE requires more intensive calculations compared to simpler methods like PDP, involving derivative estimation and estimation and integration across feature bins.

Explanation Difficulty

The mathematical sophistication that makes ALE powerful also powerful also makes it harder to explain to stakeholders without without technical backgrounds.

Binning Sensitivity

Results depend on how feature values are divided into intervals—poor binning choices can distort the interpretation. interpretation.



ALE vs PDP: Choosing Your Tool

1

Partial Dependence Plots

- Intuitive and easy to explain
- Assumes feature independence
- Can be misleading with correlated features

2

Accumulated Local Effects

- Robust to feature correlations
- Less intuitive for audiences
- Provides unbiased estimates

❏ **Best Practice:** Use PDP and ALE together for comprehensive model interpretation. PDP offers intuitive global views, while ALE ensures accuracy when features correlate.

Telecom Churn Analysis

Feature: Monthly Charges

In this telecom churn prediction model, ALE analysis of monthly charges reveals a compelling pattern:

- Churn risk increases notably with higher monthly charges
- Effect is adjusted for customer tenure correlation
- Shows true impact isolated from confounding variables

The ALE plot demonstrates that while high charges do increase churn probability, the relationship is more nuanced than raw correlation would suggest once we account for how tenure and charges relate.



Key Takeaways: Mastering ALE

1

Robust Explanations

ALE delivers reliable global interpretations that remain accurate even with correlated features

2

Complementary Methods

PDP + ALE together provide balanced insights—intuition meets accuracy

3

Trustworthy AI

Essential tool for building trustworthy, interpretable models in models in production environments environments

As you advance in AI/ML, remember that model interpretability isn't optional—it's essential for deployment, deployment, debugging, and building stakeholder trust.



THANK YOU



Elective - III

UE23AM342BA3 : Explainable AI

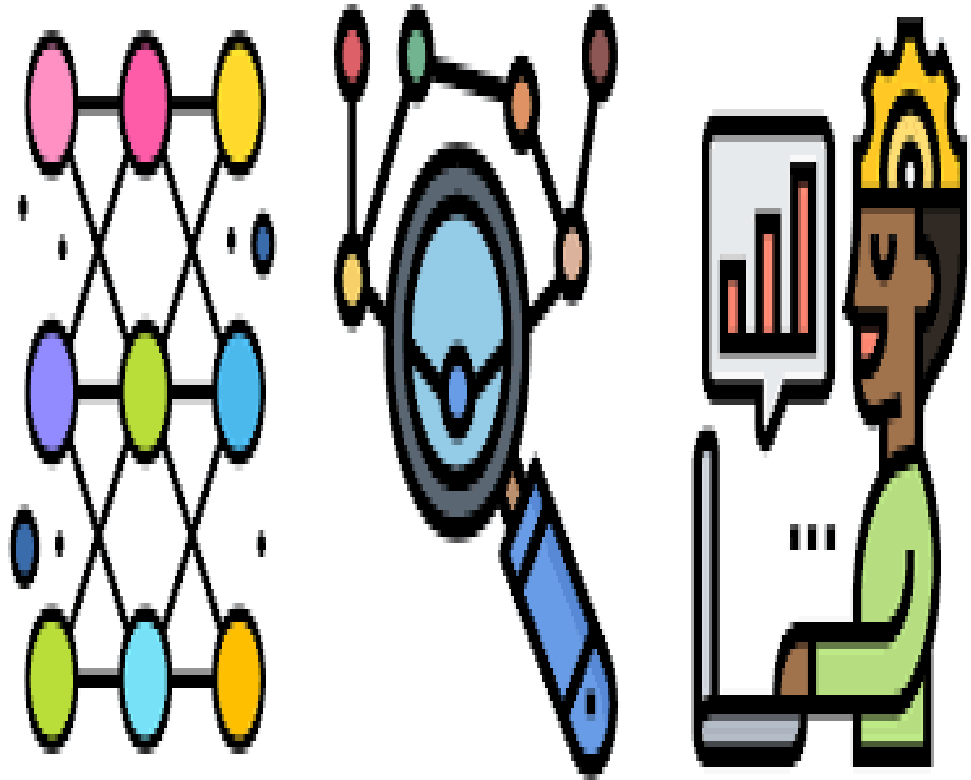
UNIT II : Global model agnostic Methods

Prof.Ayisha Noori V K

**Department of Computer Science and
Engineering(AI&ML)**

Contents:

- **Feature Interactions**
 - Friedman's H-statistic (H-stat or H-index))
- **Permutation Feature Importance**
- **Global Surrogate**



UNIT 2:

Feature Interactions

Feature Interactions

Solo Performance

Age: 25

+

Income: \$50k

Impact = $0.3 + 0.4 = 0.7$
(Just adding things up!)

Dynamic Duo!

Age: 25

 $\times$ 

Income: \$50k

Impact = $0.3 + 0.4 + 2.5$

(They multiply the effect!)

Feature Interactions

When features interact with each other in a prediction model, the prediction cannot be expressed as the sum of the feature effects, because the effect of one feature depends on the value of the other feature. -

Christoph Molnar

What is feature Interactions?

The complex collaborative effects of features towards prediction of a variable is called feature interaction. Another aspect of feature interaction is the variation of one feature with respect to another with which it is interacting. These variables are often referred to as interaction variables.

If a machine learning model makes a prediction based on two features, we can decompose the prediction into four terms:

1. a constant term,
2. a term for the first feature,
3. a term for the second feature
4. a term for the interaction between the two features.

"The whole is greater than the sum of its parts"

Interactions Definition

Research Paper - Xu, Lijuan, et al. "Finding component relationships: a deep-learning-based anomaly detection interpreter." *IEEE Transactions on Computational Social Systems* 11.3 (2024): 4149-4162.

A function $F(x)$ is said to exhibit an interaction between two of its variables x_j and x_k if the difference in the value of $F(x)$ as a result of changing the value of x_j depends on the value of x_k . For numeric variables, this can be expressed as

$$E_x \left[\frac{\partial^2 F(x)}{\partial x_j \partial x_k} \right]^2 > 0$$

Read as – rate of change of the model prediction function with respect to the variables j and k

Feature Interactions : Example

For example, a model predicts the value of a house, using house size (big or small) and location (good or bad) as features, which yields four possible predictions:

Location	Size	Prediction
good	big	300,000
good	small	200,000
bad	big	250,000
bad	small	150,000

We decompose the model prediction into the following parts: A constant term (150,000), an effect for the size feature (+100,000 if big; +0 if small) and an effect for the location (+50,000 if good; +0 if bad). This decomposition fully explains the model predictions. There is no interaction effect, because the model prediction is a sum of the single feature effects for size and location. When you make a small house big, the prediction always increases by 100,000, regardless of location. Also, the difference in prediction between a good and a bad location is 50,000, regardless of size.

Let's now look at an example with interaction:

Location	Size	Prediction
good	big	400,000
good	small	200,000
bad	big	250,000
bad	small	150,000

We decompose the prediction table into the following parts: A constant term (150,000), an effect for the size feature (+100,000 if big, +0 if small) and an effect for the location (+50,000 if good, +0 if bad). For this table we need an additional term for the interaction: +100,000 if the house is big and in a good location. This is an interaction between size and location, because in this case the difference in prediction between a big and a small house depends on the location.

Friedman's H-statistic (H-stat or H-index) -



Friedman's H-statistic (often called the H-stat or H-index) is a machine learning explainability technique developed by Jerome H. Friedman and Bogdan E. Popescu that measures the strength of interactions between features in a predictive model.



If two features do not interact, we can decompose the partial dependence function as follows (assuming the partial dependence functions are centered at zero):

$$PD_{jk}(x_j, x_k) = PD_j(x_j) + PD_k(x_k)$$

where $PD_{jk}(x_j, x_k)$ is the 2-way partial dependence function of both features and $PD_j(x_j)$ and $PD_k(x_k)$ the partial dependence functions of the single features.

Likewise, if a feature has no interaction with any of the other features, we can express the prediction function $\hat{f}(x)$ as a sum of partial dependence functions, where the first summand depends only on j and the second on all other features except j :

$$\hat{f}(x) = PD_j(x_j) + PD_{-j}(x_{-j})$$

where $PD_{-j}(x_{-j})$ is the partial dependence function that depends on all features except the j -th feature.



Friedman's H-statistic (H-stat or H-index))

Assuming no interaction:

$$pd_{jk}(x_j, x_k) = pd_j(x_j) + pd_k(x_k) =$$

If there is no interaction between the variables then the joint partial dependency is equal to the additive partial dependency and hence the H stats will be zero.

Observed:

$$pd_{jk}(x_j, x_k) = \int f(x_j, x_k, X_C) dP(X_C)$$

$$H_{jk}^2 = \frac{\sum_{i=1}^n [\overset{0}{pd_{jk}(x_j^{(i)}, x_k^{(i)})}]^2}{\sum_{i=1}^n pd_{jk}^2(x_j^{(i)}, x_k^{(i)})} = 0$$

Friedman's H-statistic (H-stat or H-index))

If the two features J and K only effect the prediction through an interaction then both the individual partial dependencies will be constant, the numerator will be same as denominator and the H stats will be ONE.

$$pd_{jk}(x_j, x_k) = pd_j(x_j) + pd_k(x_k)$$

Observed:

$$pd_{jk}(x_j, x_k) = \int f(x_j, x_k, X_C) dP(X_C)$$

$$H_{jk}^2 = \frac{\sum_{i=1}^n [pd_{jk}(x_j^{(i)}, x_k^{(i)}) - 0 - 0]^2}{\sum_{i=1}^n pd_{jk}^2(x_j^{(i)}, x_k^{(i)})} = 1$$

Friedman's H-statistic (H-stat or H-index))

If variables are non interacting then the function is just a sum of function of X_j and the function of all the variables excluding X_j .

If a given variable x_j interacts with none of the other variables in a function $F(x)$, then it can be expressed as:

$$F(X) = f(x_j) + g(X_{-j})$$

Assuming no interaction:

$$\hat{f}(X) = pd_j(x_j) + pd_{-j}(X_{-j})$$

$$H_j^2 = \frac{\sum_{i=1}^n [\hat{f}(X^{(i)}) - \overbrace{pd_j(x_j^{(i)}) + pd_{-j}(X_{-j}^{(i)})}^{\text{ }}]^2}{\sum_{i=1}^n \hat{f}^2(X^{(i)})}$$

Friedman's H-statistic (H-stat or H-index) _ Explanation

Interaction between two specific features (j and k):

$$H_{jk}^2 = \frac{\sum_{i=1}^n [PD_{jk}(x_j^{(i)}, x_k^{(i)}) - PD_j(x_j^{(i)}) - PD_k(x_k^{(i)})]^2}{\sum_{i=1}^n PD_{jk}^2(x_j^{(i)}, x_k^{(i)})}$$

- $PD_{jk} \rightarrow$ Actual combined effect of j and k
- $PD_j + PD_k \rightarrow$ What we would expect if there were NO interaction

So the numerator measures:

How much the real joint effect differs from the additive assumption

Squaring ensures:

- Positive values
- Larger penalties for strong interactions

Interaction of **one feature j with all other features**:

$$H_j^2 = \frac{\sum_{i=1}^n [f(x^{(i)}) - PD_j(x_j^{(i)}) - PD_{-j}(x^{(i)})]^2}{\sum_{i=1}^n \hat{f}^2(x^{(i)})}$$

"Does feature j interact with the rest of the model?"

- $PD_j \rightarrow$ effect of feature j alone
- $PD_{-j} \rightarrow$ effect of *all other features together*
- If the model were additive:

$$f(x) = PD_j + PD_{-j}$$

The numerator measures how badly this additive assumption fails.

Interpretation 🎯

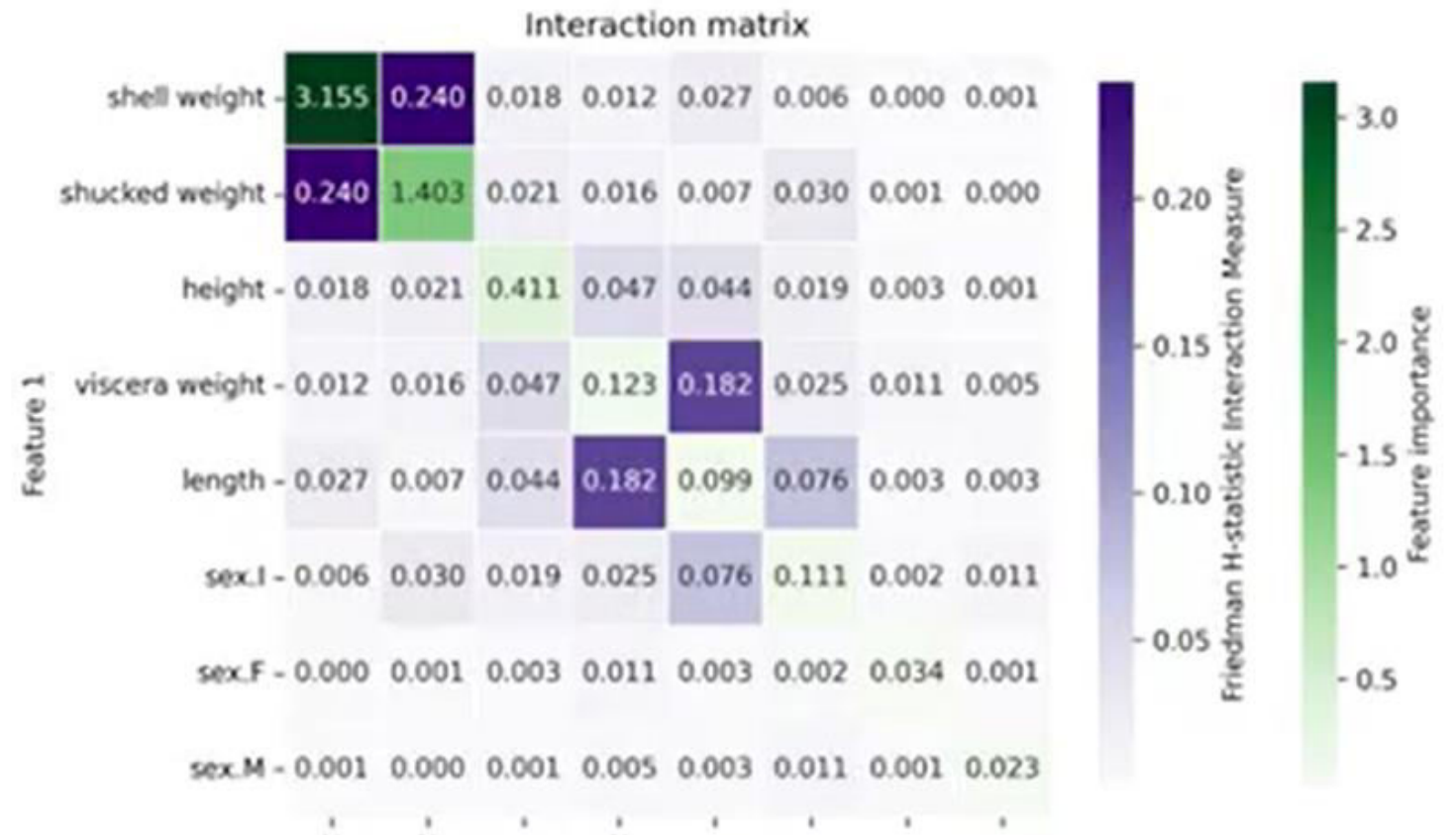
- $H_j^2 \approx 0$
 $\rightarrow$ Feature j acts independently
- H_j^2 large
 $\rightarrow$ Feature j **strongly interacts** with other features

Two versions of this statistics

Pairwise H-stat:

$$H_{jk}^2 = \frac{\sum_{i=1}^n [pd_{jk}(x_j^{(i)}, x_k^{(i)}) - pd_j(x_j^{(i)}) - pd_k(x_k^{(i)})]^2}{\sum_{i=1}^n pd_{jk}^2(x_j^{(i)}, x_k^{(i)})}$$

One for the
interaction
between
two
features

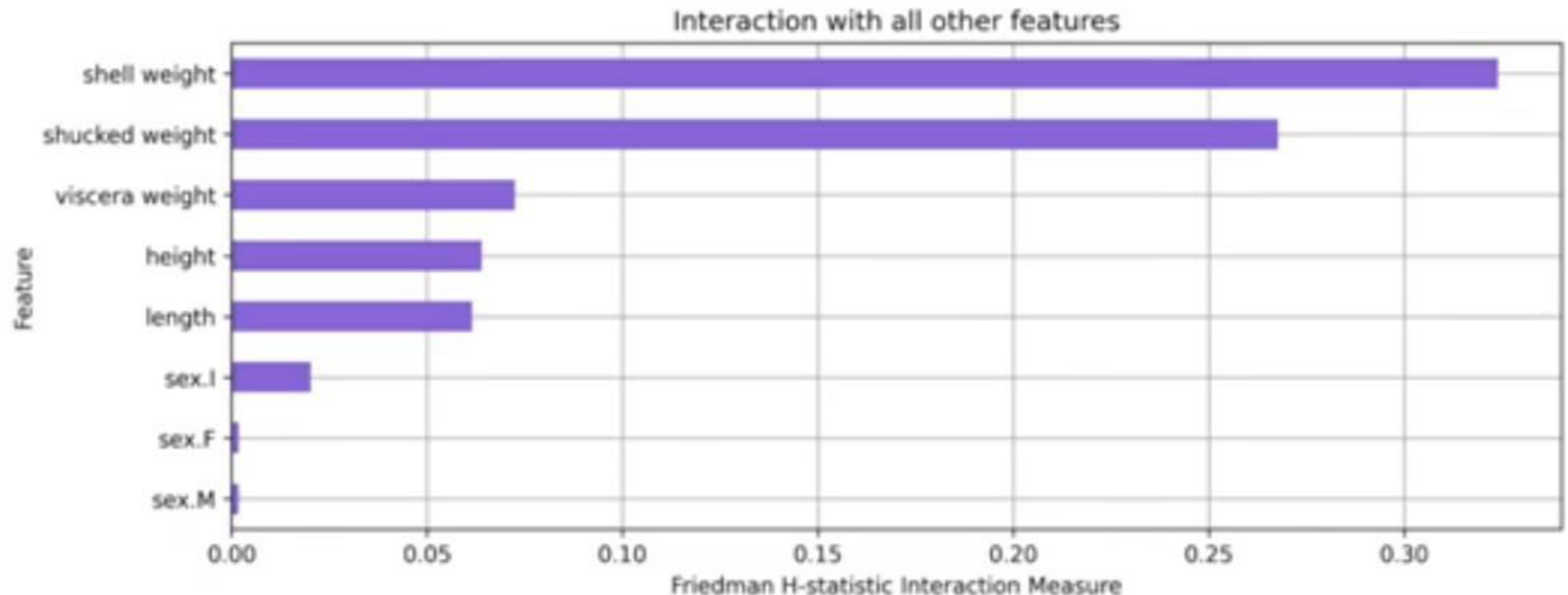


Two versions of this statistics

Overall H-stat:

$$H_j^2 = \frac{\sum_{i=1}^n [\hat{f}(X^{(i)}) - pd_j(x_j^{(i)}) - pd_{-j}(X_{-j}^{(i)})]^2}{\sum_{i=1}^n \hat{f}^2(X^{(i)})}$$

Other for
the
interaction
between
one feature
and all other
features



Friedman’s H-statistic (H-stat or H-index)) - Decomposition of Model prediction



Mathematically, the H-statistic between feature j and k is:

$$H_{jk}^2 = \sum_{i=1}^n \left[PD_{jk}(x_j^{(i)}, x_k^{(i)}) - PD_j(x_j^{(i)}) - PD_k(x_k^{(i)}) \right]^2 / \sum_{i=1}^n PD_{jk}^2(x_j^{(i)}, x_k^{(i)})$$

The same applies to measuring whether a feature j interacts with any other feature:

$$H_j^2 = \sum_{i=1}^n \left[\hat{f}(x^{(i)}) - PD_j(x_j^{(i)}) - PD_{-j}(x_{-j}^{(i)}) \right]^2 / \sum_{i=1}^n \hat{f}^2(x^{(i)})$$

Symbol	Meaning
$f(x^{(i)})$	Model prediction for the i -th data point
$PD_j(x_j)$	Partial dependence of feature j
$PD_k(x_k)$	Partial dependence of feature k
$PD_{jk}(x_j, x_k)$	Joint partial dependence of j and k together
PD_{-j}	Partial dependence of <i>all features except j</i>
n	Number of data points

These equations compute **Friedman’s H-statistic**, which answers:

“How much do features interact with each other in the model?”

If **no interaction** → the model effect is *additive*

If **strong interaction** → the effect of one feature depends on another

The value is:

0 → no interaction

Closer to 1 → strong interaction

Friedman's H-statistic (H-stat or H-index))

Normalized:

$$H_{jk}^2 = \frac{\sum_{i=1}^n [pd_{jk}(x_j^{(i)}, x_k^{(i)}) - pd_j(x_j^{(i)}) - pd_k(x_k^{(i)})]^2}{\sum_{i=1}^n pd_{jk}^2(x_j^{(i)}, x_k^{(i)})}$$

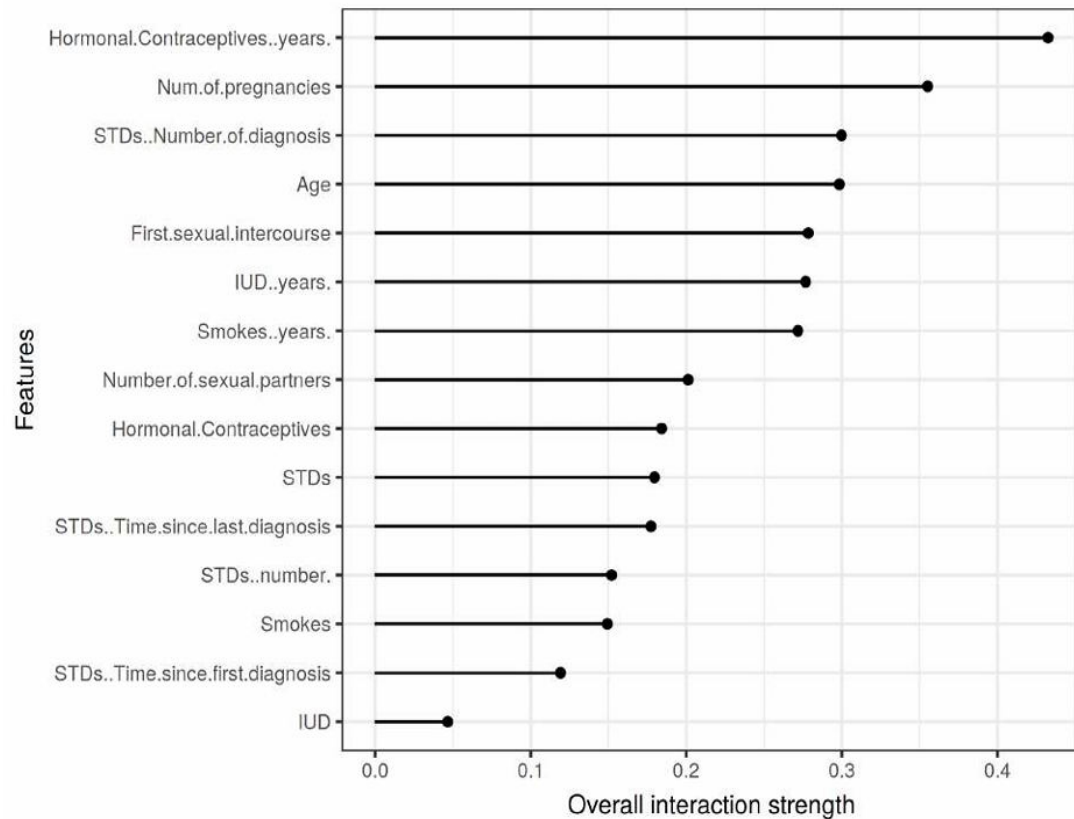
If two feature variables affect the prediction then the numerator of the above equation will be close to zero and the denominator too leading to a highly misleading H stat. To avoid this situation, a non normalized version of H stat is proposed.

Unnormalized:

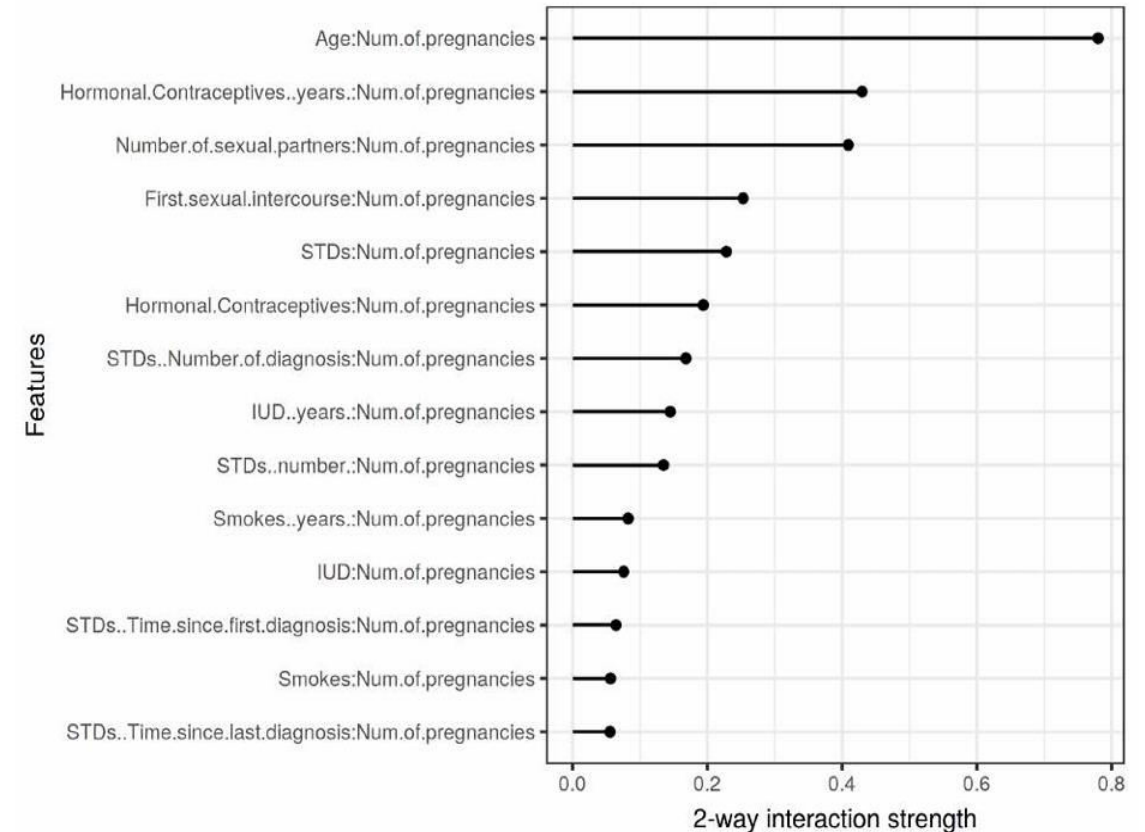
$$H_{jk}^2 = \sqrt{\sum_{i=1}^n [pd_{jk}(x_j^{(i)}, x_k^{(i)}) - pd_j(x_j^{(i)}) - pd_k(x_k^{(i)})]^2}$$

Friedman's H-statistic (H-stat or H-index) _ Explanation

The interaction statistic for a classification problem. We analyze the interactions between features in a random forest trained to predict cervical cancer, given some risk factors



The interaction strength (H-statistic) for each feature with all other features for a random forest predicting the probability of cervical cancer. The years on hormonal contraceptives has the highest relative interaction effect with all other features, followed by the number of pregnancies.



The 2-way interaction strengths (H-statistic) between number of pregnancies and each other feature. There is a strong interaction between the number of pregnancies and the age.

Feature interactions : Example by PDP extension

Let's examine the interactions between the two most important features—Parent Level of Education and student Ethnicity—in predicting grades A, B, C, and F in math. Using PDPBox, we can easily visualize pairwise feature interactions as shown in the following code snippet:

Obtains the feature interactions between two features for the learned math random forest model

```
pdp_race_parent = pdp.pdp_interact(model=math_model,
                                   dataset=df,
                                   model_features=feature_cols,
                                   features=['race_le', 'parent_le'])
```

Sets the feature column names

Uses the preloaded dataset

List of the features for which to obtain the feature interactions

```
# Parameters for the Feature Interaction plot
plot_params = {
    # plot title and subtitle
    'title': 'PDP Interaction - Math Grade',
    'subtitle': 'Race/Ethnicity (Legend): \n%s\nParent Level of Education
    (Legend): \n%s' % (race_title, parent_title),
    'title_fontsize': 15,
    'subtitle_fontsize': 12,
    # color for contour line
    'contour_color': 'white',
    'font_family': 'Arial',
    # matplotlib color map for interact plot
    'cmap': 'viridis',
    # fill alpha for interact plot
    'inter_fill_alpha': 0.8,
    # fontsize for interact plot text
    'inter_fontsize': 9,
}

# Plot feature interaction in matplotlib
fig, axes = pdp.pdp_interact_plot(pdp_race_parent, [CA] ['Race/Ethnicity',
    'Parent Level of Education'],
    plot_type='grid', plot_pdp=True,
    plot_params=plot_params)

axes['pdp_inter_ax'][0]['_pdp_x_ax'].set_xlabel('Race/Ethnicity (Grade A)')
axes['pdp_inter_ax'][1]['_pdp_x_ax'].set_xlabel('Race/Ethnicity (Grade B)')
axes['pdp_inter_ax'][2]['_pdp_x_ax'].set_xlabel('Race/Ethnicity (Grade C)')
axes['pdp_inter_ax'][3]['_pdp_x_ax'].set_xlabel('Race/Ethnicity (Grade F)')
axes['pdp_inter_ax'][0]['_pdp_x_ax'].grid(False)
```

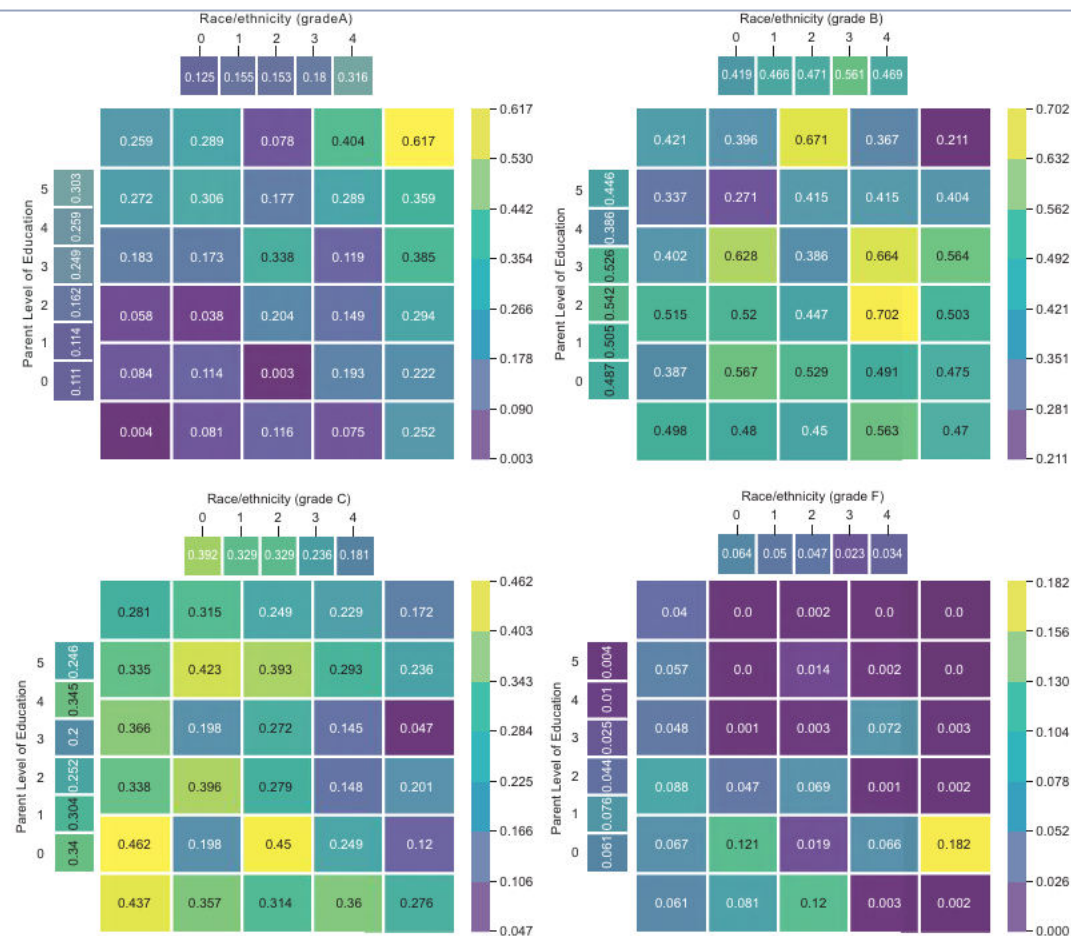


Figure 3.13 Interaction between Parent Level of Education and Ethnicity for all math grades A, B, C, and F

Feature interactions : Example by PDP extension

- Shows the feature interaction plot for math grade A. Parent Level of Education is on the y-axis, and the anonymized Ethnicity of the student is on the x-axis. As you go from the bottom to the top of the y-axis, Parent Level of Education increases from high school all the way to a master's degree. High school education is represented by a value of 0, and a master's degree is represented by a value of 5. The x-axis shows the five distinct ethnicity groups—A, B, C, D, and E.
- Ethnicity group A is represented by a value of 0, group B is represented by a value of 1, group C by a value of 2, and so on. The number in each cell represents the impact of a given Parent Level of Education and student Ethnicity on getting grade A.

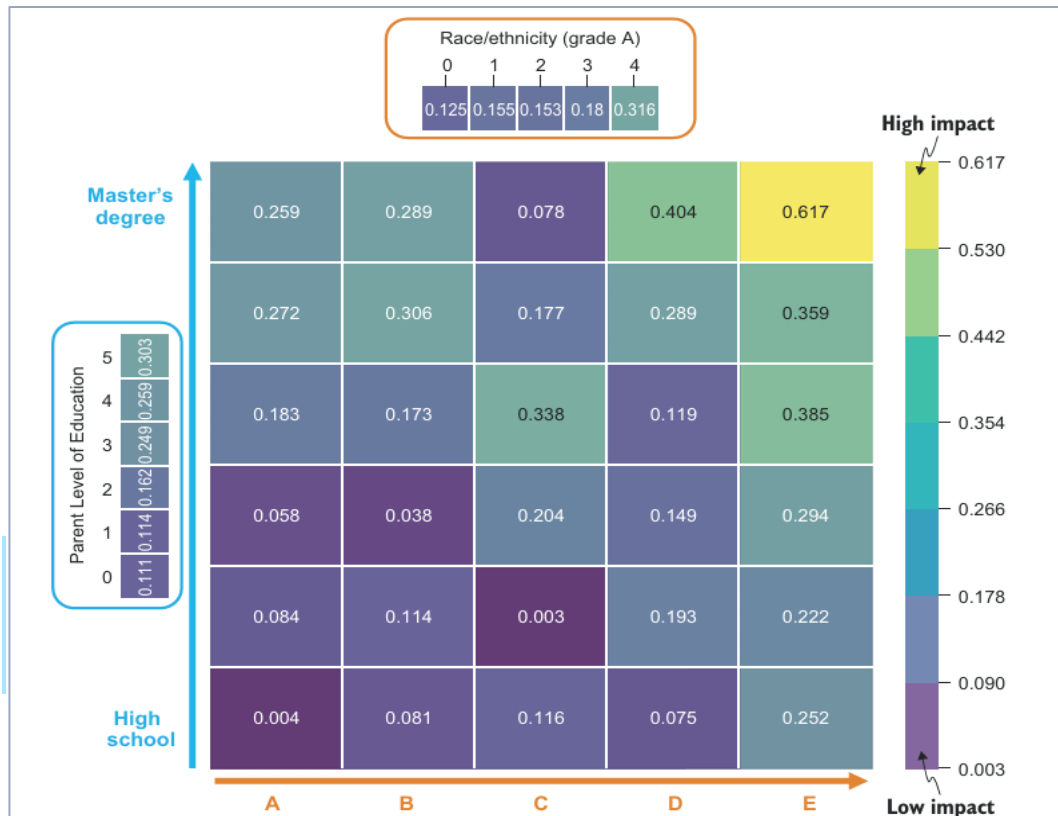


Figure 3.14 Zooming in on math grade A and decomposing the feature interaction plot

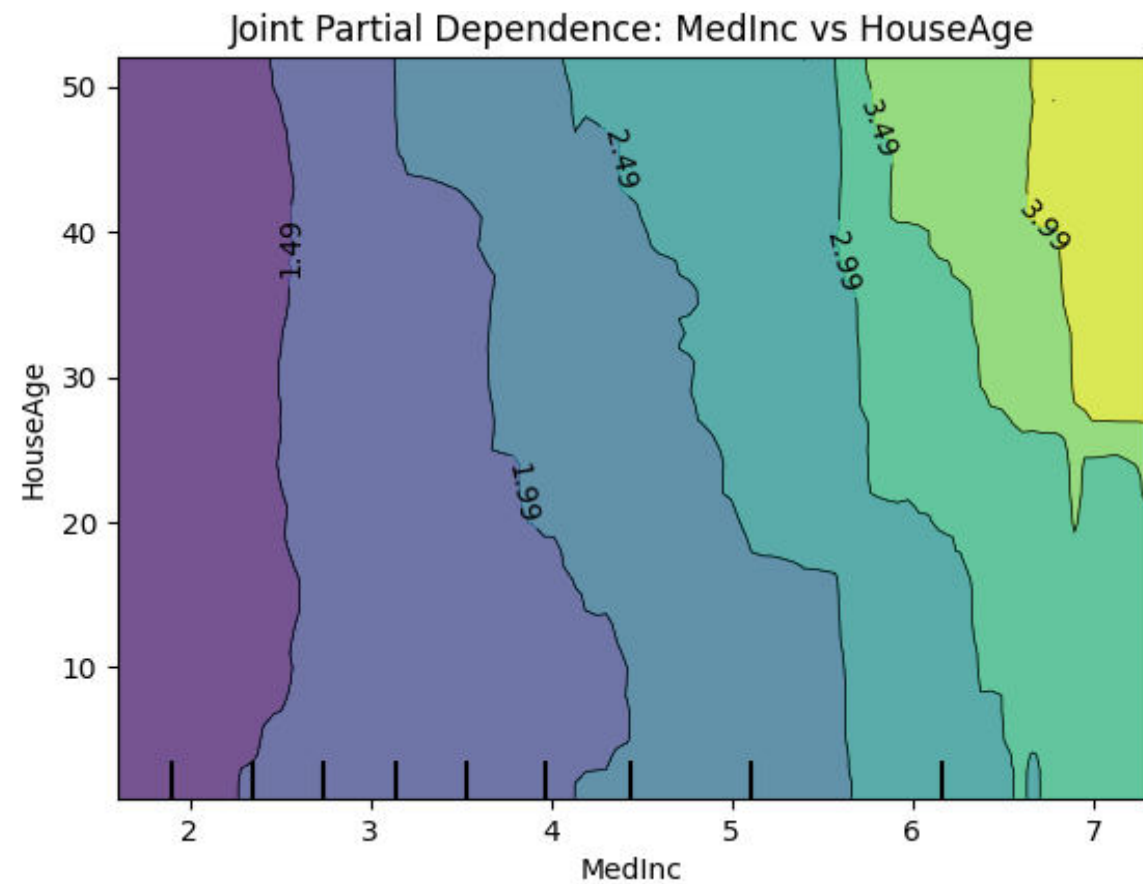
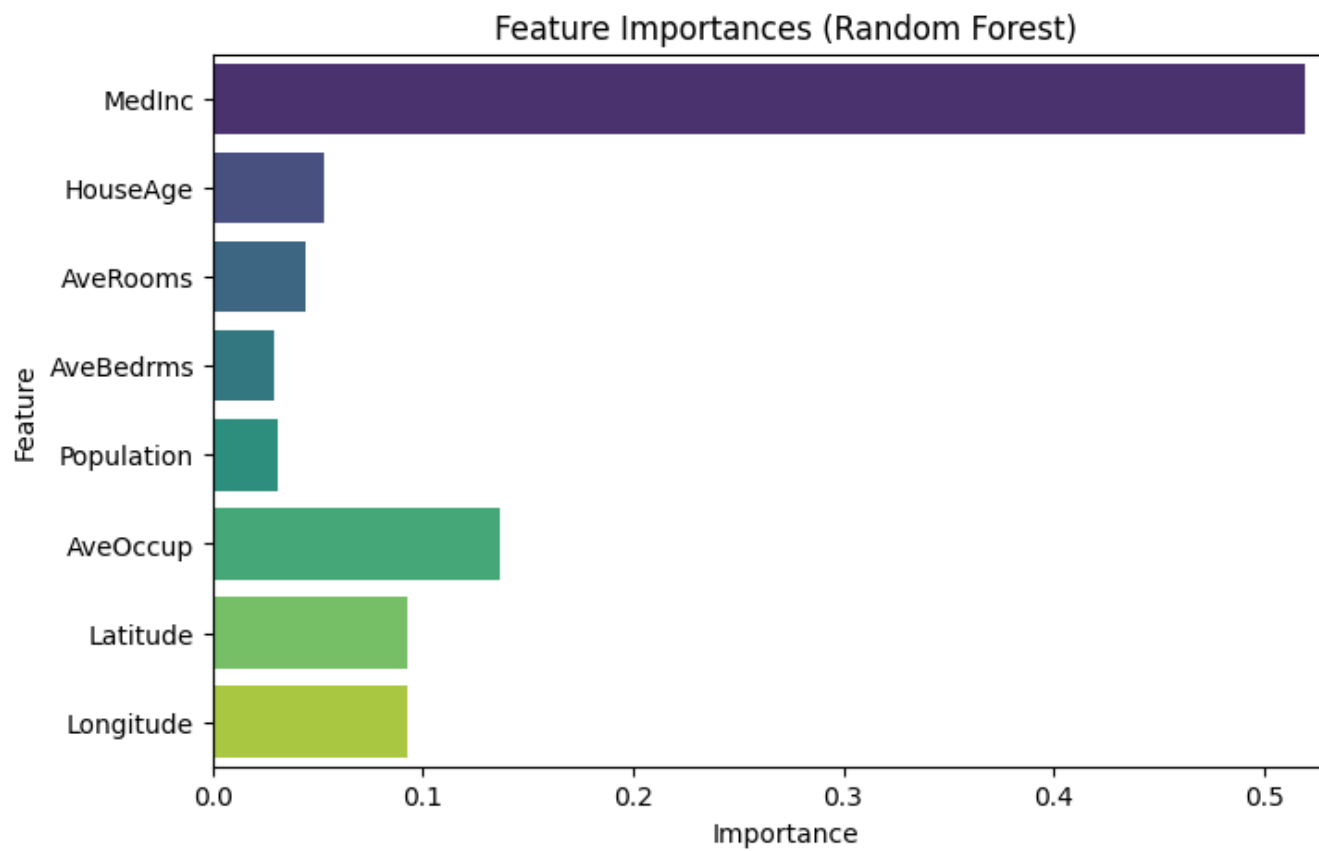
For instance, the cell in the bottom-most row and the left-most column represents the average impact of a student belonging to ethnicity group A and having a parent who has a high school education on getting grade A. Please also note the numerical values in each cell of the grid—a lower number represents lower impact, and a higher number represents a higher impact in predicting grade A.

Friedman's H-statistic (H-stat or H-index))

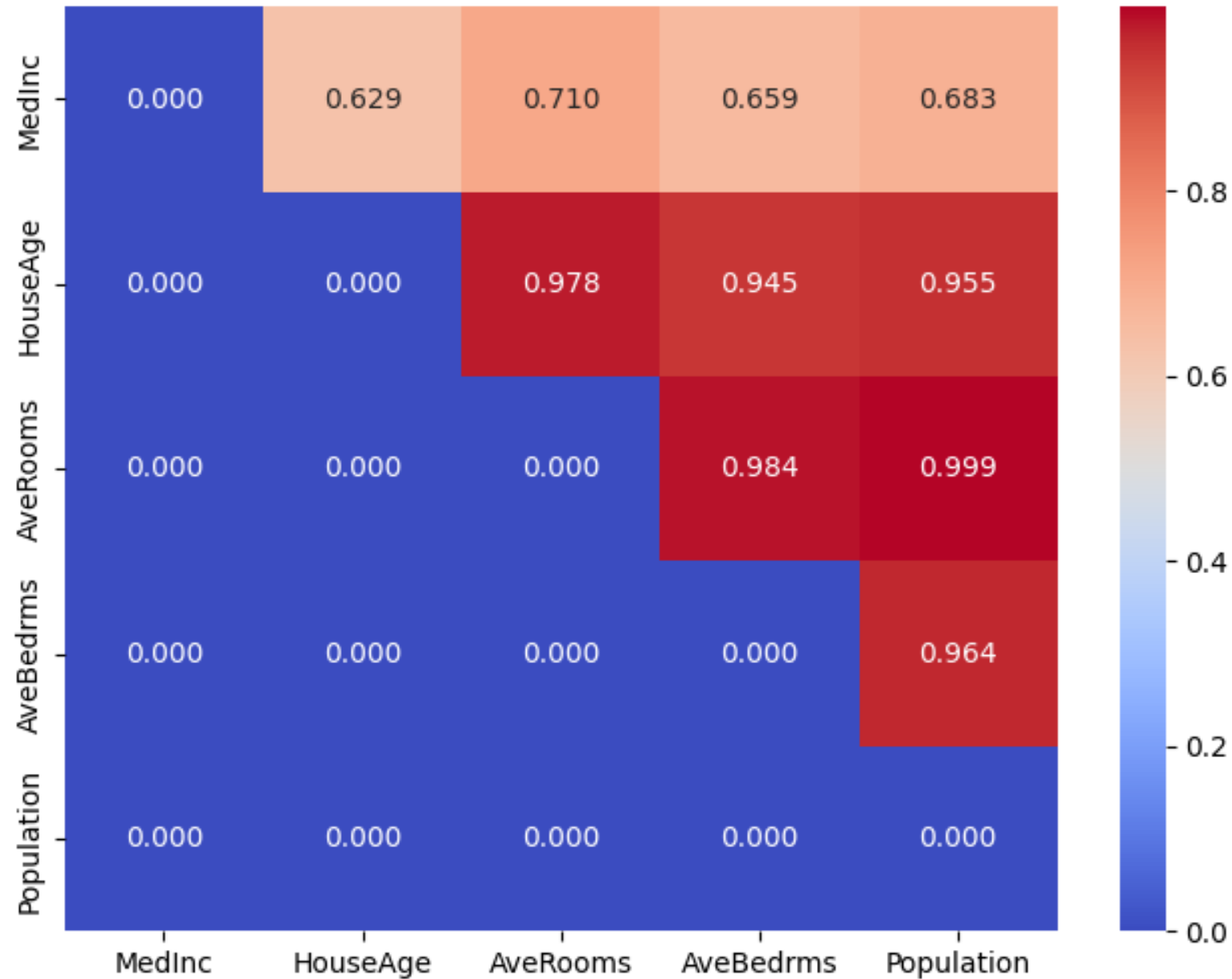
Limitations of H stats

- Computationally expensive
- Complexity
- Impractical to calculate for a fairly large dataset.
- Exaggerated interactions
- This does not give idea about how much of interaction has actually affected the prediction.
- Spurious interactions – when interactions are highly correlated.





Pairwise H-statistics (Top 5 Features)

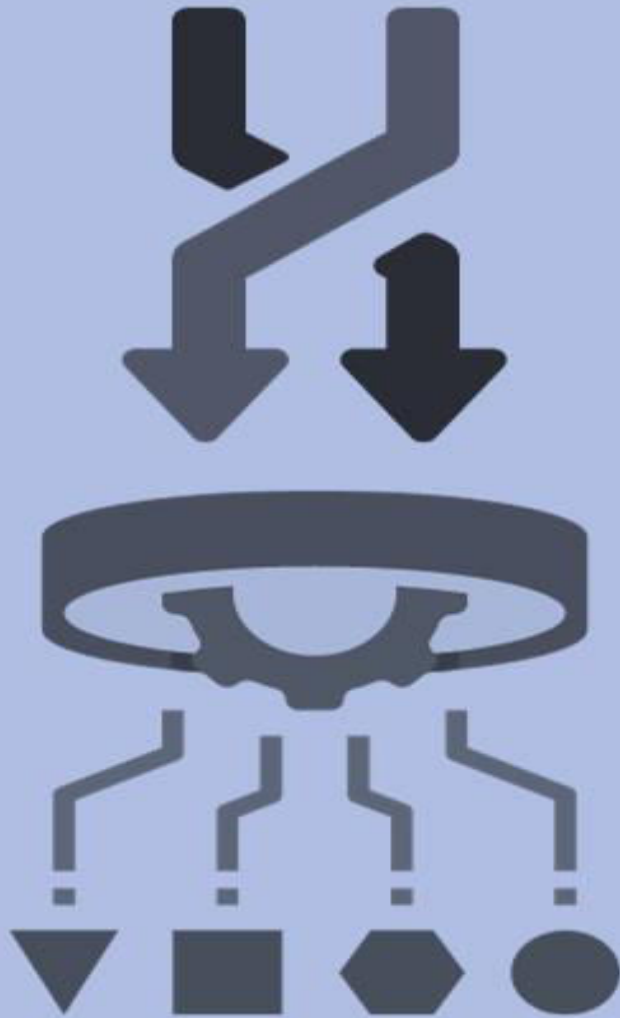


Friedman's H-statistic (H-stat or H-index))

Hands on practice

Analysing Interactions with Friedman's H-stat and Python

<https://medium.com/data-science/analysing-interactions-with-friedmans-h-stat-and-python-b0aa9beeb7a3>



UNIT 2:

Permutation Feature Importance (PFI)

Permutation Feature Importance:

What is Permutation Feature Importance?

- A Model-Agnostic Technique for Interpretable Machine Learning
- A method to measure the contribution of each feature to a trained model's predictive performance. 
- It quantifies importance by measuring how much the model's error increases when the values of a feature are randomly shuffled.

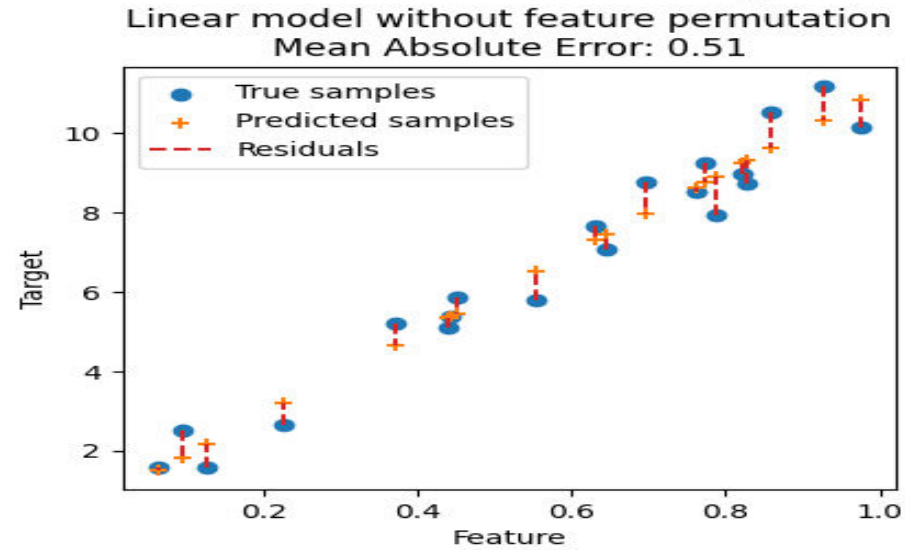
Core Idea: If shuffling a feature's values ruins the model's prediction accuracy, the feature is important.

Why Use Permutation Importance??

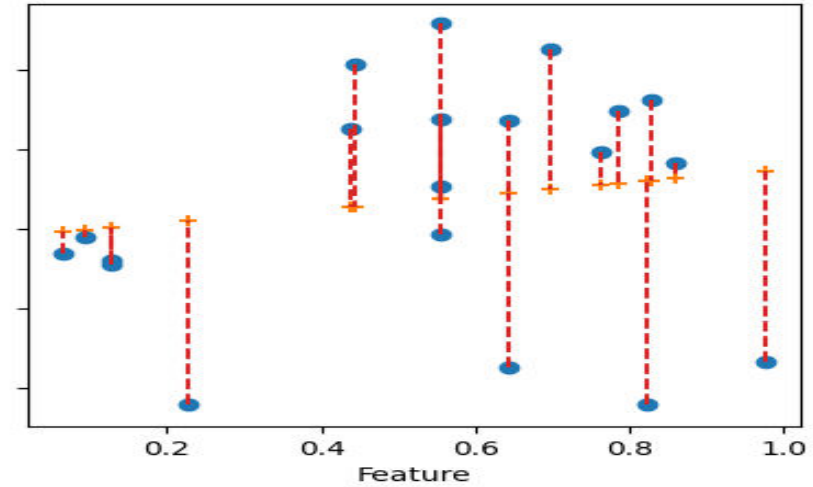
- Model-Agnostic: Can be applied to any fitted estimator (Random Forest, SVM, Neural Networks, etc.).
- Global Interpretation: Provides an overview of which features are globally important.
- Reliable: Does not suffer from the high-cardinality bias inherent in impurity-based (MDI) feature importance, particularly in tree-based models.

Feature Importance:

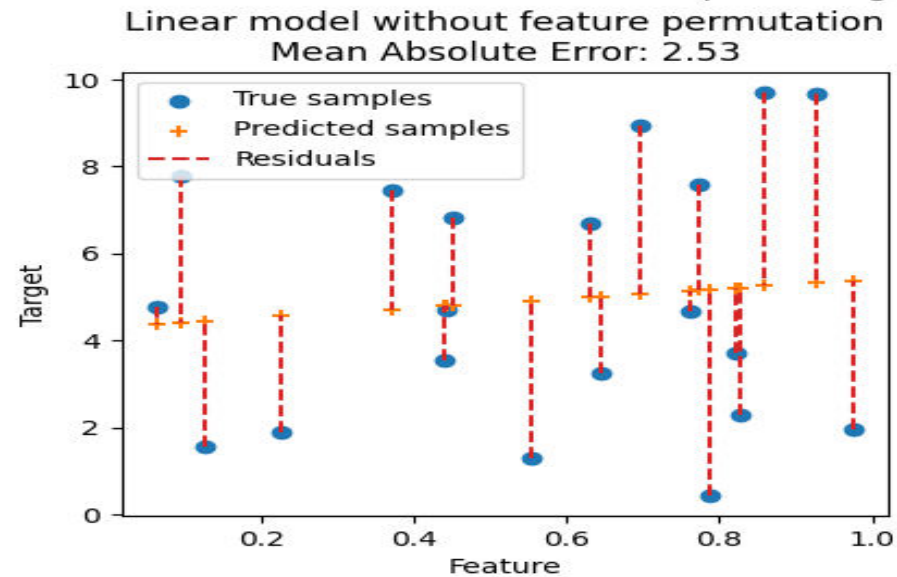
Effect of permuting a predictive feature



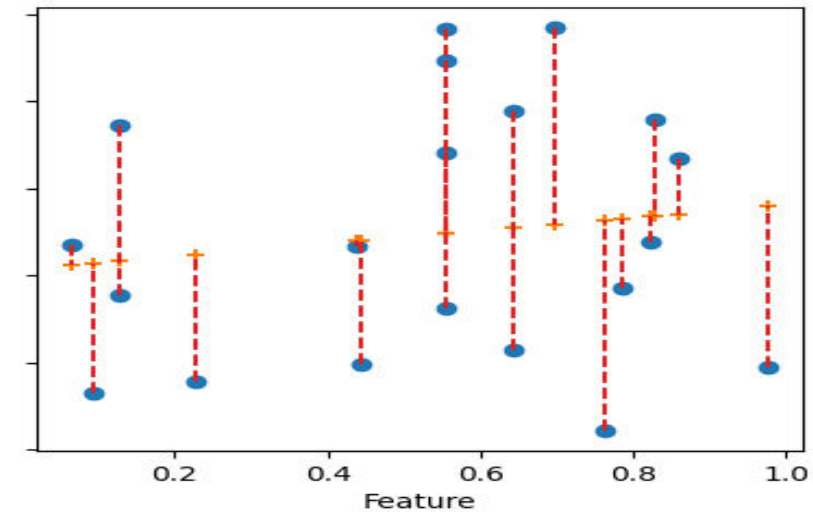
Linear model with feature permutation
Mean Absolute Error: 2.28



Effect of permuting a non-predictive feature



Linear model with feature permutation
Mean Absolute Error: 2.49



PFI Algorithm:

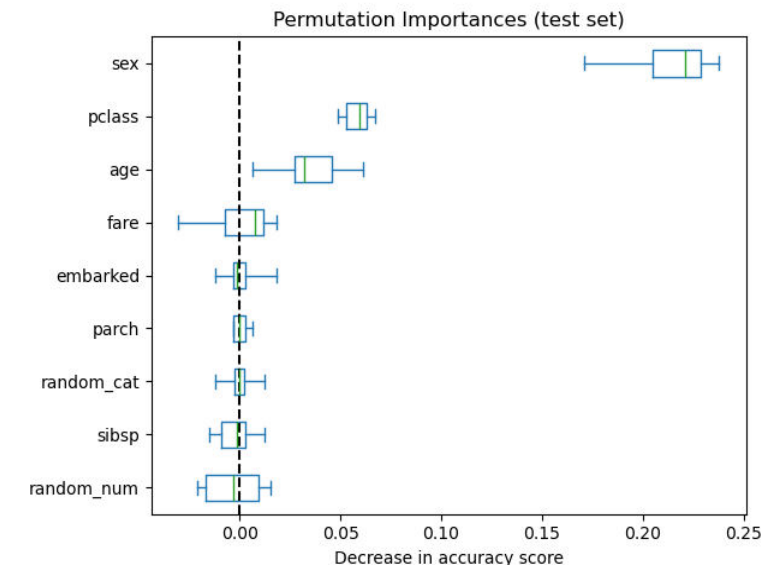
The permutation feature importance algorithm based on Fisher, Rudin, and Dominici (2018):

Input: Trained model f , feature matrix X , target vector y , error measure $L(y, f)$.

1. Estimate the original model error $e^{\text{orig}} = L(y, f(X))$ (e.g. mean squared error)
2. For each feature $j = 1, \dots, p$ do:
 - Generate feature matrix X^{perm} by permuting feature j in the data X . This breaks the association between feature j and true outcome y .
 - Estimate error $e^{\text{perm}} = L(Y, f(X^{\text{perm}}))$ based on the predictions of the permuted data.
 - Calculate permutation feature importance $FI^j = e^{\text{perm}} / e^{\text{orig}}$. Alternatively, the difference can be used: $FI^j = e^{\text{perm}} - e^{\text{orig}}$
3. Sort features by descending FI.

Visualizing PFI Results:

- Typically shown as a bar chart.
- Features are listed in order from highest influence (top) to lowest (bottom).
- The bar length represents the decrease in model performance (higher = more important).



Permutation Feature Importance: Advantages & Limitations

Advantages & Best Practices:

- **Intuitive:** Easy to understand and explain.
- **Use Validation Data:** Always use a held-out test set to avoid overestimating importance.
- **Repeated Permutations:** Shuffling multiple times and averaging the results gives a more reliable estimate.

Permutation Feature Importance is a versatile, model-agnostic, and reliable tool for interpreting machine learning models by measuring performance degradation upon feature perturbation

Limitations (Important!)

- **Correlated Features:** If two features are highly correlated, permuting one might not degrade performance much because the model can still use the other, leading to an underestimation of both features' importance.
- **Unrealistic Data:** Permuting breaks joint distributions, potentially creating data combinations the model has never seen, which can skew results.

Workflow: Train → Measure Base Performance → Shuffle Column → Measure New Performance → Calculate Difference.

PFI Example using Python:

- There are three open source Python libraries that support permutation feature importance: Scikit-learn, Eli5, and Feature-engine.

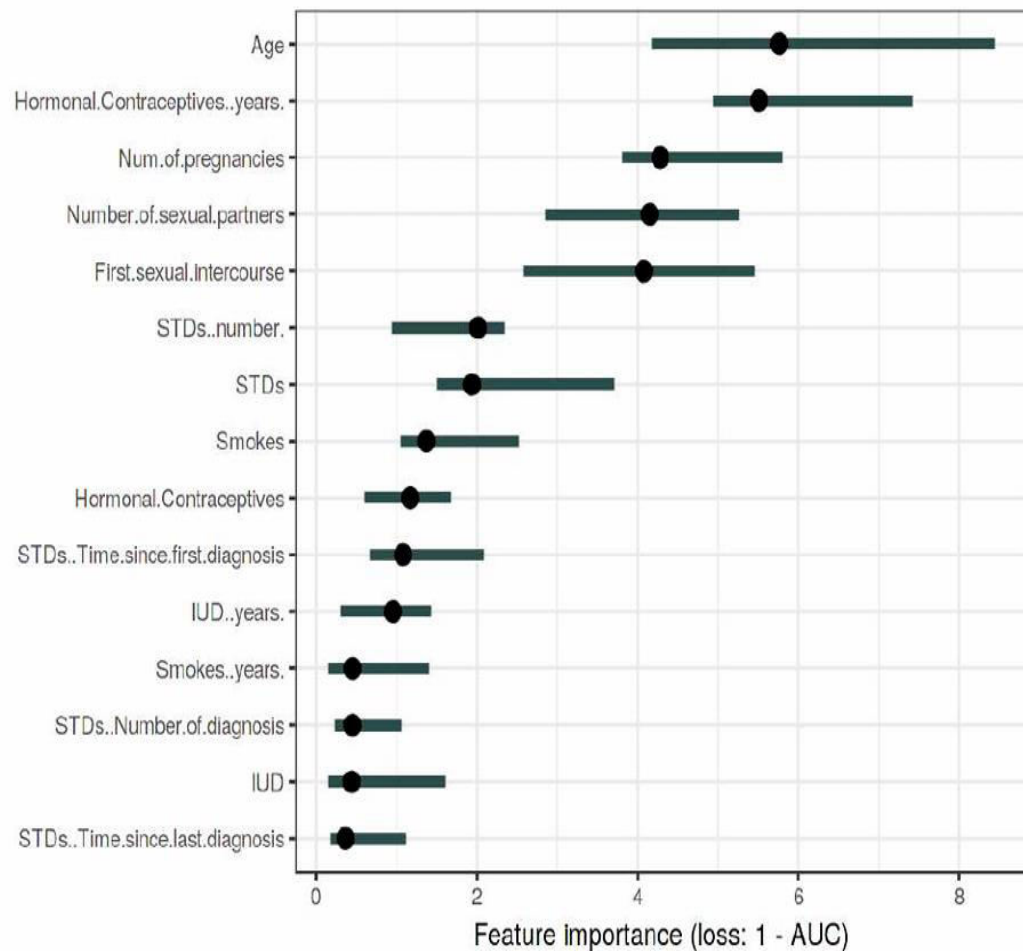
```
from sklearn.inspection import permutation_importance
```

- Scikit-learn and ELI5 obtain variable importance by shuffling the same feature multiple times and then averaging the results.
- In contrast, Feature-engine utilizes cross-validation.
- It takes a portion of the training set to train the model and evaluates feature importance on a held-out sample, averaging the drop in performance across multiple held-out samples.

[Understanding Permutation Feature Importance for Model Interpretation - Train in Data's Blog](#)

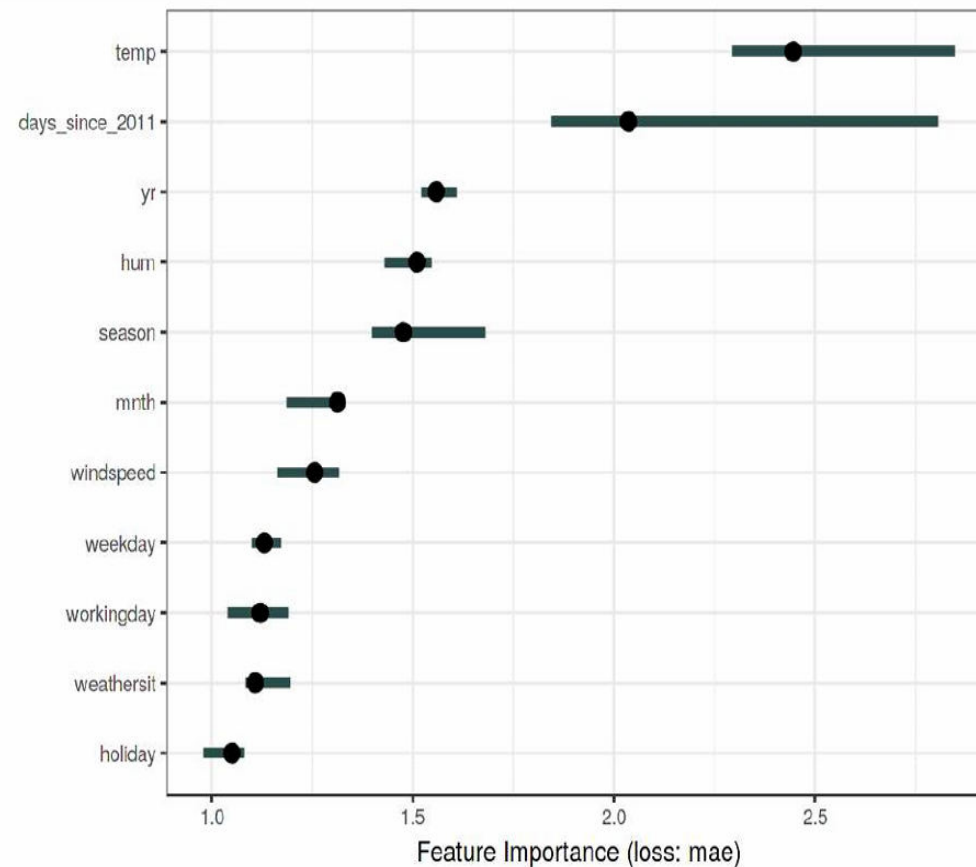
Permutation Feature Importance: Classification and Regression Plots

Classification:

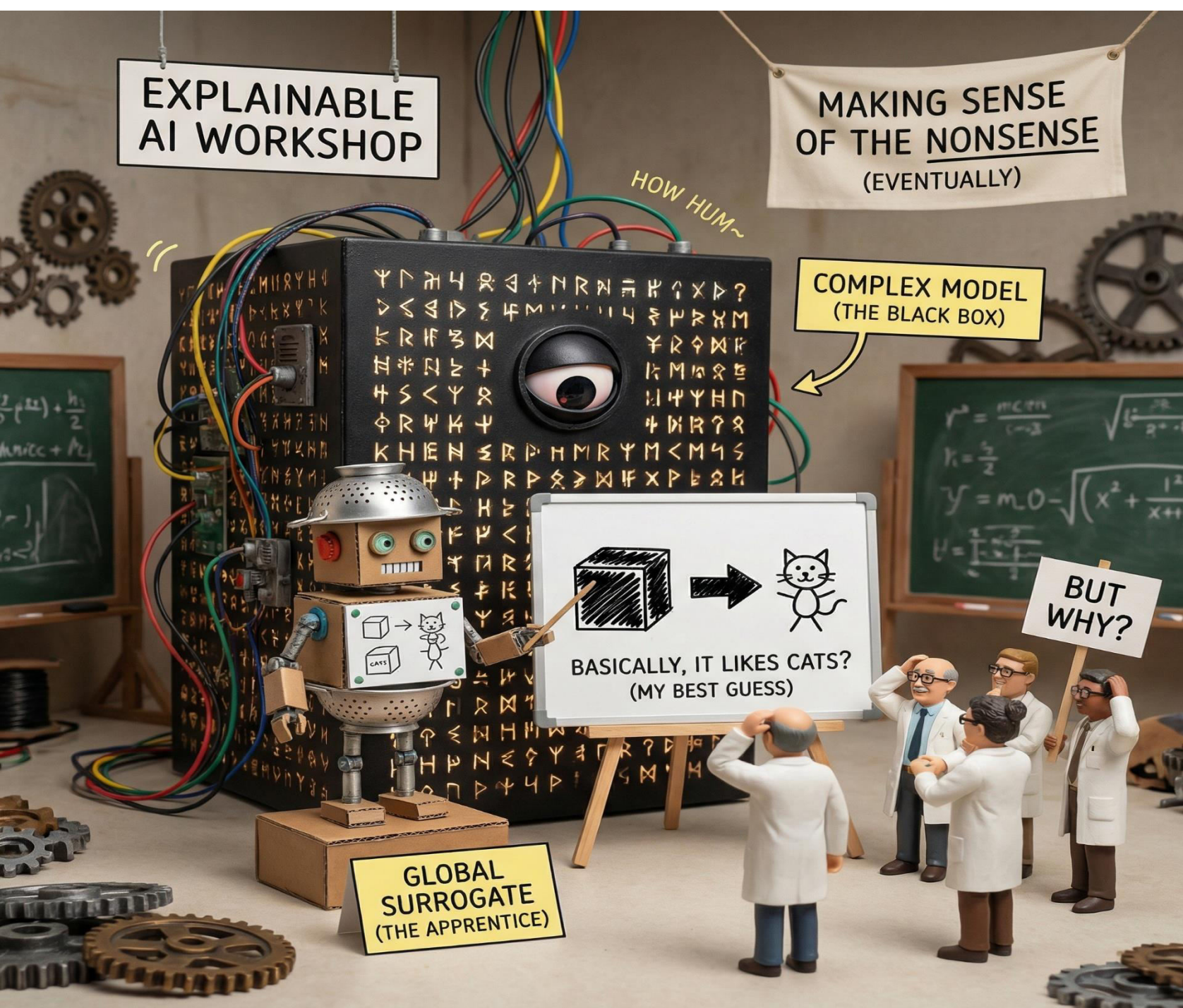


The importance of each of the features for predicting cervical cancer with a random forest. The most important feature was Age. Permuting Age resulted in an increase in 1-AUC by a factor of 5.76

Regression



The importance for each of the features in predicting bike counts with a support vector machine. The most important feature was temp, the least important was holiday.



UNIT 2:

Global Surrogate

Global Surrogate:

What is Global Surrogate?

- A simple, interpretable model (S) trained to approximate the predictions of a complex, black-box model (B).
- **Goal:** To understand the global logic of the black-box model rather than explaining a single prediction.
- Eg : Decision Trees, Linear Regression, Rule-based models.
- ***Solving machine learning interpretability by using more machine learning!***

One way to measure how well the surrogate replicates the black box model is the R-squared measure:

$$R^2 = 1 - \frac{SSE}{SST} = 1 - \frac{\sum_{i=1}^n (\hat{y}_*^{(i)} - \hat{y}^{(i)})^2}{\sum_{i=1}^n (\hat{y}^{(i)} - \bar{\hat{y}})^2}$$

Steps to obtain a surrogate model:

1. Select a dataset X. This can be the same dataset that was used for training the black box model or a new dataset from the same distribution. You could even select a subset of the data or a grid of points, depending on your application.
2. For the selected dataset X, get the predictions of the black box model.
3. Select an interpretable model type (linear model, decision tree, ...).
4. Train the interpretable model on the dataset X and its predictions.
5. You now have a surrogate model.
6. Measure how well the surrogate model replicates the predictions of the black box model.
7. Interpret the surrogate model

Global Surrogate - Example

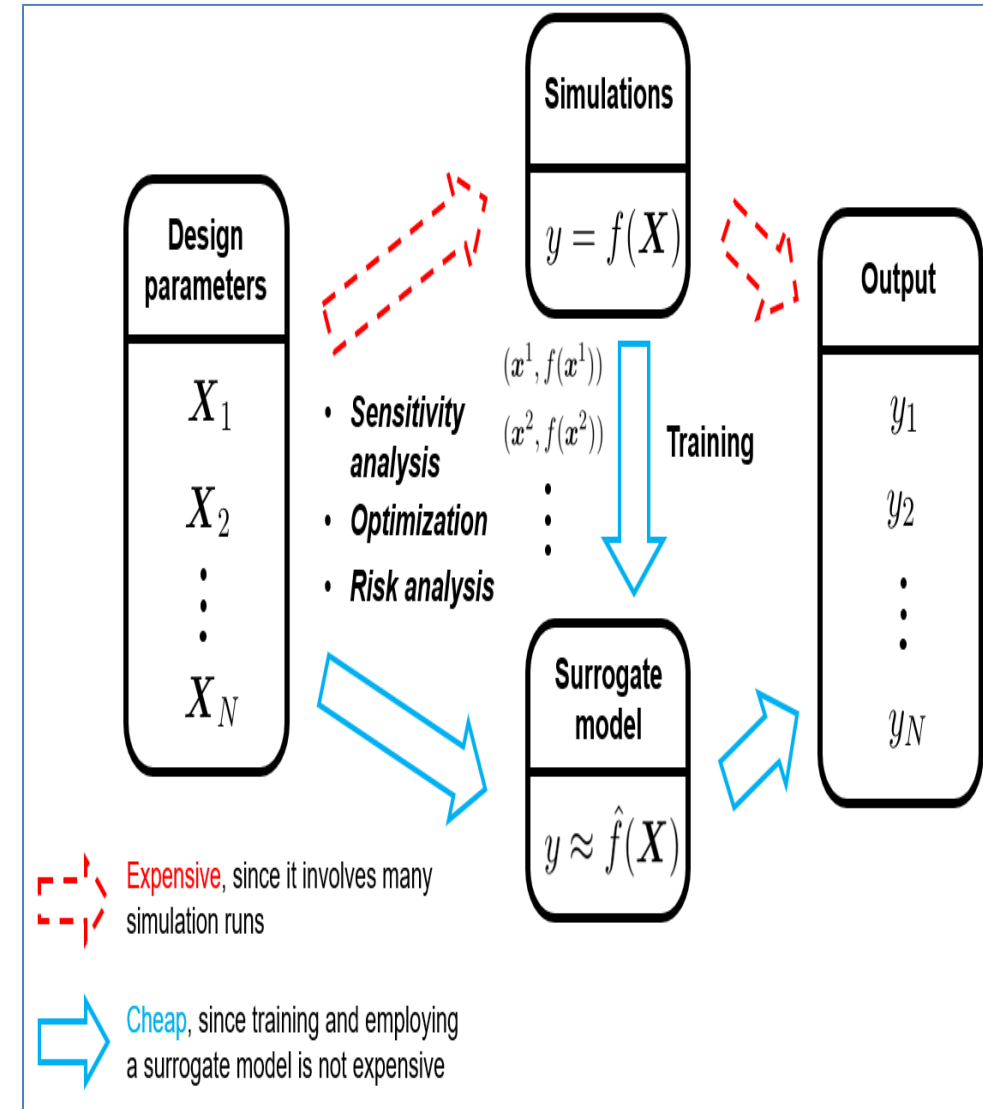
→ Surrogate

Tree-based Surrogate for Complex Models: A CART (Classification and Regression Tree) is trained on the input data and the predicted output of a Support Vector Machine (SVM) to interpret which features (e.g., weather and day in a bike rental dataset) dominate the prediction, [FirmAI](#).

Linear Model for Complex Models: A Logistic Regression model is used to approximate the predictions of a, Gradient Boosting classifier in a healthcare dataset [Towards Data Science](#).

Engineering Simulation Surrogate: In aerospace, a, Kriging model or Response Surface Method (RSM) serves as a fast, cheap substitute for a computationally expensive, high-fidelity, CFD (Computational Fluid Dynamics) model for turbine blade, thermal analysis [ResearchGate](#).

Auto Insurance Pricing: A Generalized Linear Model (GLM) is used as a surrogate to explain a complex, non-linear machine learning model, [SOA](#)



THANK YOU



UE23AM342BA3
Explainable AI

Local model agnostic Methods

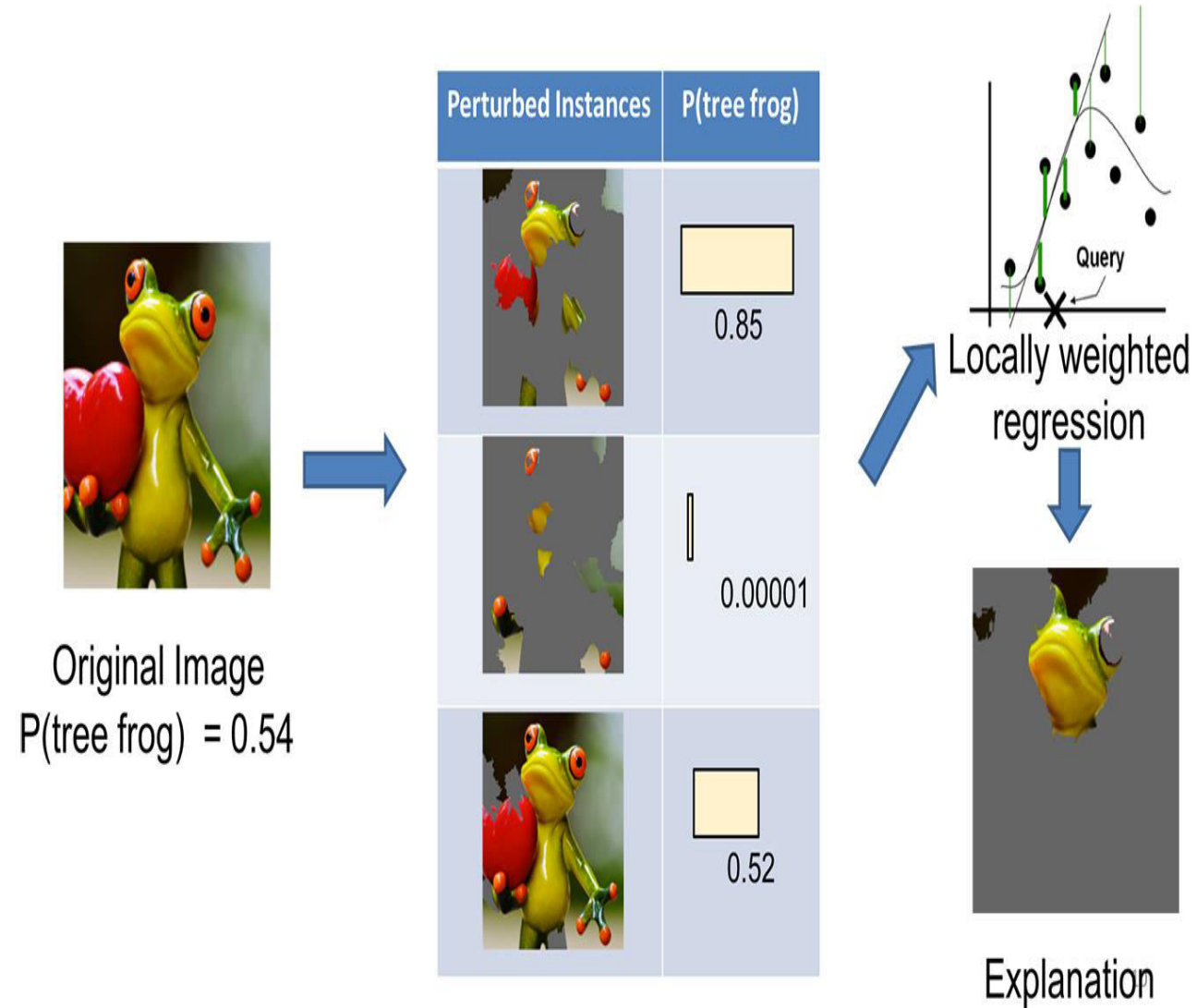
Prof. Ayisha Noori VK, Dr. Siba Ram Baral
Dept. of CSE(AI&ML), PES University,
Bengaluru

Contents:

- ICE plot
- LIME

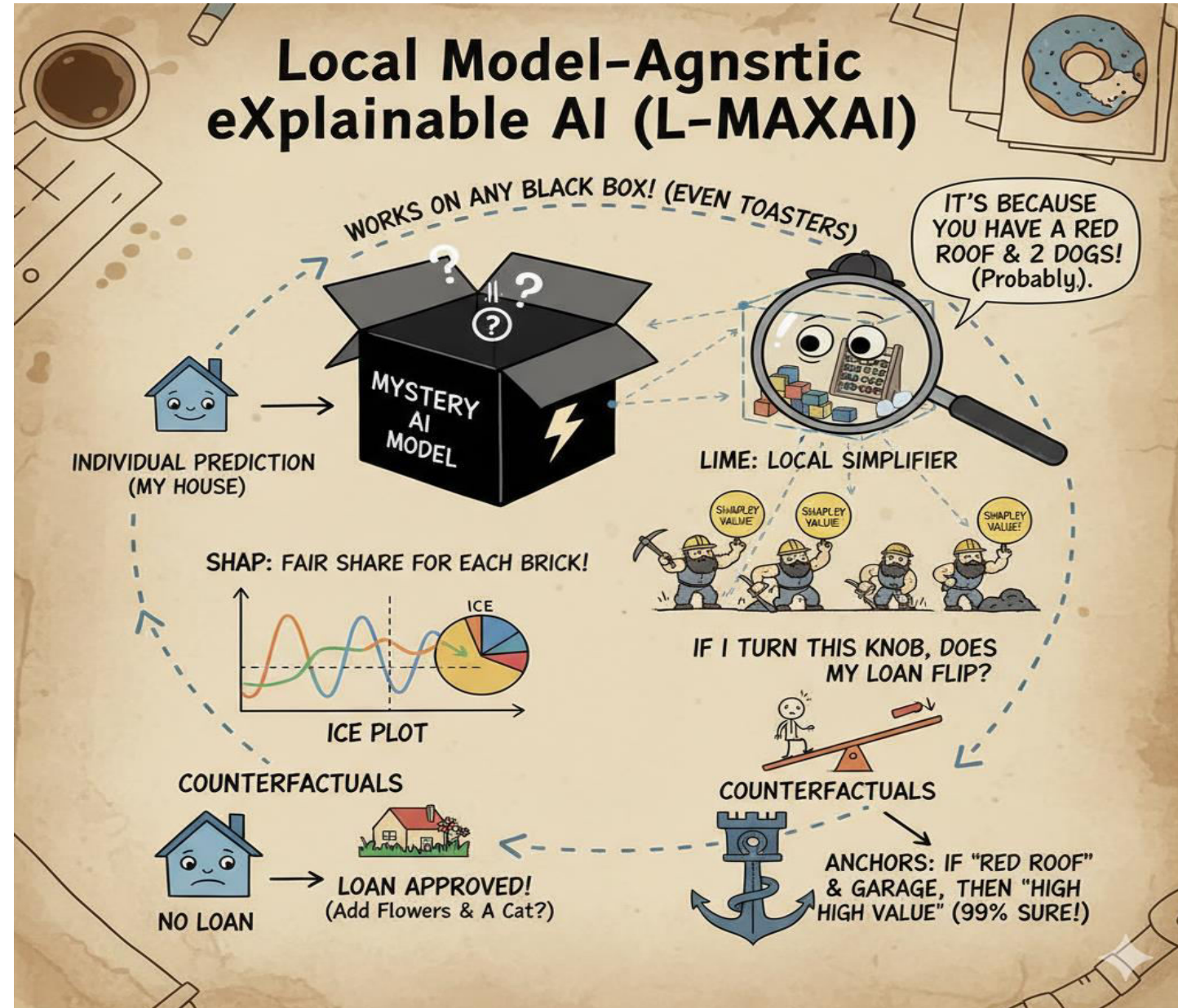
Local model agnostic Methods

- Local model-agnostic methods are techniques used to explain individual predictions of any machine learning model, regardless of its internal architecture.
- By treating the model as a "black box," these methods provide flexibility, allowing you to use the same interpretability tool even if you switch the underlying model
- ICE , LIME, SHAP, Counter Factual Explanations, Anchors



Local model agnostic Methods

- ICE
- LIME
- SHAP
- Counter Factual Explanations
- Anchors



Individual Conditional Expectations(ICE Plot)

Not Your Freezer's ICE! 

How your ML model treats each individual differently!

"In Machine Learning, as in life, averages hide the interesting stories!"



Individual Conditional Expectations(ICE Plot)

The Problem

Your model is a black box. You feed it data, it spits out predictions. But WHY?

Translation: "It works, but I have no idea how... 🤔"

The Solution

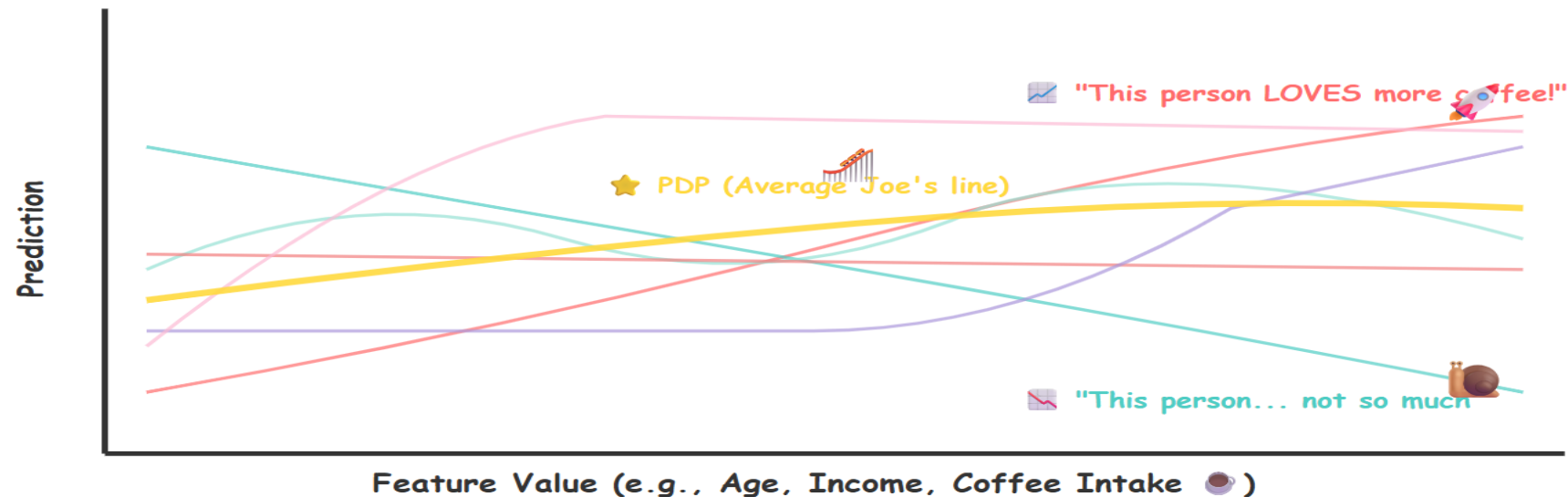
ICE plots show you how predictions change for EACH individual as you vary one feature!

Translation: "Now I can see who my model is discriminating against!" (in a good, analytical way)

The Insight

Each line = one data point's journey through different feature values

Translation: "Everyone gets their own line! Democracy! 🗳️"



Each colored line represents ONE individual's prediction journey

Individual Conditional Expectations(ICE Plot)

- Individual Conditional Expectation (ICE) plots are a powerful machine learning interpretability tool used to visualize the relationship between a single feature and the target prediction for each individual observation.
- Unlike Partial Dependence Plots (PDP) that show the average effect, ICE plots display one line per sample, allowing for the detection of interactions and heterogeneous effects

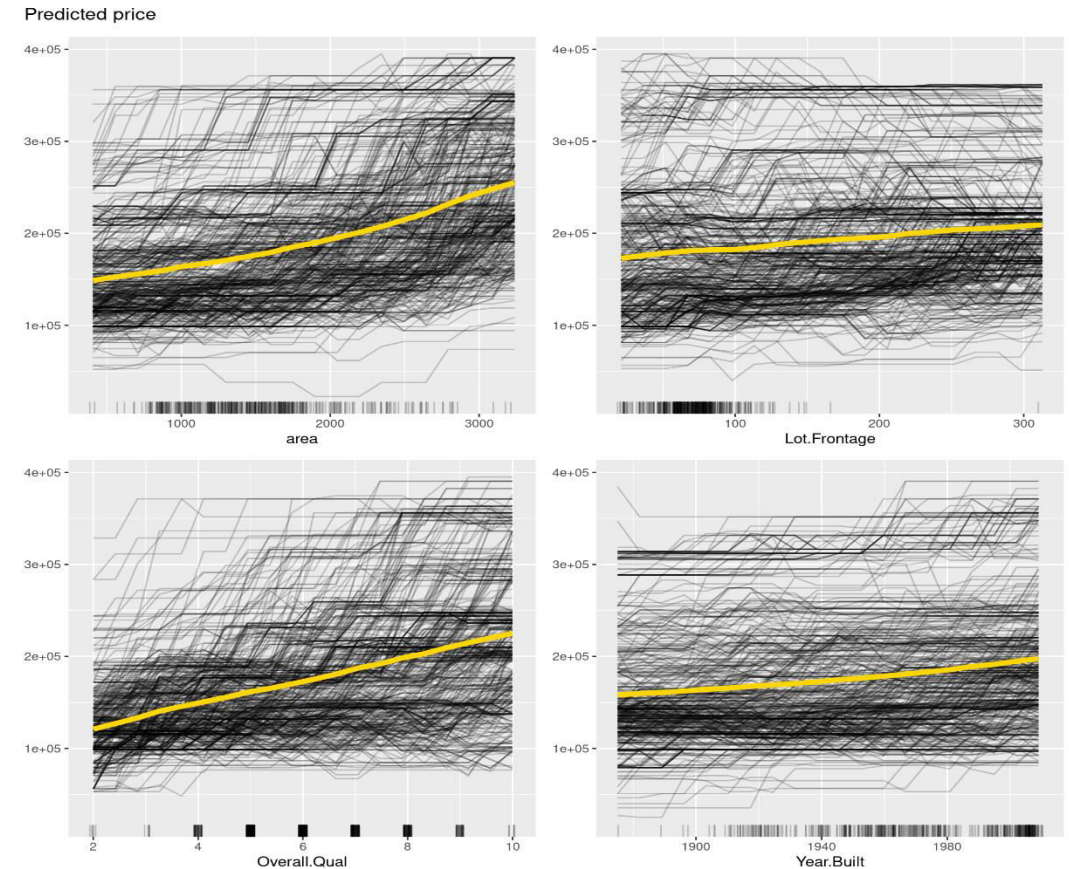
Why Use ICE Plots?

- Uncover heterogeneous relationships (e.g., if a feature has a positive effect for some, but negative for others).
- Reveal interaction effects

How to Interpret:

- Parallel lines indicate no interaction.
- Converging/diverging lines indicate interaction effects.

ICE vs. PDP (Visual Comparison)



Shows an ICE plot (multiple black lines) with a PDP plot (one thick yellow average line).

ICE plot – Case Study: California Housing Data

In a typical housing price prediction model, a PDP might show that as "Median Income" increases, house prices generally rise. However, an ICE plot can reveal more nuance

The Global Trend: Most individual lines follow the upward trend, confirming a positive relationship. ✓

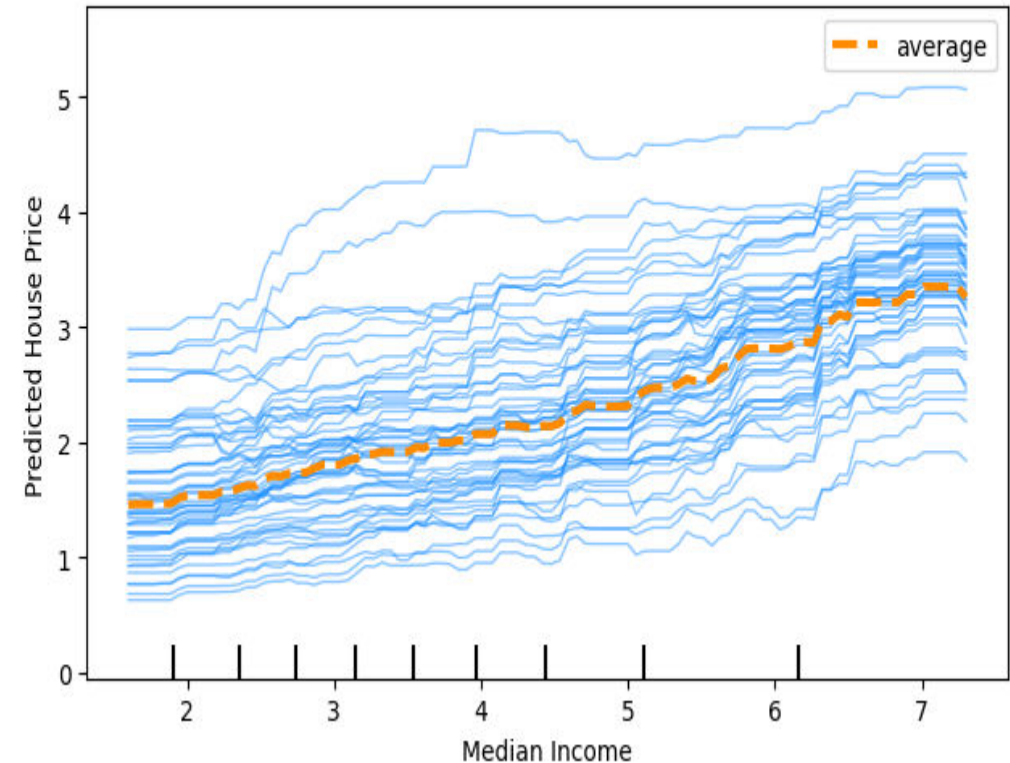
The Exceptions: You might find specific "plateau" lines—these represent instances where increasing income doesn't raise the predicted price, perhaps because the house is in a remote area or has poor amenities. ✓

Interaction Detection: If some lines are steep and others are flat, it suggests an interaction between the feature being plotted and other features in your dataset.

The easiest way to implement ICE plots is using [Scikit-learn's PartialDependenceDisplay](#)

ICE vs. PDP (Visual Comparison)

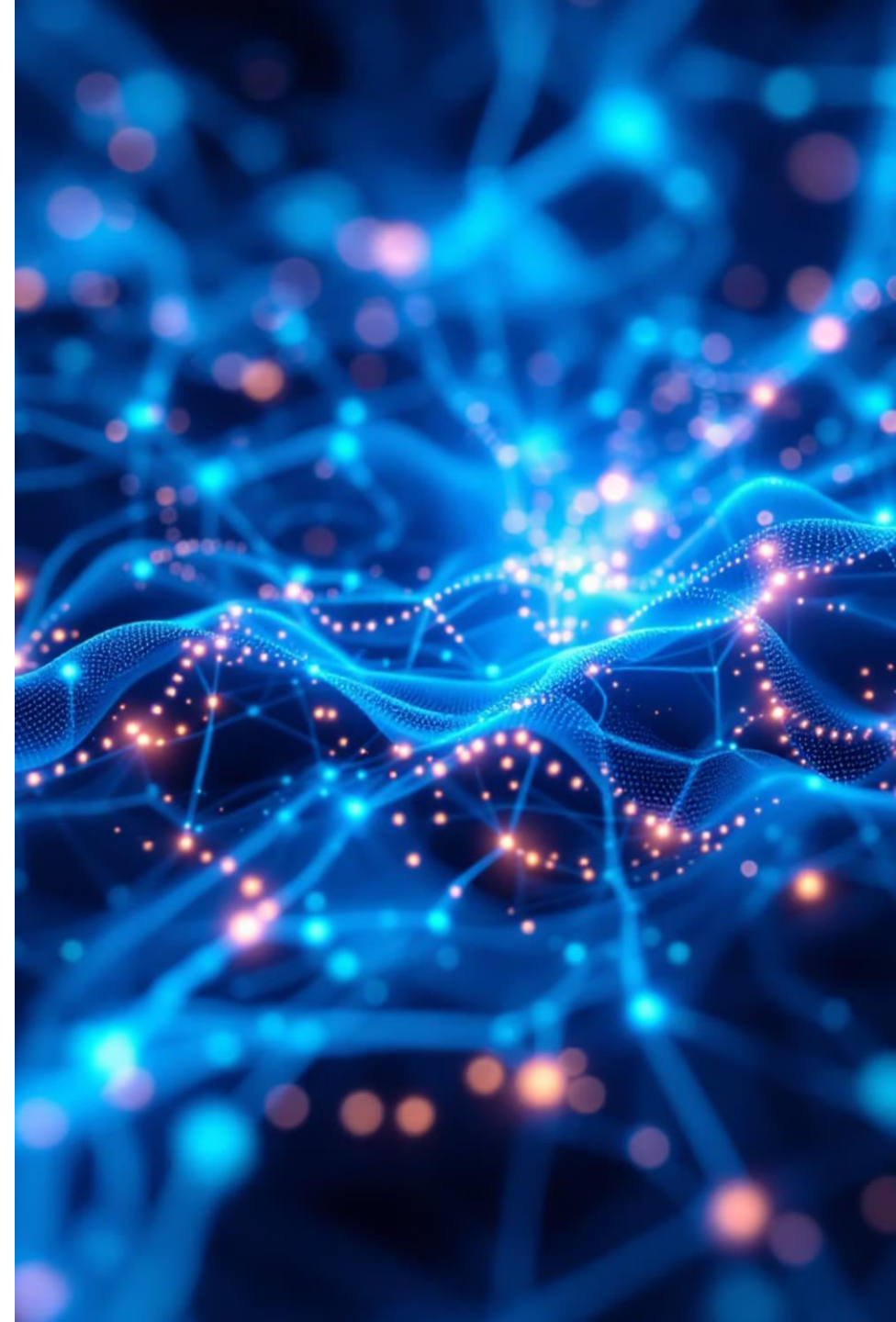
ICE Plot for Median Income (MedInc)



Shows an ICE plot (multiple blue lines) with a PDP plot (one thick orange average line).

LIME – Local Interpretable Model-Agnostic Explanations

A practical guide to understanding and implementing Local Interpretable Model-Agnostic Explanations across multiple data types Using Tabular, Text, and Image Data in Python



What Is LIME?

The Fundamentals

LIME stands for **Local Interpretable Model-Agnostic Explanations**. It's a powerful technique that explains predictions of any black-box machine learning model, regardless of its complexity or architecture.

The core innovation?

LIME approximates the model locally around a specific prediction using a simple, interpretable model like linear regression. This allows us to understand individual predictions without knowing the model's internal mechanics.

Key Capabilities

- **Model-agnostic:** Works with any classifier or regressor ✓
- **Local explanations:** Focuses on individual predictions ✓
- **Interpretable output:** Uses human-readable features ✓
- **Universal applicability:** Handles tabular, text, and image data ✓

- LIME tests what happens to the predictions when you give variations of your data into the machine learning model.
- LIME generates a new dataset consisting of permuted samples and the corresponding predictions of the black box model.
- On this new dataset LIME then trains an interpretable model, which is weighted by the proximity of the sampled instances to the instance of interest.
- The interpretable model can be any whitebox models, for example Lasso or a decision tree. The learned model should be a good approximation of the machine learning model predictions locally, but it does not have to be a good global approximation.
- This kind of accuracy is also called **local fidelity**. Mathematically, local surrogate models with interpretability constraint can be expressed as follows:

$$\text{explanation}(x) = \arg \min_{g \in G} L(f, g, \pi_x) + \Omega(g)$$

How LIME Works (Step-by-Step)

Select Instance: Choose the specific data instance (e.g., a single image or text document) for which you want an explanation.

Perturb Data: Create new, synthetic data points by slightly modifying (perturbing) the original input (e.g., hiding words in a text or turning off pixels in an image).

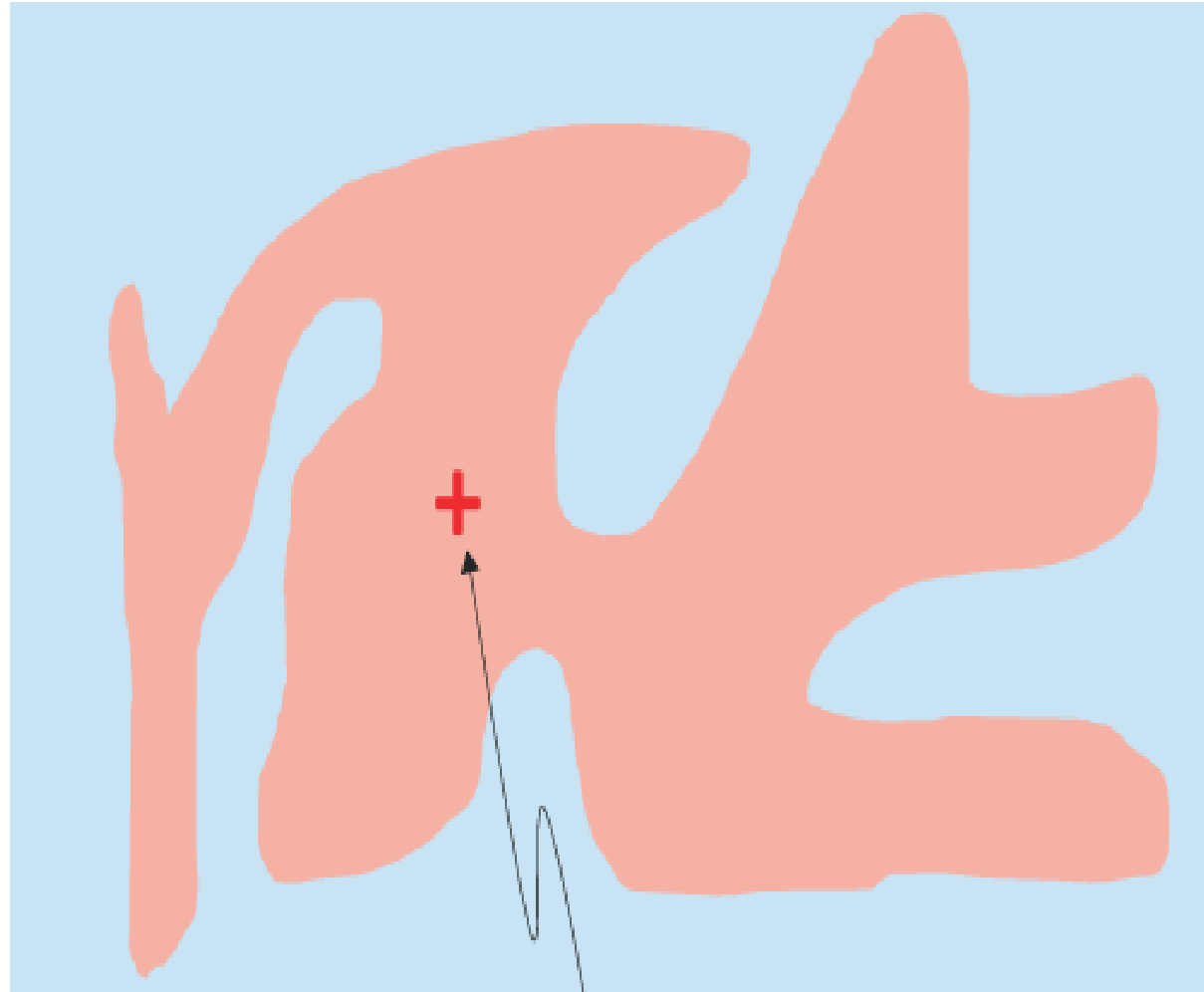
Get Predictions: Use the complex "black-box" model to get predictions for these new, synthetic data points.

Weight Samples: Weight the synthetic samples based on their proximity to the original instance—closer samples matter more.

Train Interpretable Model: Fit a simple, interpretable model (like a linear model) to the weighted, perturbed samples.

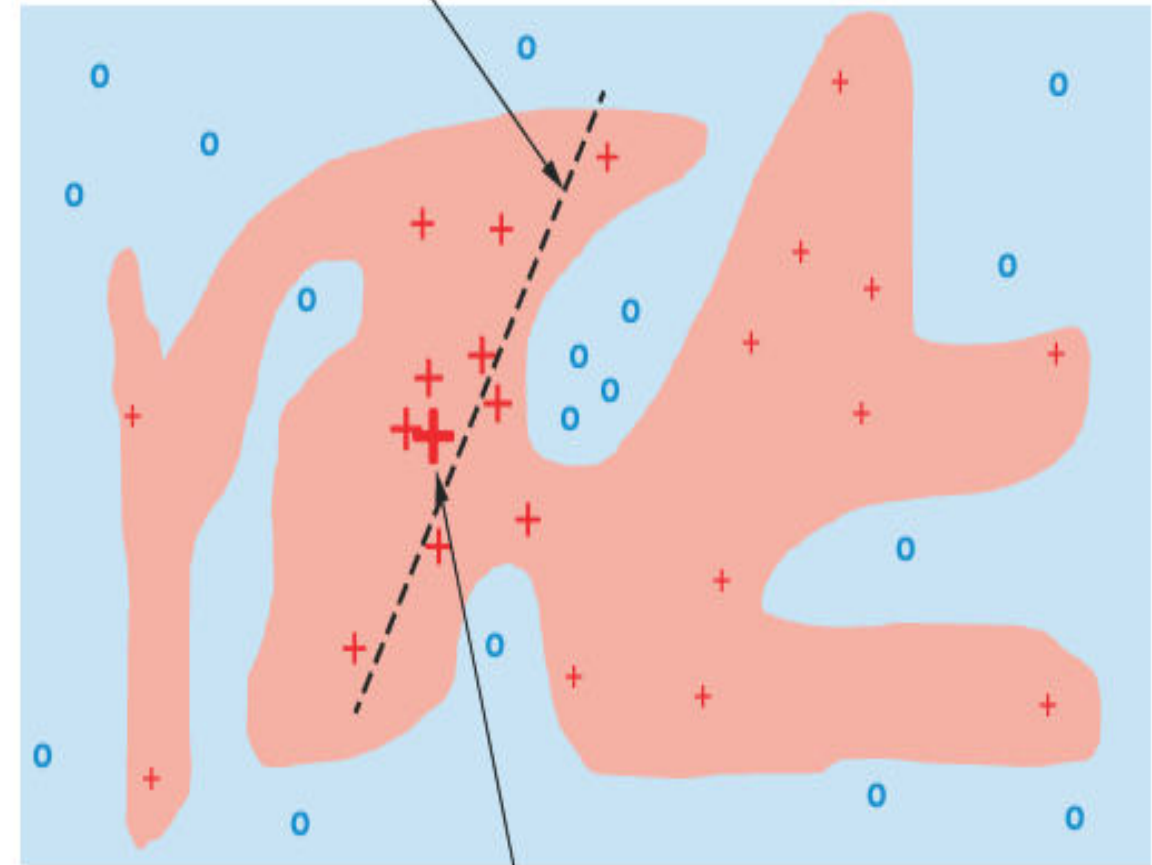
Explain: Use the simple model's features to explain the original prediction

LIME – Local Surrogate



Malignant case picked to interpret

Interpretable linear model that is locally faithful to the picked malignant case



Malignant case picked to interpret

Why LIME Matters?

Real-World Scenarios Where Explanations Are Critical



Healthcare Decisions

A doctor uses AI to predict cancer risk. When the patient asks "Why this result?", they deserve a clear answer. LIME reveals which medical features—tumor size, cell texture, or other markers—drove the prediction, enabling informed treatment decisions.



Financial Services

A bank's AI rejects a loan application. The customer asks "Why was I denied?" LIME shows exactly which factors—credit score, income level, or debt ratio—influenced the decision, ensuring transparency and regulatory compliance.



Building Trust

LIME bridges the gap between complex AI systems and human understanding. By showing which features influenced each decision, it transforms black-box models into trustworthy tools that stakeholders can confidently rely on.

Eg.1 Explaining Medical Predictions with Tabular Data

The Scenario

We use the classic breast cancer dataset containing medical features like cell size, texture, radius, and symmetry. Our goal: predict whether a tumor is malignant or benign using a Random Forest classifier.

The Challenge: Random Forests are powerful but opaque. LIME reveals exactly which features drove each prediction.

1. Load Dataset

Import breast_cancer.csv with features like mean radius, texture, perimeter, area, and smoothness for 569 patients

3. Generate Explanation

Use LimeTabularExplainer to show which features pushed the prediction toward malignant or benign

2. Train Model

Build Random Forest classifier to predict malignant vs. benign tumors with high accuracy

4. Visualize Results

Display interactive bar chart showing feature importance for a specific patient's prediction



Code Implementation

Building the Tabular Explainer Step-by-Step

01

Data Preparation

Load the breast cancer dataset using sklearn's `load_breast_cancer()` function and export as CSV for reproducibility

02

Model Training

Split data into training and test sets, then train a Random Forest classifier with optimal hyperparameters

03

LIME Configuration

Initialize `LimeTabularExplainer` with feature names, class labels, and the training dataset for context

04

Explain Prediction

Select a test instance and generate an explanation showing the top features influencing the prediction

05

Interactive Visualization

Display the explanation as an interactive bar chart in your Jupyter notebook using `show_in_notebook()`

```
from lime.lime_tabular import LimeTabularExplainer
from sklearn.ensemble import RandomForestClassifier

# Train model
rf = RandomForestClassifier()
rf.fit(X_train, y_train)

# Create explainer
explainer = LimeTabularExplainer(X_train, feature_names=feature_names,
                                 class_names=['benign', 'malignant'])

# Explain prediction
exp = explainer.explain_instance(X_test[0], rf.predict_proba)
exp.show_in_notebook()
```

True label: malignant

Prediction probabilities

malignant		1.00
benign		0.00

malignant

benign

worst concave points > ...	0.14
worst area > 1030.75	0.13
worst perimeter > 123.47	0.10
worst radius > 18.42	0.08
radius error > 0.46	0.06

Feature	Value
worst concave points	0.27
worst area	1821.00
worst perimeter	184.60
worst radius	25.74
radius error	0.73

Eg. 2 Understanding Sentiment Analysis with Text Data

The Text Challenge

We work with movie reviews labeled as positive or negative sentiment. Using a Logistic Regression classifier with TF-IDF features, we predict whether new reviews express positive or negative opinions.

The Insight: LIME highlights specific words that pushed the model toward its prediction—revealing which terms like "boring," "brilliant," or "disappointing" had the strongest influence.

Why This Matters

- Identifies biased or unexpected word associations
- Validates that the model focuses on meaningful terms
- Helps improve model performance by revealing weaknesses
- Provides transparency for content moderation systems

Positive Example

"This movie was absolutely brilliant and captivating from start to finish!"

Negative Example

"The plot was boring and the acting felt completely wooden."

Part 2: Code Implementation

Building the Text Explainer Pipeline



Create Dataset

Build a small sentiment.csv with labeled movie reviews covering diverse opinions and writing styles



Feature Engineering

Transform text into numerical features using TF-IDF vectorization to capture word importance



Train Classifier

Build Logistic Regression model on vectorized text to predict positive vs. negative sentiment



Generate Explanation

Use LimeTextExplainer to highlight influential words in a specific review

```
from lime.lime_text import LimeTextExplainer
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
```

```
# Build text classifier pipeline
vectorizer = TfidfVectorizer()
classifier = LogisticRegression()
pipeline = make_pipeline(vectorizer, classifier)
pipeline.fit(reviews, labels)
```

```
# Create text explainer
explainer = LimeTextExplainer(class_names=['negative', 'positive'])
```

```
# Explain a review
exp = explainer.explain_instance(test_review, pipeline.predict_proba)
exp.show_in_notebook()
```

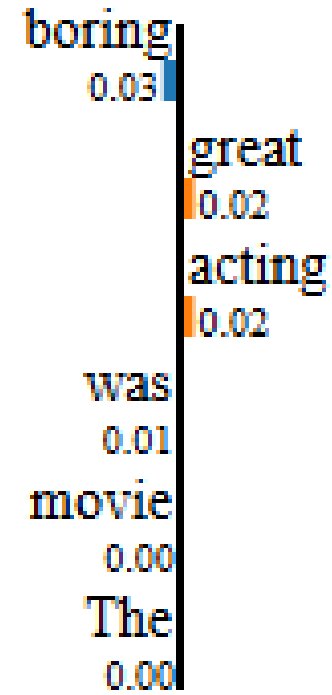
Prediction probabilities

Negative 0.48

Positive 0.52

Negative

Positive



Text with highlighted words

The movie was boring but the acting was great.

Eg. 3 Interpreting Image Classification with Pixel-Level Insights

The Vision Problem

Using the handwritten digits dataset, we work with 8×8 grayscale images representing numbers 0-9. Each image contains 64 pixels that the model uses to classify which digit is shown.

The Breakthrough: LIME reveals exactly which pixels or regions of the image drove the classification decision—showing us what the model "sees" when making predictions.



Dataset Details

- 8×8 pixel grayscale images
- 64 features per image
- 10 digit classes (0-9)

Model Approach

- Logistic Regression classifier
- Flattened pixel values as features
- Normalized intensity values

LIME Output

- Pixel importance map
- Bar chart of feature influence
- Visual explanation overlay

Eg.3 Explaining Image Predictions with LIME

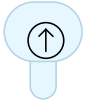


Image Preprocessing

Upload or generate handwritten digit image, convert to 8×8 grayscale format, and ensure proper pixel value ranges



Feature Extraction

Normalize pixel values to 0-1 range and flatten the 2D image array into a 64-element feature vector for the model



Data Export

Save processed images and labels as digits.csv for reproducibility and sharing with team members



LIME Explanation

Initialize LimeTabularExplainer treating each pixel as a feature, then generate explanation showing pixel importance



Visualization

Display original digit image alongside LIME explanation showing which pixels influenced the prediction most strongly

```
from lime.lime_tabular import LimeTabularExplainer
from sklearn.linear_model import LogisticRegression
import numpy as np

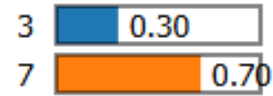
# Prepare image data
digits_flat = digits.reshape(n_samples, -1) # Flatten to 64 features
digits_normalized = digits_flat / 16.0 # Normalize

# Train classifier
clf = LogisticRegression()
clf.fit(X_train, y_train)

# Explain image prediction
explainer = LimeTabularExplainer(X_train, feature_names=pixel_names,
                                class_names=digit_classes)
exp = explainer.explain_instance(test_image, clf.predict_proba)
exp.show_in_notebook()
```

Predicted label (mock): 7

Prediction probabilities



3

pixel_0 <= 14.56
0.00
pixel_1 <= 14.56
0.00
pixel_2 <= 15.06
0.00
pixel_3 <= 14.87
0.00
pixel_4 <= 4.83
0.00
pixel_5 <= 14.49
0.00
pixel_6 <= 15.25
0.00
pixel_7 <= 15.00
0.00
pixel_8 <= 13.11
0.00
pixel_9 <= 7.40
0.00

Feature Value

pixel_0 14.56
pixel_1 14.56
pixel_2 15.06
pixel_3 14.87
pixel_4 4.83
pixel_5 14.49
pixel_6 15.25
pixel_7 15.00
pixel_8 13.11
pixel_9 7.40

7

Uploaded Digit Image



Bringing It All Together

How Three Data Types Demonstrate LIME's Universal Power



Tabular Data

Medical Predictions:

LIME explains which patient features—cell radius, texture, symmetry—drive cancer diagnosis predictions, enabling doctors to validate AI recommendations and explain results to patients with confidence.



Text Data

Sentiment Analysis:

LIME highlights specific words that influence opinion classification, revealing whether the model focuses on meaningful terms or spurious correlations, ensuring transparency in content moderation and review systems.



Image Data

Visual Classification:

LIME shows which pixels drive digit recognition, demonstrating what the model "sees" and helping us validate that it focuses on relevant image regions rather than background noise or artifacts.

The Universal Message

LIME empowers us to trust AI by revealing the **"why"** behind every prediction—whether analyzing numbers, understanding language, or interpreting images. This transparency is essential for responsible AI deployment in healthcare, finance, and beyond.

LIME transforms black-box models into explainable systems across all data types, enabling stakeholders to understand, validate, and trust AI-driven decisions in critical applications.



THANK YOU



Elective - III

UE23AM342BA3 : Explainable AI

**UNIT II : Global and Local model agnostic
Methods**

Dr. Siba Ram Baral, Prof. Ayisha Noori V K
Department of Computer Science and
Engineering(AI&ML)

UNIT 2:

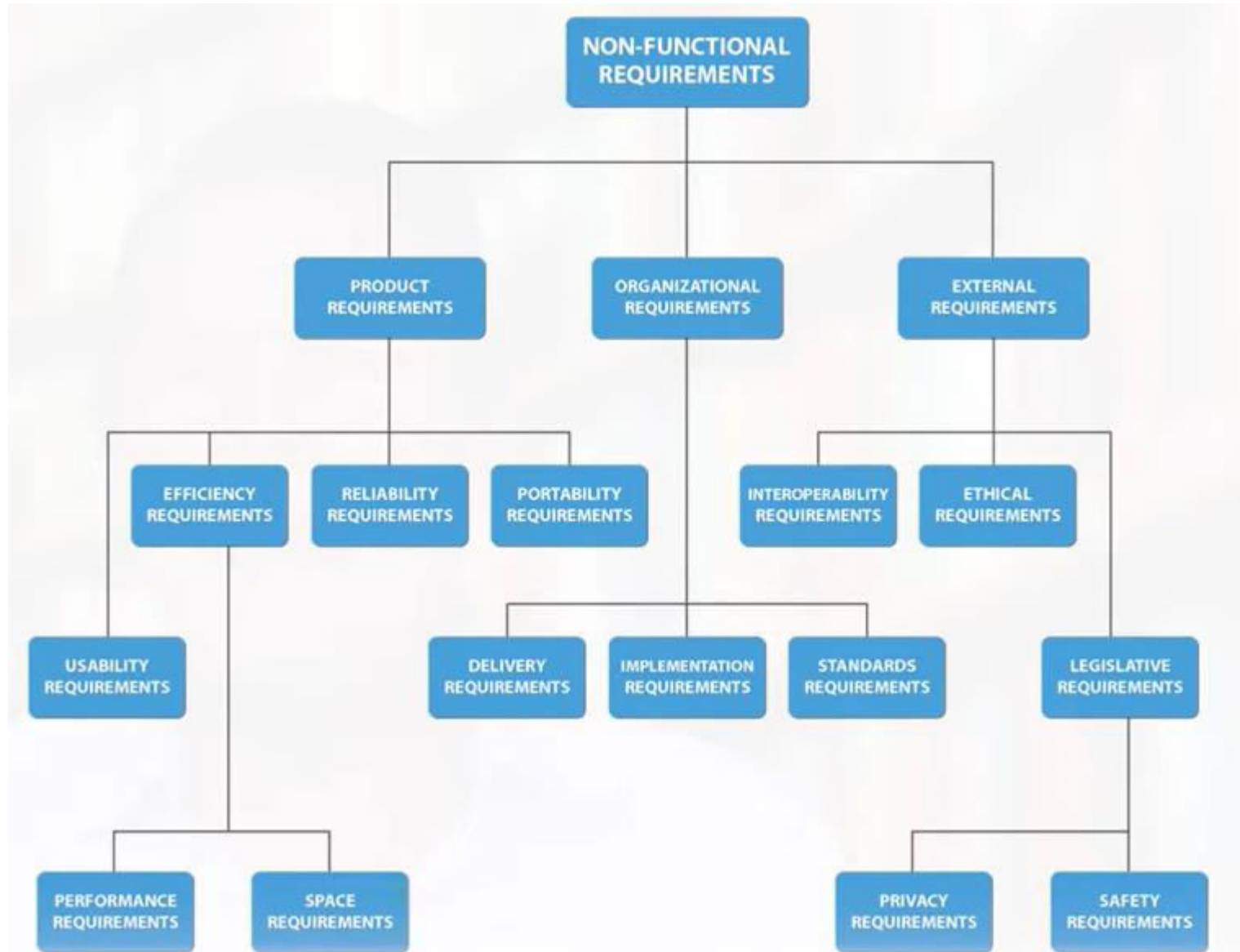
Functional Decomposition

What is Functional Decomposition

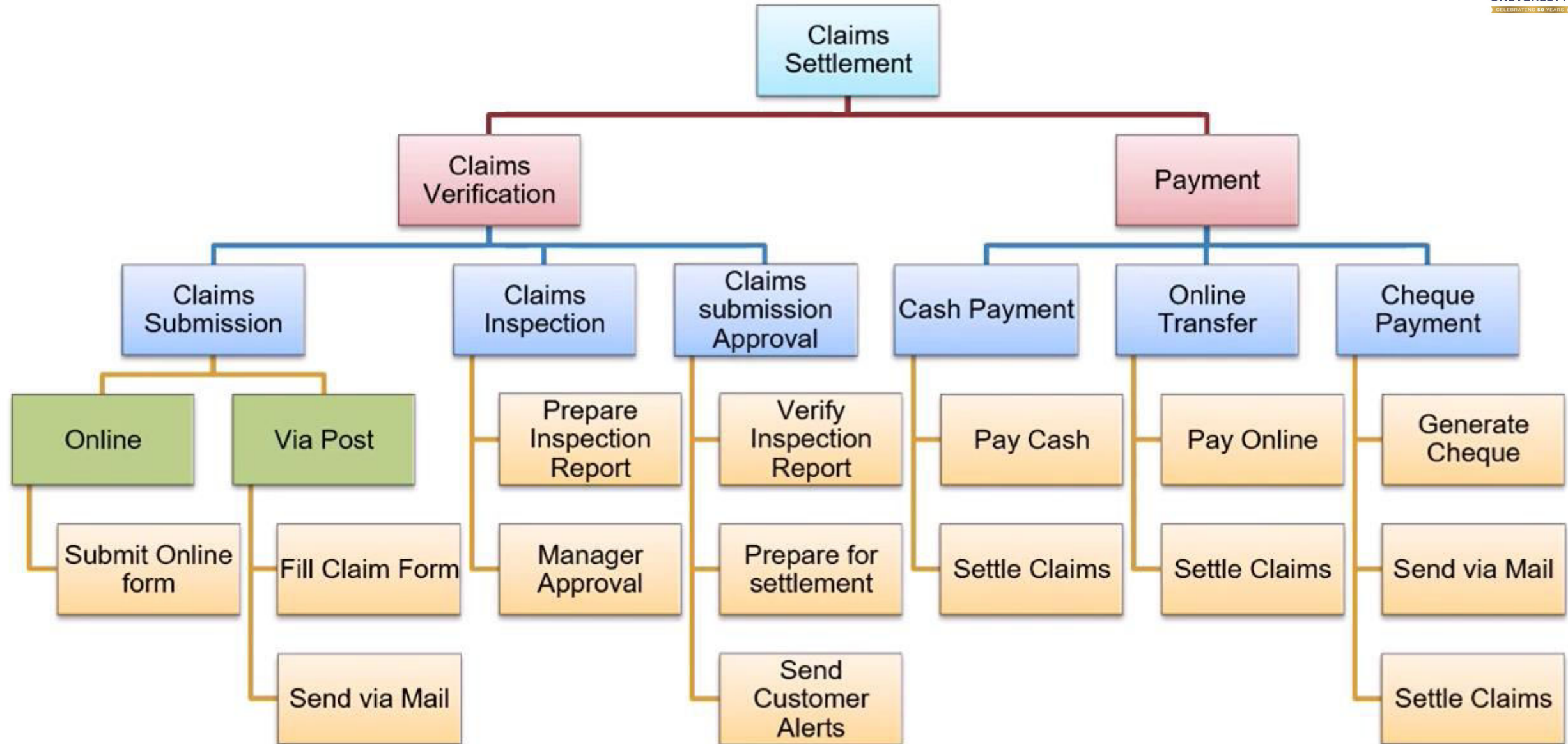
Definition

Process of breaking down complex functions, problems or tasks into small independent granular modules so that debugging, troubleshooting and logical breakdown can be done in order to simplify the complexity

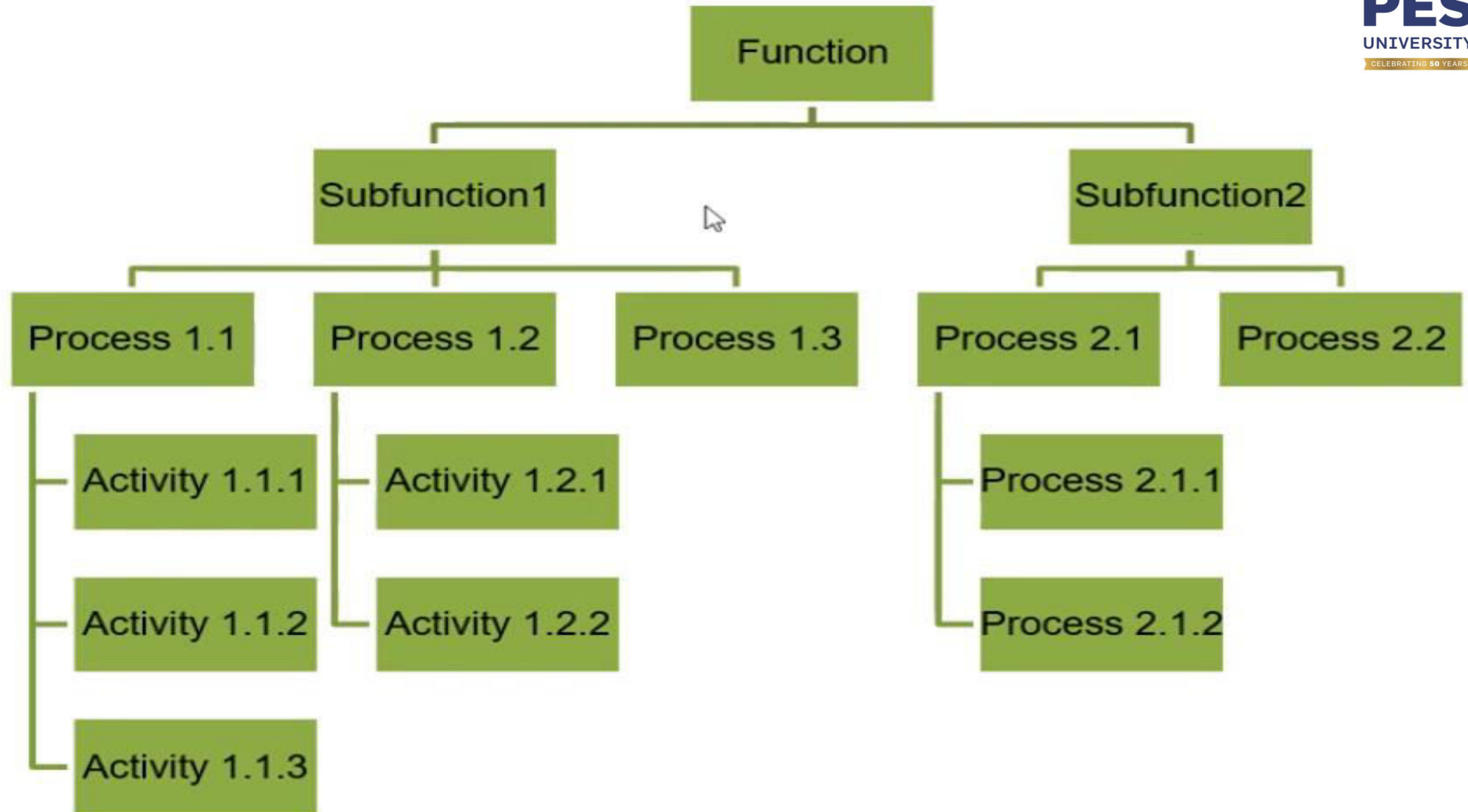
- Data decomposition (Ex : Matrix multiplication)
- Recursive decomposition (Ex : Merge Sort)
- Exploratory decomposition. (Ex : 15 puzzle problem)
- Speculative decomposition. (Ex : Switch Statement)
 - Conservative approach
 - Speculative approach
- Hybrid decomposition.



Functional Decomposition- *Breaking down approach*



Functional Hierarchy

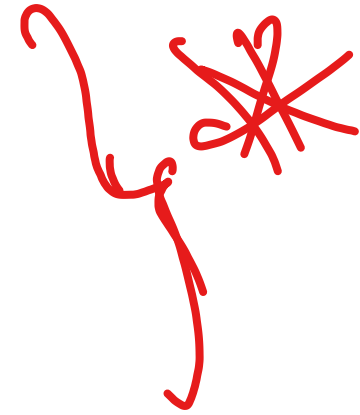


Modularity -> Decomposition

- We now know:
 - What a module is
 - How they help us come up with “good” software system designs
 - What the desirable properties of modules are
- Next Step - Given a problem statement:
 - How do we start to break a problem down into modules?
 - How do we compare design decisions?
 - How do we communicate our design?

Functional vs. Object-Oriented

- There are numerous methodologies for systematically going through a problem statement and coming up with a design
- However, *most* all of these fall under two large categories:
 - Functional Decomposition
 - Based around “verbs” ✓
 - What actions are taking place in the system? ✓
 - How do we control the flow of data and decisions? ✓
 - Object-Oriented Decomposition
 - Based around “nouns” ✓
 - What are the main actors / entities in the system and how do they interact with each other?
 - How do we define “state” and “behavior” in our system? ✓



Split Up Complex Functions!

```
def is_user_freezing(text:str)->bool:  
    celsius = int(text[-2:])  
    fahrenheit = (celsius*9/5) + 32  
    return fahrenheit < 32
```



```
def capture_temperature(text:str)->int:  
    return int(text[-2:])
```

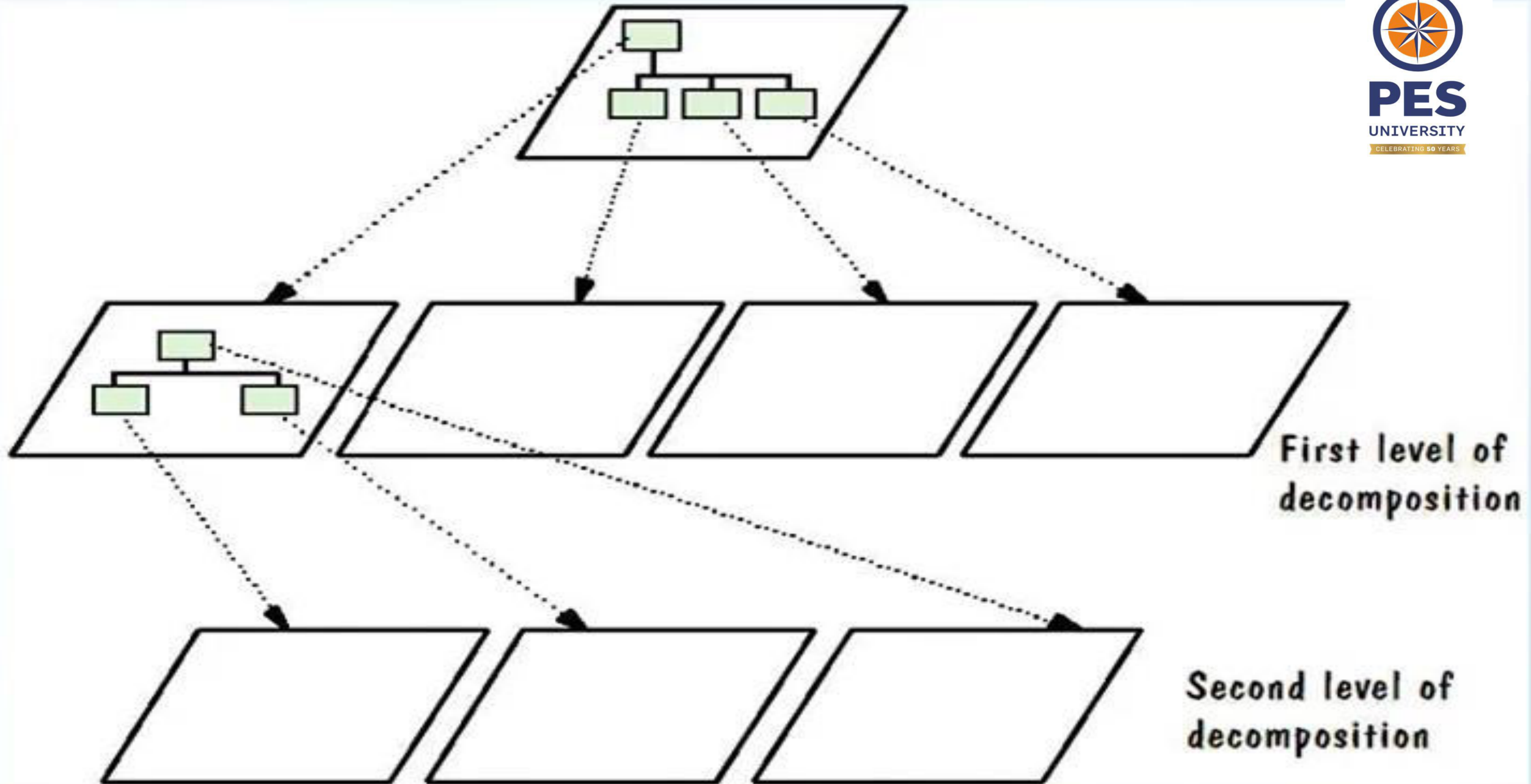
```
def to_fahrenheit(celsius:int)->float:  
    return (celsius*8/5) + 32
```

```
def is_freezing(temperature:float)->bool:  
    return temperature < 32
```



```
def is_user_freezing(text:str)->bool:  
    celsius = capture_temperature(text)  
    fahrenheit = to_fahrenheit(celsius)  
    return is_freezing(fahrenheit)
```

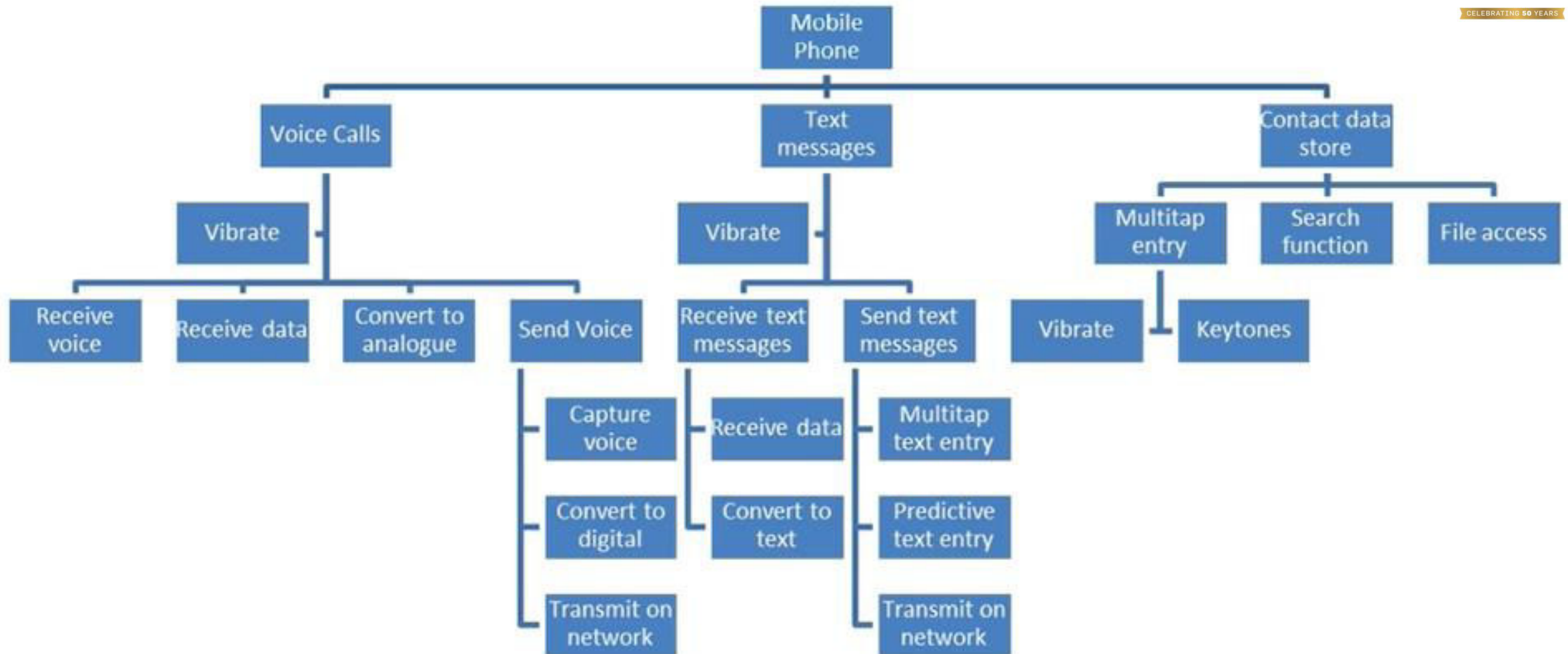

Functional Decomposition



Assemble..

- Eventually all small functions can be combined together
- Creates solution for main problem

A Mobile phone system



Functional Decomposition – Interpretation Technique

- A supervised machine learning model can be viewed as a function that takes a high-dimensional feature vector as input and produces a prediction or classification score as output
- Functional decomposition is an interpretation technique that deconstructs the high-dimensional function and expresses it as a sum of individual feature effects and interaction effects that can be visualized
- functional decomposition is a fundamental principle underlying many interpretation techniques – it helps you better understand other interpretation methods

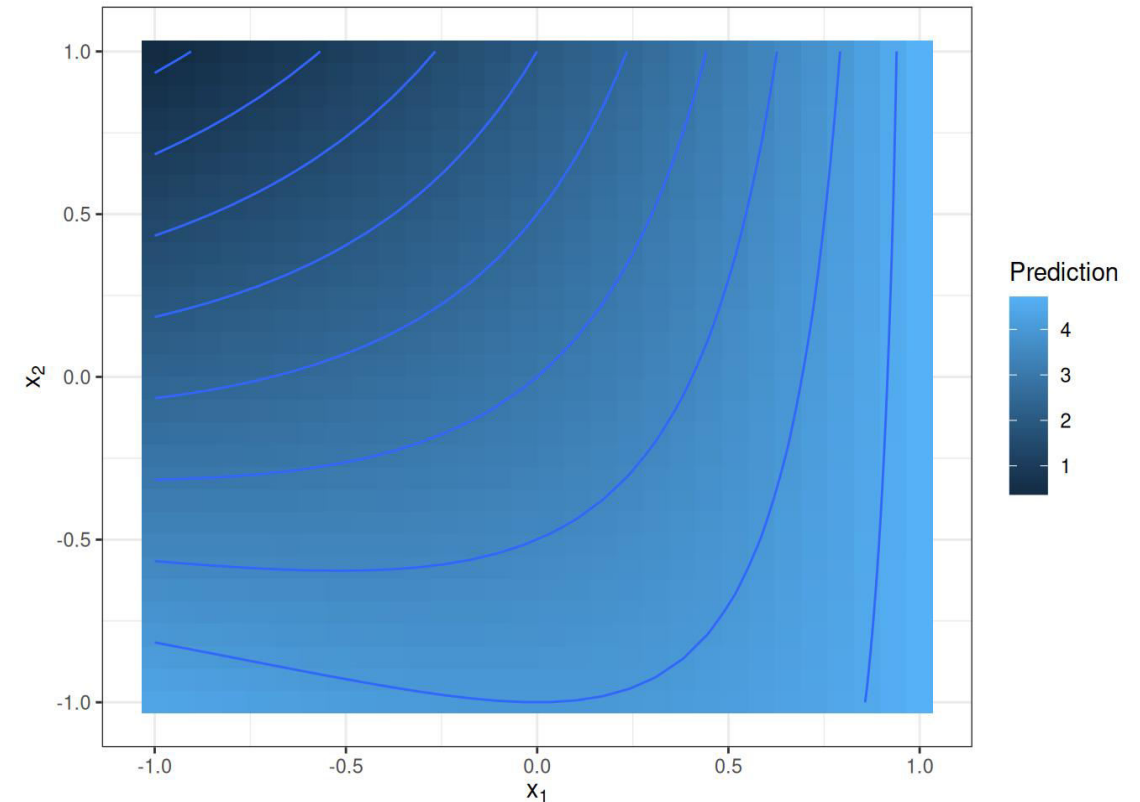
Consider a function which takes two features as input and produces a one-dimensional output:

$$\hat{f}(x_1, x_2) = 2 + e^{x_1} - x_2 + x_1 \cdot x_2$$

Think of the function as a machine learning model. We can visualize the function with a 3D plot or a heatmap with contour lines as in figure.

The lighter the color, the larger the prediction.

The function takes large values when x_1 is large and x_2 is small, and it takes small values for large x_2 and small x_1 . The prediction function is not simply an additive effect between the two features, but an interaction between the two. The presence of an interaction can be seen in the figure – the effect of changing values for feature x_1 depends on the value that feature x_2 has.



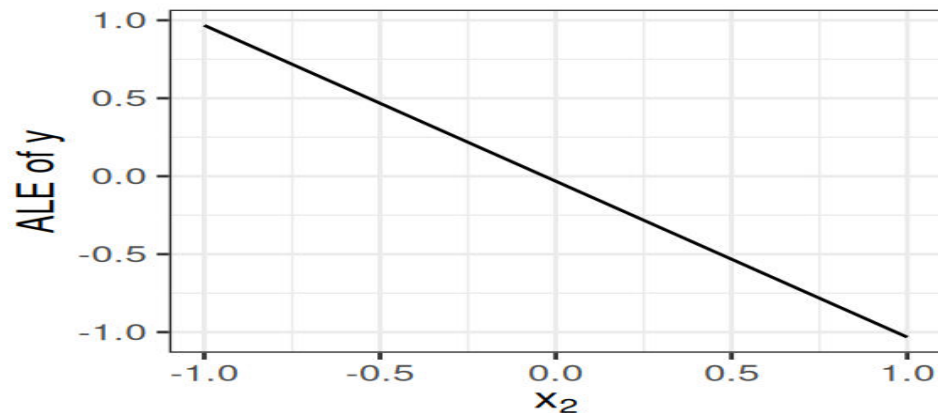
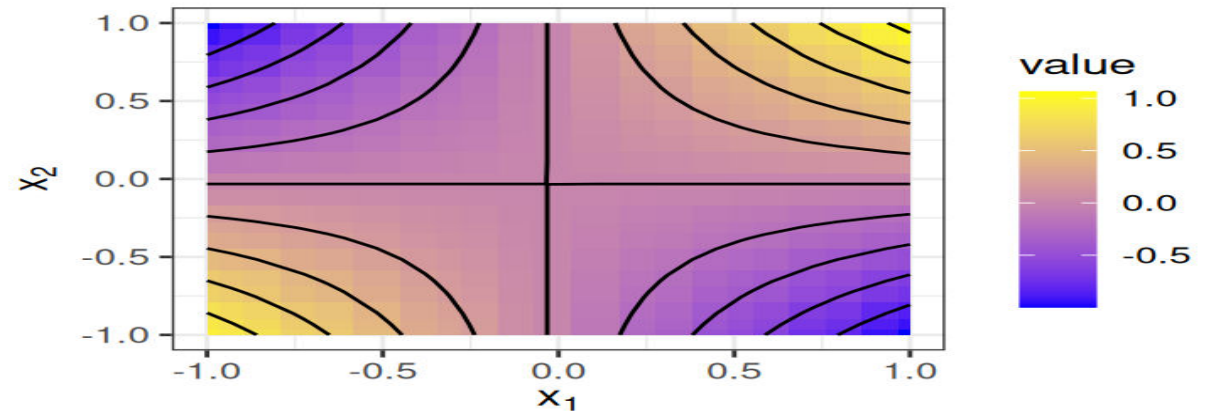
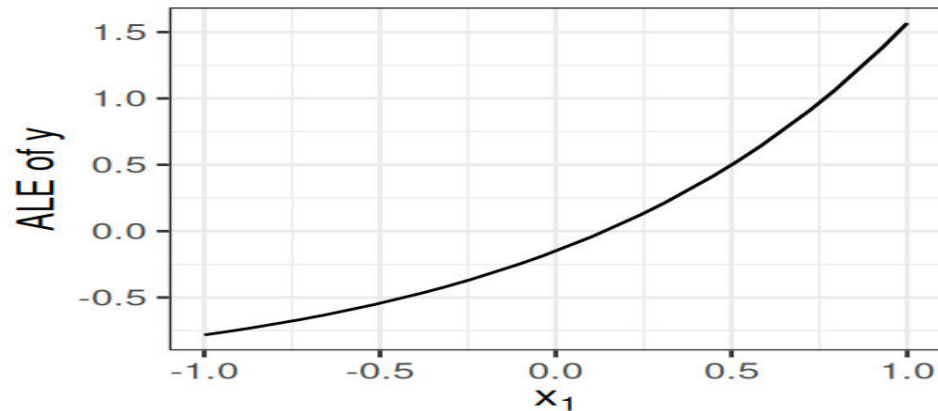
Functional Decomposition – Interpretation Technique

A two dimensional function can be decomposed as : $\hat{f}(x_1, x_2) = \hat{f}_0 + \hat{f}_1(x_1) + \hat{f}_2(x_2) + \hat{f}_{1,2}(x_1, x_2)$

To decompose this function into main effects of features X1 and X2 and an interaction term

$$\hat{f}(x_1, x_2) = 2 + e^{x_1} - x_2 + x_1 \cdot x_2$$

The intercept is given as $\hat{f}_0 \approx 3.18$

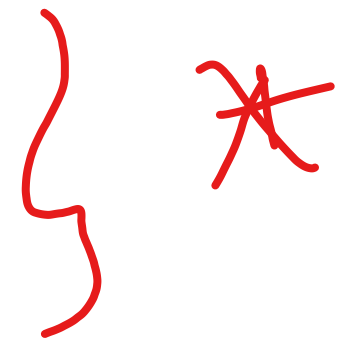


X1 feature shows an exponential main effect, and X2 shows a negative linear effect

Decomposing a function

A prediction function takes features as input, p , and $\hat{f}: \mathbb{R}^p \mapsto \mathbb{R}$ produces an output. This can be a regression function, but it can also be the classification probability for a given class or the score for a given cluster (unsupervised machine learning). Fully decomposed, we can represent the prediction function as the sum of functional components:

$$\begin{aligned}\hat{f}(\mathbf{x}) = & \hat{f}_0 + \hat{f}_1(x_1) + \dots + \hat{f}_p(x_p) \\ & + \hat{f}_{1,2}(x_1, x_2) + \dots + \hat{f}_{1,p}(x_1, x_p) + \dots + \hat{f}_{p-1,p}(x_{p-1}, x_p) \\ & + \dots \\ & + \hat{f}_{1,\dots,p}(x_1, \dots, x_p)\end{aligned}$$



We can make the decomposition formula a bit nicer by indexing all possible subsets of feature combinations: $S \subseteq \{1, \dots, p\}$. This set contains the intercept ($S = \emptyset$), main effects ($|S| = 1$), and all interactions ($|S| \geq 2$). With this subset defined, we can write the decomposition as follows:

$$\hat{f}(\mathbf{x}) = \sum_{S \subseteq \{1, \dots, p\}} \hat{f}_S(\mathbf{x}_S)$$

In the formula, $\mathbf{x}_S$ is the vector of features in the index set S . And each subset S represents a functional component, for example, a main effect if S contains only one feature, or an interaction if $|S| > 1$.

Approaches for Functional Decomposition

- (Generalized) functional ANOVA ✓
- Accumulated Local Effects ✓
- Statistical regression models ✓
- Decomposing tree ensembles ✓

Generalized Functional ANOVA for dependent features

Similar to most interpretation techniques based on sampling data (such as the PDP), the functional ANOVA can produce misleading results when features are correlated.

If we integrate over the uniform distribution, when in reality features are dependent, we create a new dataset that deviates from the joint distribution and extrapolates to unlikely combinations of feature values.

Hooker ([2007](#)) proposed the generalized functional ANOVA, a decomposition that works for dependent features.

It's a generalization of the functional ANOVA \

The components are defined as projections of $\hat{f}$ onto the space of additive functions:

$$\checkmark \quad \hat{f}_S(\mathbf{x}_S) = \arg \min_{g_S \in L^2(\mathbb{R}^S)_{S \in P}} \int \left(\hat{f}(\mathbf{x}) - \sum_{S \subset P} g_S(\mathbf{x}_S) \right)^2 w(\mathbf{x}) d\mathbf{x}.$$

Instead of orthogonality, the components satisfy a hierarchical orthogonality condition:

$$\forall \hat{f}_S(\mathbf{x}_S) | S \subset U : \int \hat{f}_S(\mathbf{x}_S) \hat{f}_U(\mathbf{x}_U) w(\mathbf{x}) d\mathbf{x} = 0$$

Hierarchical orthogonality is different from orthogonality. For two feature sets S and U , neither of which is a subset of the other (e.g., $S = \{1, 2\}$ and $U = \{2, 3\}$), the components $\hat{f}_S$ and $\hat{f}_U$ need not be orthogonal for the decomposition to be hierarchically orthogonal.

The Problem:

"A customer wants to know why their house was valued at \$260,000 when the neighborhood average is \$200,000."

Functional Decomposition Solution:

We break down the final prediction into individual feature weights (marginal contributions). This allows us to explain the \$60,000 premium.

● **Base Value:** Starting average market price. **+2,00,000**

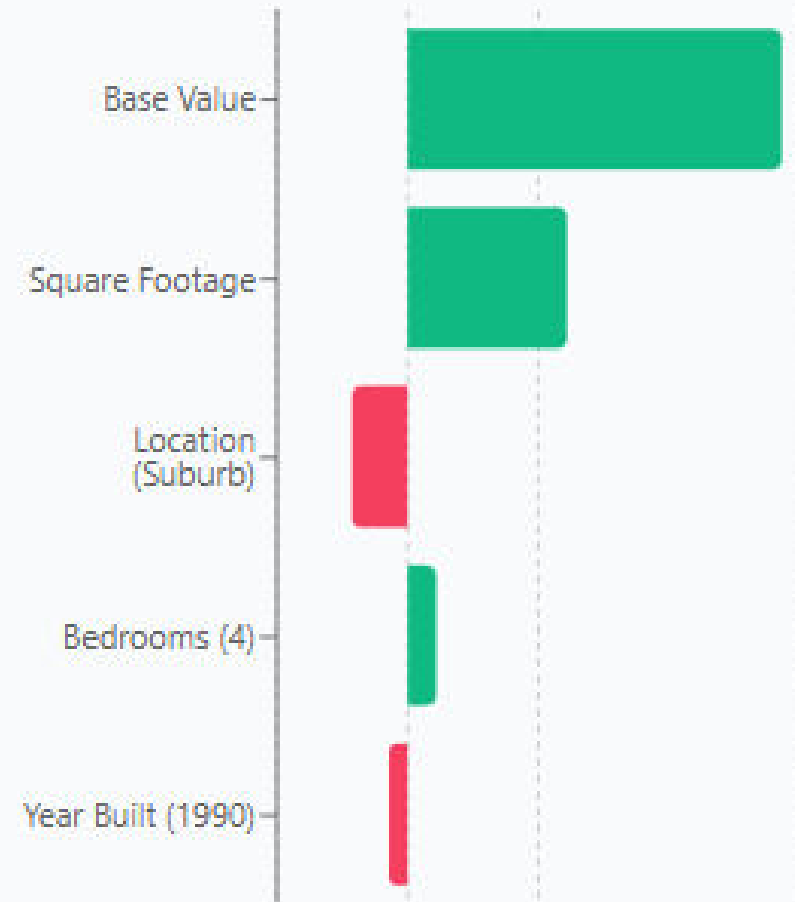
● **Square Footage:** Extra 500 sq ft increased value. **+85,000**

● **Location (Suburb):** Slightly further from city center. **-30,000**

● **Bedrooms (4):** Additional bedroom bonus. **+15,000**

● **Year Built (1990):** Age depreciation factor. **-10,000**

VISUALIZING CONTRIBUTIONS (WATERFALL CONCEPT)



Problem statement - 1

Let's learn how a loan approval model can be decomposed into smaller functions (income check, credit score check, debt ratio check). Each function contributes to the final decision, making the AI explainable. A bank uses AI to decide if a loan should be approved. Instead of one opaque model, we break it into functional components.

Functional Decomposition

Income Check $\rightarrow$ verifies if $\text{income} \geq \text{threshold}$

Credit Score Check $\rightarrow$ verifies if $\text{credit score} \geq \text{threshold}$

Debt Ratio Check $\rightarrow$ verifies if $\text{debt ratio} \leq \text{threshold}$

Final Decision $\rightarrow$ combines results

Problem statement - 2

Let's learn how spam detection can be decomposed into interpretable modules (keyword check, sender reputation, link count). Each function explains part of the decision.

An AI model classifies emails as spam or not. Instead of a black-box classifier, we break it into functional checks.

Functional Decomposition

Keyword Check → looks for suspicious words

Sender Reputation Check → checks if sender is trusted

Link Count Check → counts suspicious links

Final Decision → aggregates results



THANK YOU





UE23AM342BA3
Explainable AI

Local model agnostic Methods

Prof. Ayisha Noori VK
Dept. of CSE(AI&ML), PES University,
Bengaluru

Contents:

- SHAP



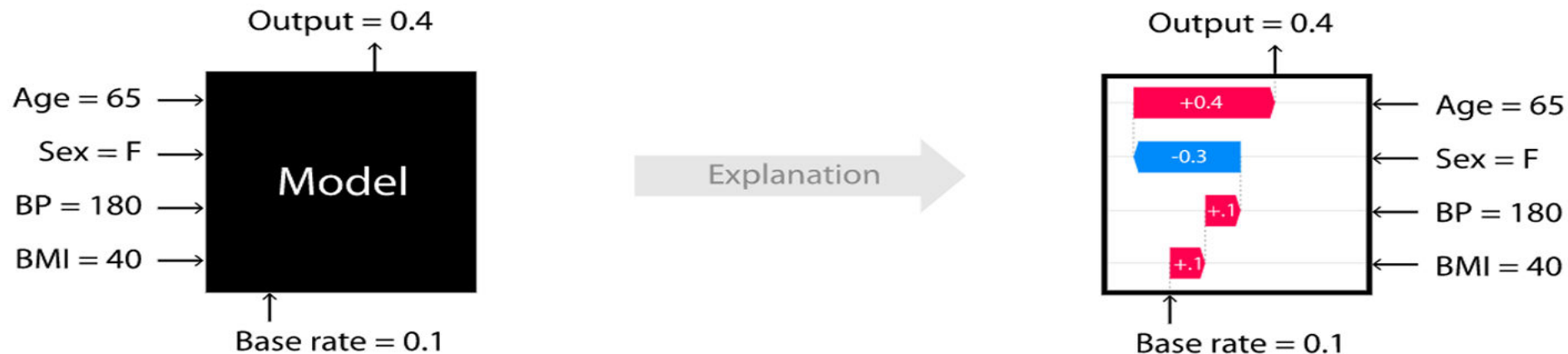
SHAP – SHapley Additive exPlanations

SHAP is a model agnostic explainer:

- Its purpose is to "imitate" the model used.
- It gives an understandable explanation of a local prediction of a model by assigning to each feature a value.
- Based on **Shapely** values from **cooperative game theory**

The *Shapley value* is a mathematical concept in game theory that was introduced by Lloyd Shapley in 1951 [72], later awarded the Nobel Prize in Economics for this work.

SHAP is a method that enables a fast computation of Shapley values and can be used to explain the prediction of an instance x by computing the contribution (Shapley value) of each feature to the prediction.

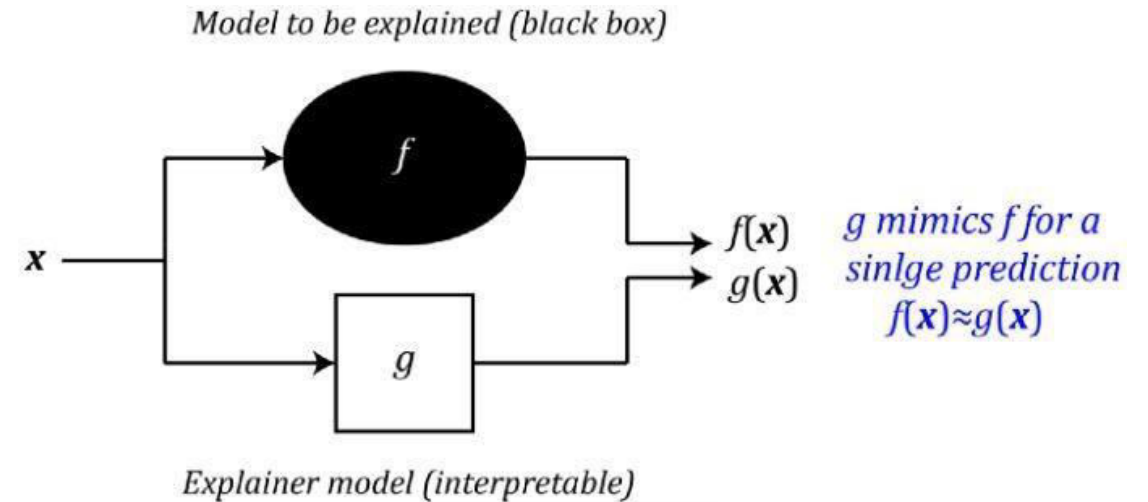


SHAP-How it works?

How to achieve this?

It uses the concept of **Shapley Value**.

The *Shapley value* is a mathematical concept in game theory that was introduced by Lloyd Shapley in 1951 [72], later awarded the Nobel Prize in Economics for this work.



-worth of a coalition on which is Shapley value based on

SHAP-How it works?

- Let's consider a game in co-operative with m numbered players and call F the set of such players.

$$F = \{1, 2, 3, 4, \dots, m\}$$

- We then define an S coalition as a subset of F , which also includes the empty set without players.
- An example of the possible coalitions with 3 players

$$\{\emptyset, \{1\}, \{2\}, \{3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}, \{1, 2, 3\}\}$$

SHAP-How it works?

- Let us now define a function v , called a **characteristic function**, which associates each coalition with a real number. The value $v(S)$ will be called the **worth of the coalition S** and represents the total gain obtained by the coalition if the members act together.
- Obviously in the trivial case:

$$v(\emptyset) = 0$$

- At this point, for the calculation of the value of $\{3\}$, for example, you can proceed by subtracting:

$$v(\{1, 2, 3\}) - v(\{1, 2\})$$

The aim now is to assess what each player's contribution is in the total payoff.

SHAP-How it works?

Considering the number of permutations and summing up on all other possible combinations we get:

$$\phi_i = \sum_{S \subseteq F - \{i\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} (\nu(S \cup \{i\}) - \nu(S))$$

where $|F|!$ is the total number of permutations of the grand coalition, S indicates the coalition and $S \cup \{i\}$ indicates the coalition with the addition of i .

The ϕ_i value is called **Shapley Value** and represents the average contribution of the player, or variable, i .

Shapley Values in Machine Learning?

How translate this in Machine Learning?

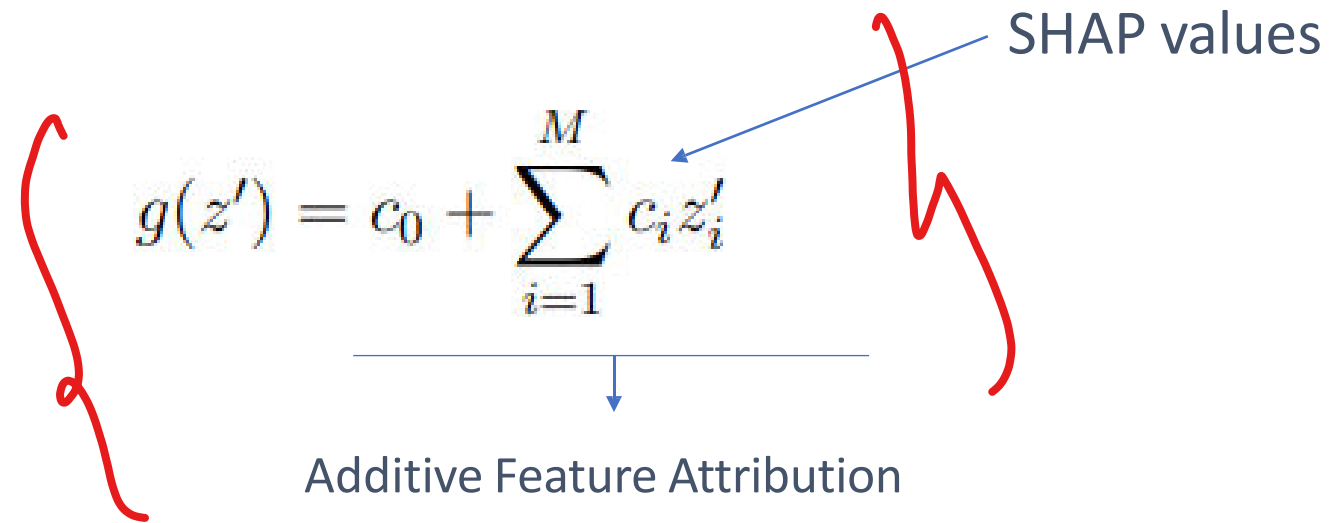
- **SHAP** (Shapley Additive Explanation)

Input data	x	z'	Simplified local inputs (generally $z' \in (0,1)$)
Input model	$f(x)$	$g(z')$	Explanatory models

$$\text{If } x \approx z' \text{ then } f(x) = g(z')$$

Shapley Values in Machine Learning

We assume:



The diagram shows the equation $g(z') = c_0 + \sum_{i=1}^M c_i z'_i$. A blue arrow points from the text "SHAP values" to the summation term $\sum_{i=1}^M c_i z'_i$. A red bracket is drawn around the entire equation. Below the equation, a horizontal line with a downward-pointing arrow is labeled "Additive Feature Attribution".

$$g(z') = c_0 + \sum_{i=1}^M c_i z'_i$$

SHAP values

Additive Feature Attribution

- c_i represent the contribution of each feature.
- c_0 is the average output of the model.
- Easy to interpret.

Shapley Values in Machine Learning

How we calculate SHAP values?

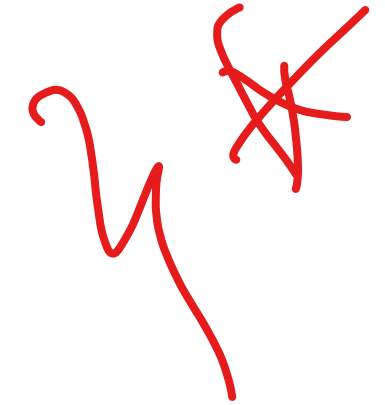
Player $\longleftrightarrow$ Feature

$$\nu(\emptyset) = 0 \longleftrightarrow f(NA, NA, \dots, NA) = E[f(x)] = \frac{1}{k} \sum_{j=1}^k f(x^{(j)})$$

$$f_S(x_S) = E[f(x) | x_S]$$

$$\nu(S) = f_S(x_S) - E[f(x)]$$

Shapley Values in Machine Learning



Finally we get:

$$\phi_i(f, x) = \sum_{z' \subseteq x'} \frac{(|z'| - 1)(|x'| - |z'|)!}{|x'|!} (E[f(z)|z_S] - E[f(z)|z_{S \setminus \{i\}}])$$

Shapley values calculated using the conditional expectations are called **SHAP** values.

SHAP – Desirable Properties

SHAP describes the following three desirable properties:

It is proved that the function $g(z')$ has the following property:

Local Accuracy:

- If $x \approx z'$ then $f(x) = g(z')$.
- *SHAP values add up to the difference between the expected model output and the actual output for a given input. This means that SHAP values provide an accurate and local interpretation of the model's prediction for a given input.*

Missingness:

- If $z'_i = 0$, then $\phi_i = 0$.
- *SHAP values are zero for missing or irrelevant features for a prediction. This makes SHAP values robust to missing data and ensures that irrelevant features do not distort the interpretation.*

Consistency:

- If feature contribution changes, the feature effect cannot change in the opposite way. ✓
- *SHAP values do not change when the model changes unless the contribution of a feature changes. This means that SHAP values provide a consistent interpretation of the model's behavior, even when the model architecture or parameters change.*

SHAP Explainability Explained

"Who Actually Did the Work?" - A Capstone Project Mystery

The Scenario: Final Capstone Project

Project: "AI-Powered Recipe Recommender App"

Team: 4 students

Final Grade: A- (92%)

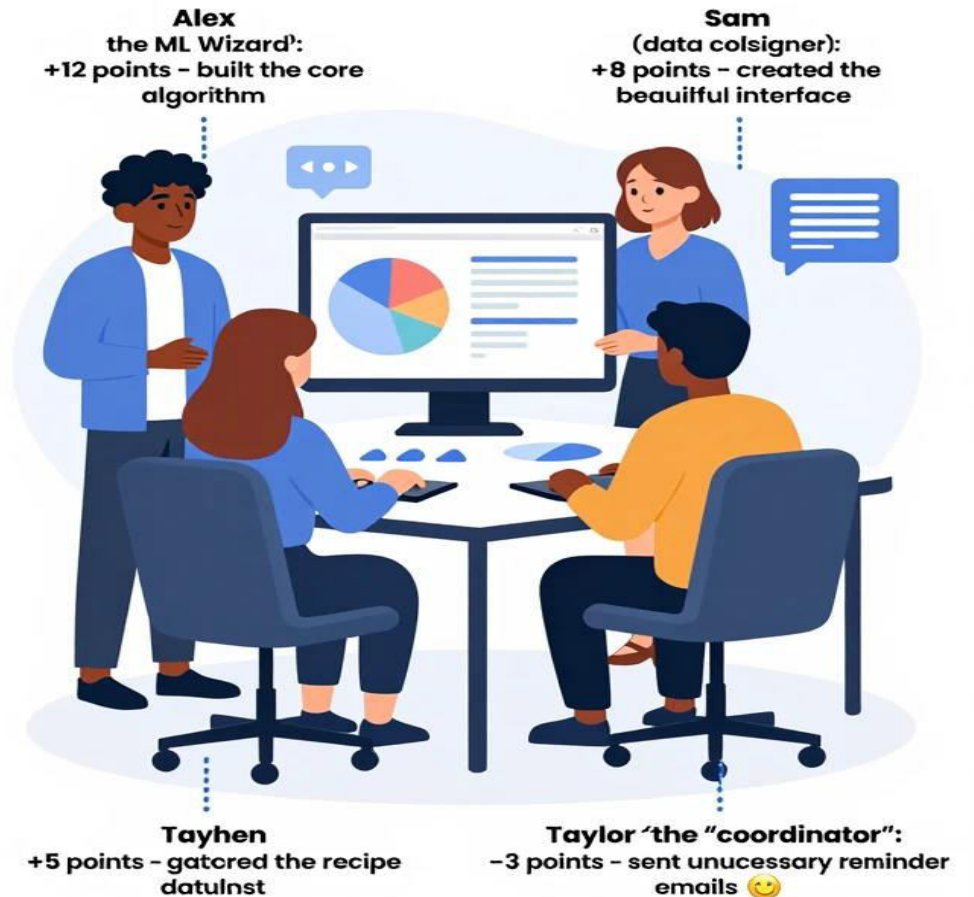
The Question: How much did each team member contribute to achieving this grade?

🙄 **What is SHAP?**

SHAP (SHapley Additive exPlanations) is like being the professor who needs to figure out exactly how much each student contributed to the final project grade. It's based on game theory and answers: "What happens to the outcome if we add or remove this person?"

"In machine learning, SHAP does the same thing but for features instead of people!"

AI-Powered Recipe Recommender



SHAP Analysis: Individual Contributions

Base value (starting point without anyone): **70%** (a functioning but basic app)

Final grade: **92%**

Total improvement to explain: **+22 points**

Team Member	SHAP Value	Impact	Interpretation
Alex 🧑💻	+12 points	★ ★ ★	The ML algorithm was the core innovation
Sam 🧑💻	+8 points	★ ★	Great UX impressed the professors
Jordan 🧑💻	+5 points	★	Good data enabled the algorithm
Taylor 🧑	-3 points	⚠️	Coordination emails were... unnecessary spam

💡 The Magic of SHAP

SHAP considers ALL possible team combinations!

What if Alex worked alone? → 82%

What if Sam worked alone? → 78%

What if Alex + Jordan worked together? → 87%

What if everyone except Taylor worked together? → 95%!

Visual Breakdown:

Alex's Contribution

+12 points (55%)

Sam's Contribution

+8 points (36%)

Jordan's Contribution

+5 points (23%)

Taylor's Contribution

-3 points
-4.3%

SHAP looks at every possible subset of the team and calculates each person's average marginal contribution across all those scenarios. It's like running every possible "what if" experiment!

Capestone Project Example : Python Implementation

```
# Install if needed
# pip install shap scikit-learn pandas numpy matplotlib

import numpy as np
import pandas as pd
import shap
import matplotlib.pyplot as plt

from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split

# -----
# 1. Create Synthetic Dataset
# -----
# Each column = contribution score [0-10 scale for example]

np.random.seed(42)

data_size = 200

data = pd.DataFrame({
    "Alex_ML_Algorithm": np.random.uniform(5, 10, data_size),
    "Sam_UI_Interface": np.random.uniform(4, 10, data_size),
    "Tayhen_Dataset": np.random.uniform(3, 10, data_size),
    "Taylor_Coordination": np.random.uniform(1, 8, data_size)
})
```



```
# Create target (Final Grade %)
# Assume weighted importance (hidden true logic)
data["Final_Grade"] = (
    0.4 * data["Alex_ML_Algorithm"] +
    0.3 * data["Sam_UI_Interface"] +
    0.2 * data["Tayhen_Dataset"] +
    0.1 * data["Taylor_Coordination"]
) * 10 + np.random.normal(0, 2, data_size)

# -----
# 2. Train ML Model
# -----
X = data.drop("Final_Grade", axis=1)
y = data["Final_Grade"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = RandomForestRegressor(n_estimators=200, random_state=42)
model.fit(X_train, y_train)

print("Model Test Score:", model.score(X_test, y_test))

# -----
# 3. SHAP Explainability
# -----
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test)
```



Capestone Project Example : Python Implementation

```
# -----
# 4. Global Contribution Plot
# -----
shap.summary_plot(shap_values, X_test)

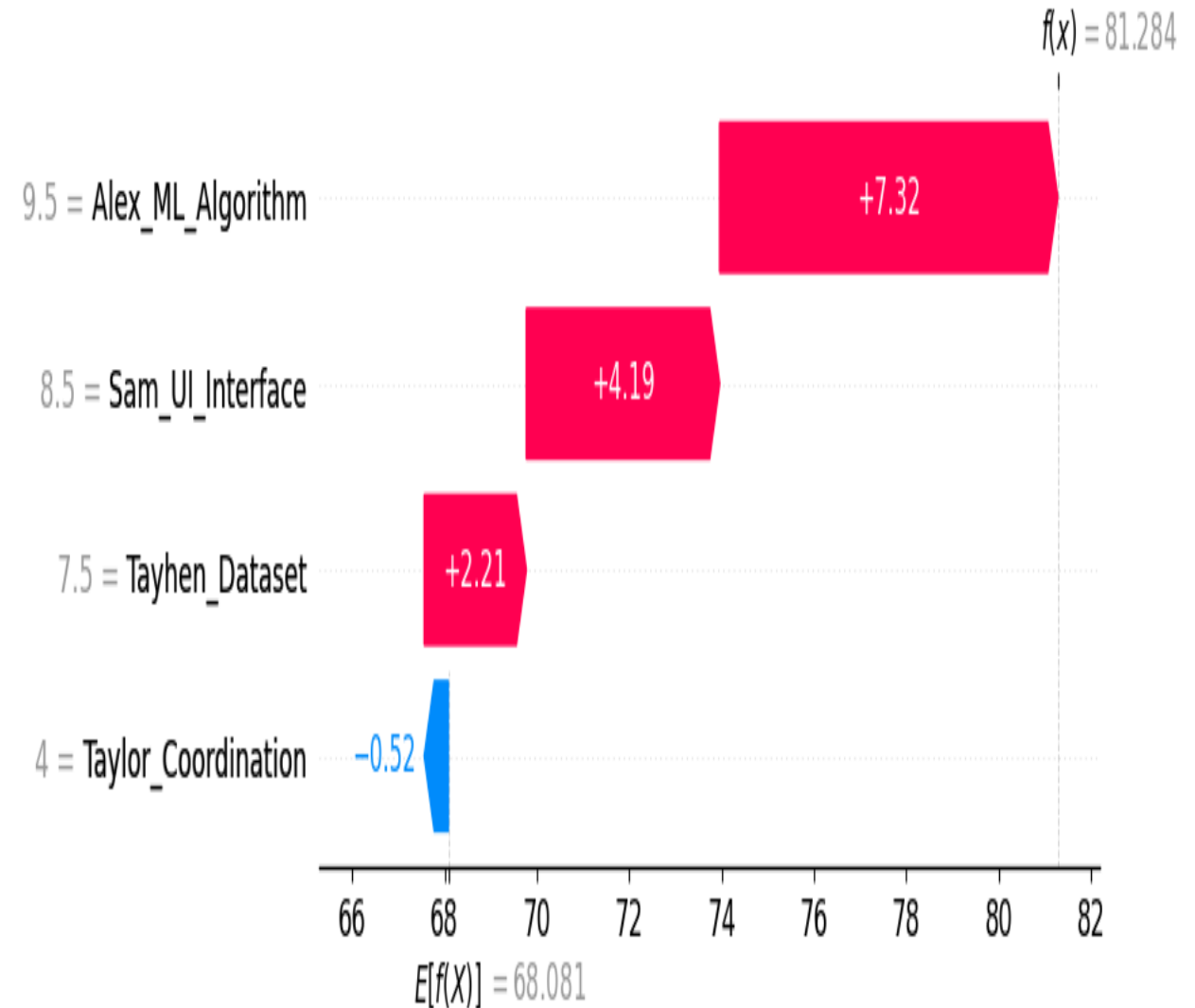
# -----
# 5. Explain One Specific Project (Capstone Team Case)
# -----
# Example: Your actual team scenario

team_project = pd.DataFrame({
    "Alex_ML_Algorithm": [9.5],
    "Sam_UI_Interface": [8.5],
    "Tayhen_Dataset": [7.5],
    "Taylor_Coordination": [4.0]
})

predicted_grade = model.predict(team_project)
print("Predicted Final Grade:", predicted_grade[0])

# SHAP values for this project
team_shap = explainer.shap_values(team_project)

# Waterfall plot (individual contribution)
shap.plots._waterfall.waterfall_legacy(
    explainer.expected_value,
    team_shap[0],
    team_project.iloc[0]
)
```



SHAP Explainers

The SHAP library offers several explainers, each optimized for different types of machine learning models to approximate Shapley values efficiently. The appropriate explainer depends on the specific model architecture you are using.

The main explainers are:

- shap.Explainer**: This is the primary, unified interface for the SHAP library. It can automatically select the appropriate underlying algorithm (e.g., permutation, partition, tree, linear, deep) based on the model and masker you provide. This is the recommended starting point for most users.

MODEL-SPECIFIC

- shap.TreeExplainer**: This explainer is highly optimized for tree-based models, such as decision trees, random forests, XGBoost, LightGBM, and CatBoost. It is a model-specific method and significantly faster than the model-agnostic alternatives for these model types.

- shap.DeepExplainer**: Tailored for deep learning models (e.g., TensorFlow/Keras, and some PyTorch models), DeepExplainer builds upon the DeepLIFT algorithm to approximate SHAP values efficiently for high-dimensional data like images.

- shap.GradientExplainer**: Also used for deep learning models, this explainer uses expected gradients to compute explanations, which is an extension of integrated gradients. Both DeepExplainer and GradientExplainer are primary options for neural networks.

- shap.LinearExplainer**: This explainer is optimized for linear models, such as linear regression and logistic regression. It leverages the model's coefficients for fast computation of SHAP values.

MODEL-AGNOSTIC

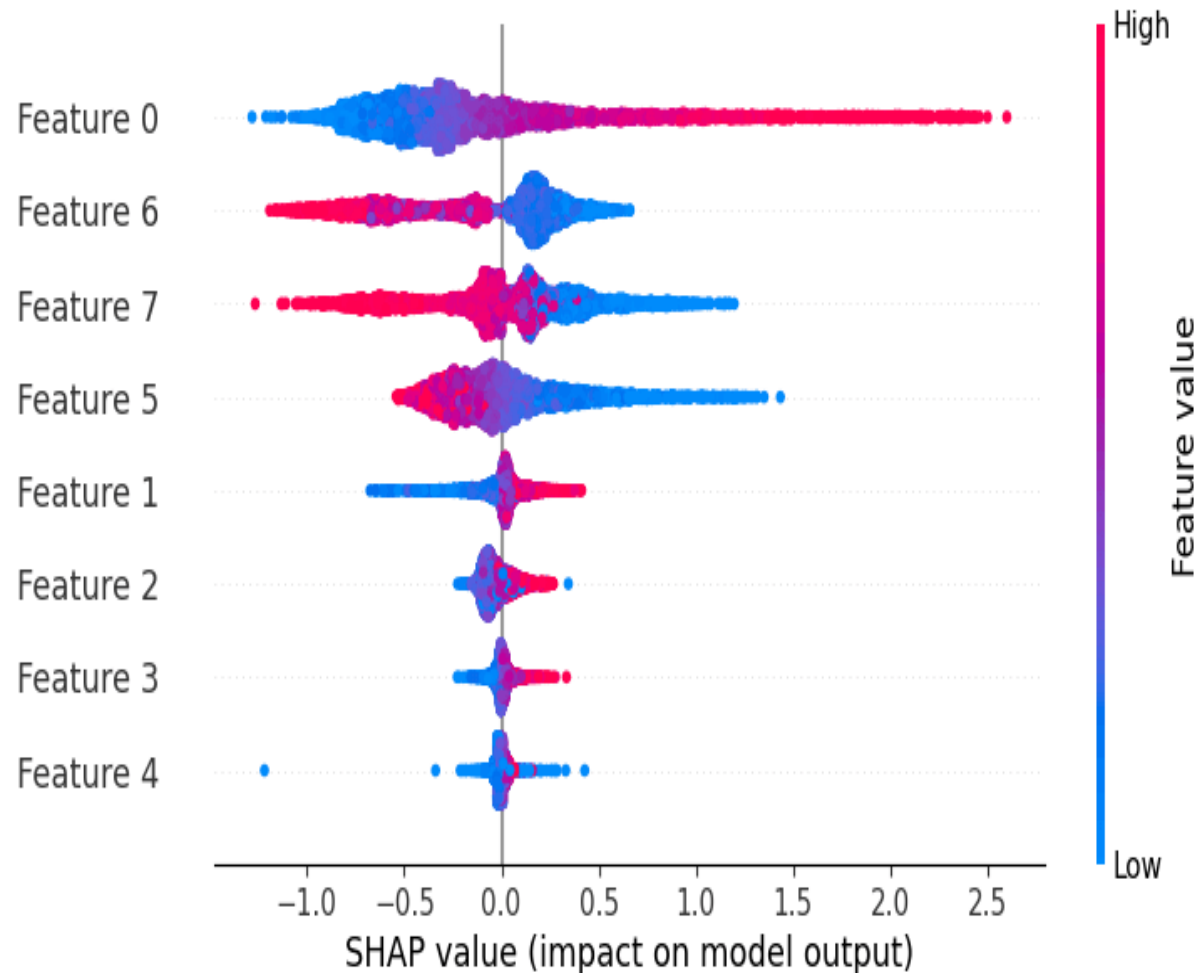
- shap.KernelExplainer**: This is a model-agnostic explainer that can be used with any machine learning model or Python function. It uses an approximation method (local surrogate models) to estimate Shapley values, but it is computationally slower than model-specific explainers, especially with a large number of features.

- shap.PermutationExplainer**: This is another model-agnostic explainer that estimates SHAP values by permuting features.

- shap.PartitionExplainer**: This uses Owen values and is optimized for models that can handle hierarchical masking (e.g., in natural language processing with the shap.maskers.Text masker).

- shap.ExactExplainer**: This computes the exact SHAP values but has an exponential time complexity $O(2^m)$ where m is the no. of features

SHAP Visualization



This plot is made of all the dots in the train data. It demonstrates the following information:

- 1.Feature importance:** Variables are ranked in descending order.
- 2. Impact:** The horizontal location shows whether the effect of that value is associated with a higher or lower prediction.
- 3. Original value:** Color shows whether that variable is high (in red) or low (in blue) for that observation.
- 4. Correlation:** A high level of the "Type_of_Cab" content has a low and Positive impact on the "Surge_Pricing_Type". The "low" comes from the blue color, and the "positive" impact is shown on the X-axis. Similarly, the "Trip_Distance" negatively correlates with the target variable.

Red indicates a higher, and **blue** indicates a lower. We can verify the impact (Positive or Negative) from the X-axis for that specific data.

SHAP Visualization Plots

SHAP (SHapley Additive exPlanations) visualization plots are tools used to interpret machine learning model predictions by showing how much each feature contributes to the output.

They are divided into global interpretability (overall model behavior) and local interpretability (individual predictions).

- **Global Interpretation Plots**

- **Local Interpretation Plots**

- **Feature Relationship Plots**

- Dependence Plot, Interaction Plot

- **Specialized Plots**

- Image plot, text plot, monitoring plot

Common Use Cases:

- **Identifying Top Features:** Bar and Beeswarm plots.
- **Debugging Individual Predictions:** Waterfall and Force plots.
- **Understanding Feature Trends:** Dependence plots.

https://shap.readthedocs.io/en/latest/example_notebooks/api_examples/plots

1. Global Interpretation Plots(Dataset Level)

Beeswarm Plot (shap.plots.beeswarm): The standard plot for summarizing feature importance. It displays the distribution of SHAP values for each feature across all samples. Colors represent feature values (red=high, blue=low), allowing you to see if a high value increases or decreases the prediction.

Summary Plot (shap.summary_plot): Similar to the beeswarm, it provides a comprehensive view of feature importance and the direction of their impact.

Bar Plot (shap.plots.bar): Displays the mean absolute SHAP values for each feature, showing which features are most important on average.

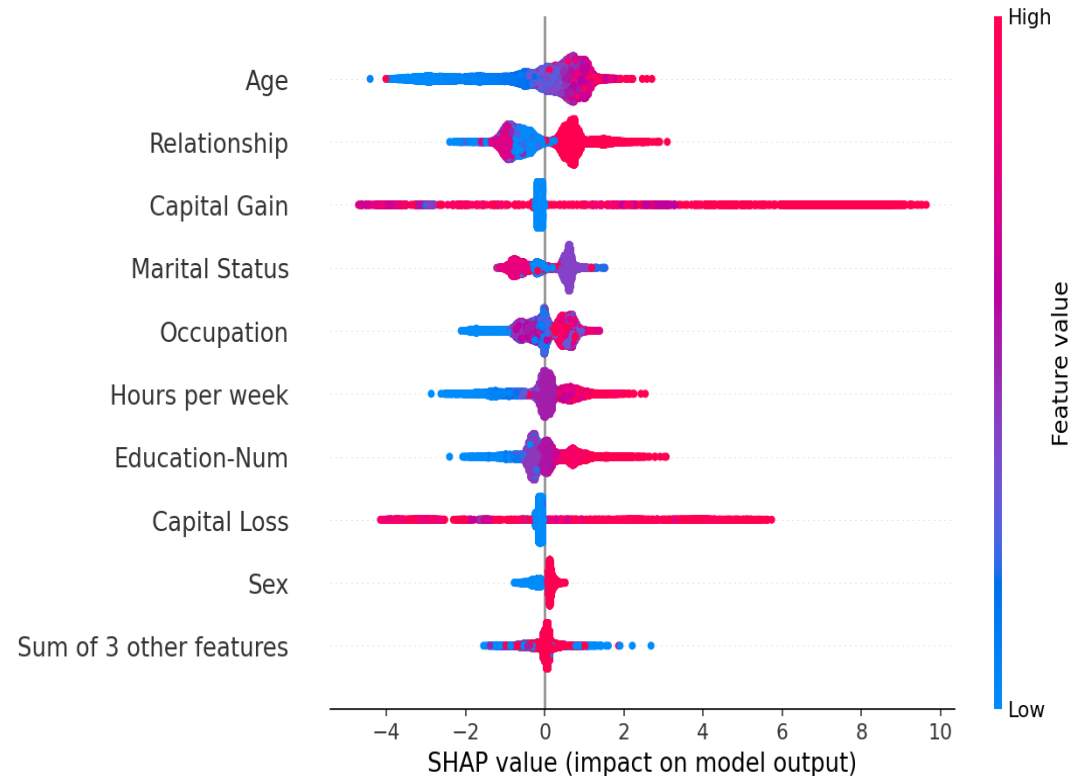
Heatmap (shap.plots.heatmap): Shows the population structure of a dataset using supervised clustering, which helps identify patterns in feature contributions across instances.

Uses an XGBoost model trained on the classic UCI adult income dataset (which is a classification task to predict if people made over \(\$50k in the 1990s).

1. Global Interpretation Plot : Beeswarm Plot

- **(shap.plots.beeswarm):** The standard plot for summarizing feature importance.
- It displays the distribution of SHAP values for each feature across all samples.
- Each instance the given explanation is represented by a single dot on each feature row. The x position of the dot is determined by the SHAP value **(shap_values.value[instance,feature])** of that feature, and dots “pile up” along each feature row to show density.
- Color is used to display the original value of a feature **(shap_values.data[instance,feature])**.
- Summary plot also similar to Beeswarm plots

In the plot below we can see that Age is the most important feature on average, and than young (**blue**) people are less likely to make over \\$50k.



By default the maximum number of features shown is ten, but this can be adjusted with the **max_display** parameter: **shap.plots.beeswarm(shap_values, max_display=20)**

1. Global Interpretation Plot : Bar Plot

- **(shap.plots.bar)**: Displays the mean absolute SHAP values for each feature, showing which features are most important on average.
- Passing a matrix of SHAP values to the bar plot function creates a global feature importance plot, where the global importance of each feature is taken to be the mean absolute value for that feature over all the given samples.

```
import xgboost
```

```
import shap
```

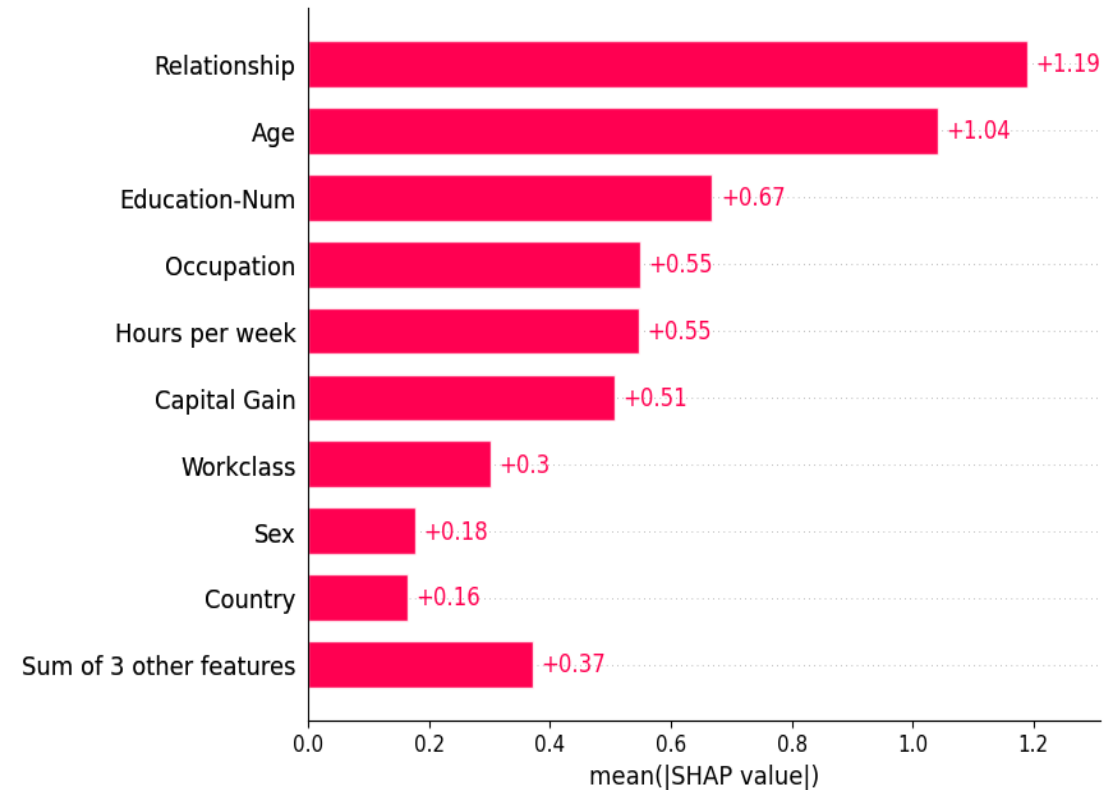
```
# train XGBoost model
```

```
X, y = shap.datasets.adult(n_points=2000)
model = xgboost.XGBClassifier().fit(X, y)
```

```
# compute SHAP values
```

```
explainer = shap.Explainer(model, X)
shap_values = explainer(X)
```

In the plot below we can see that Age is the most important feature on average, and that young (blue) people are less likely to make over \$50k.

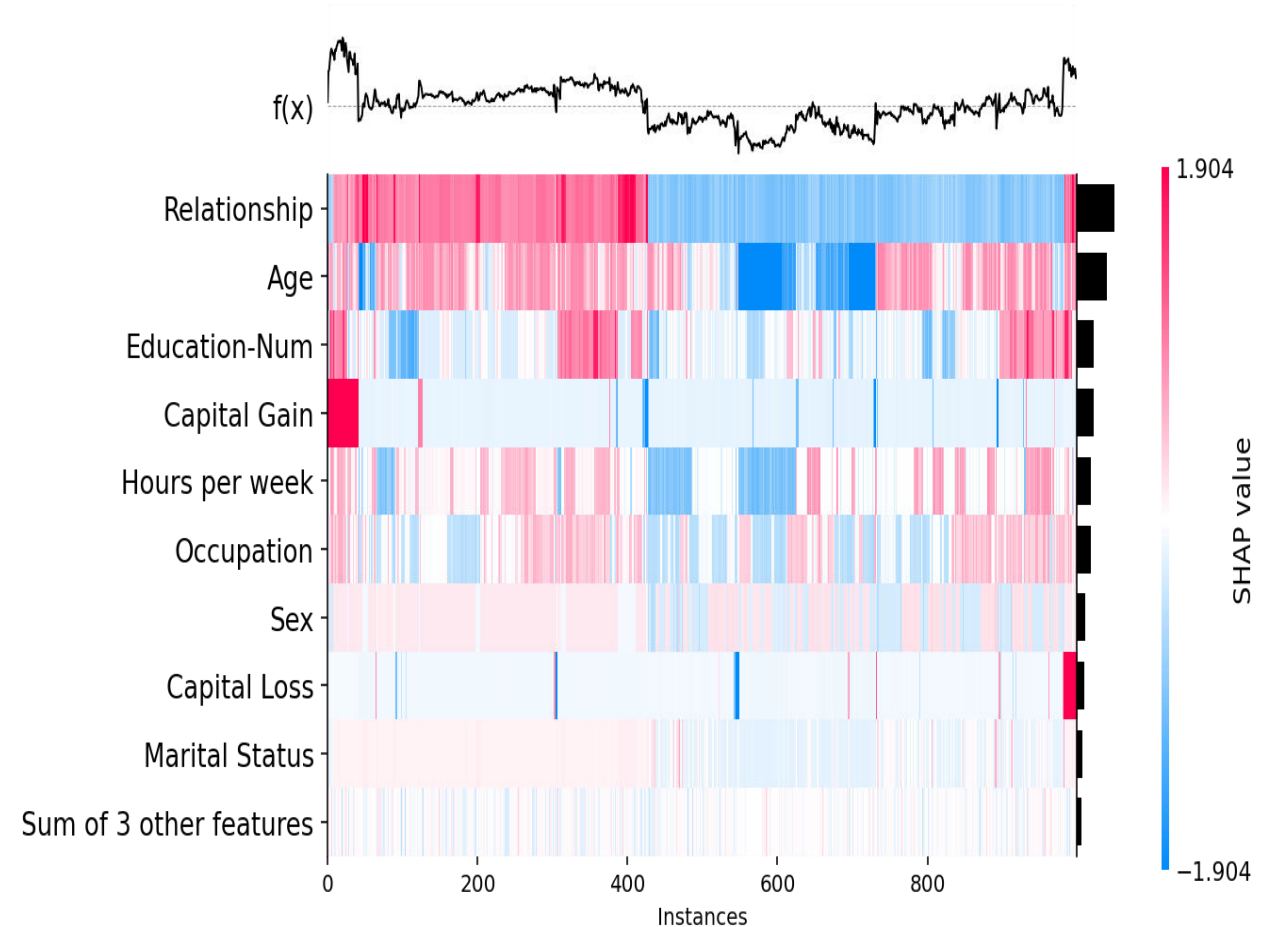


By default the maximum number of features shown is ten, but this can be adjusted with the `max_display` parameter: `shap.plots.bar(shap_values, max_display=12)`

1. Global Interpretation Plot : Heat Map

- **(shap.plots.heatmap):** Shows the population structure of a dataset using supervised clustering, which helps identify patterns in feature contributions across instances.
- Passing a matrix of SHAP values to the heatmap plot function creates a plot with the instances on the x-axis, the model inputs on the y-axis, and the SHAP values encoded on a color scale.
- **The output of the model is shown above the heatmap matrix (centered around the explanation's .base_value), and the global importance of each model input shown as a bar plot on the right hand side of the plot (by default this is the shap.order.abs.mean measure of overall importance).**

`shap.plots.heatmap(shap_values)`

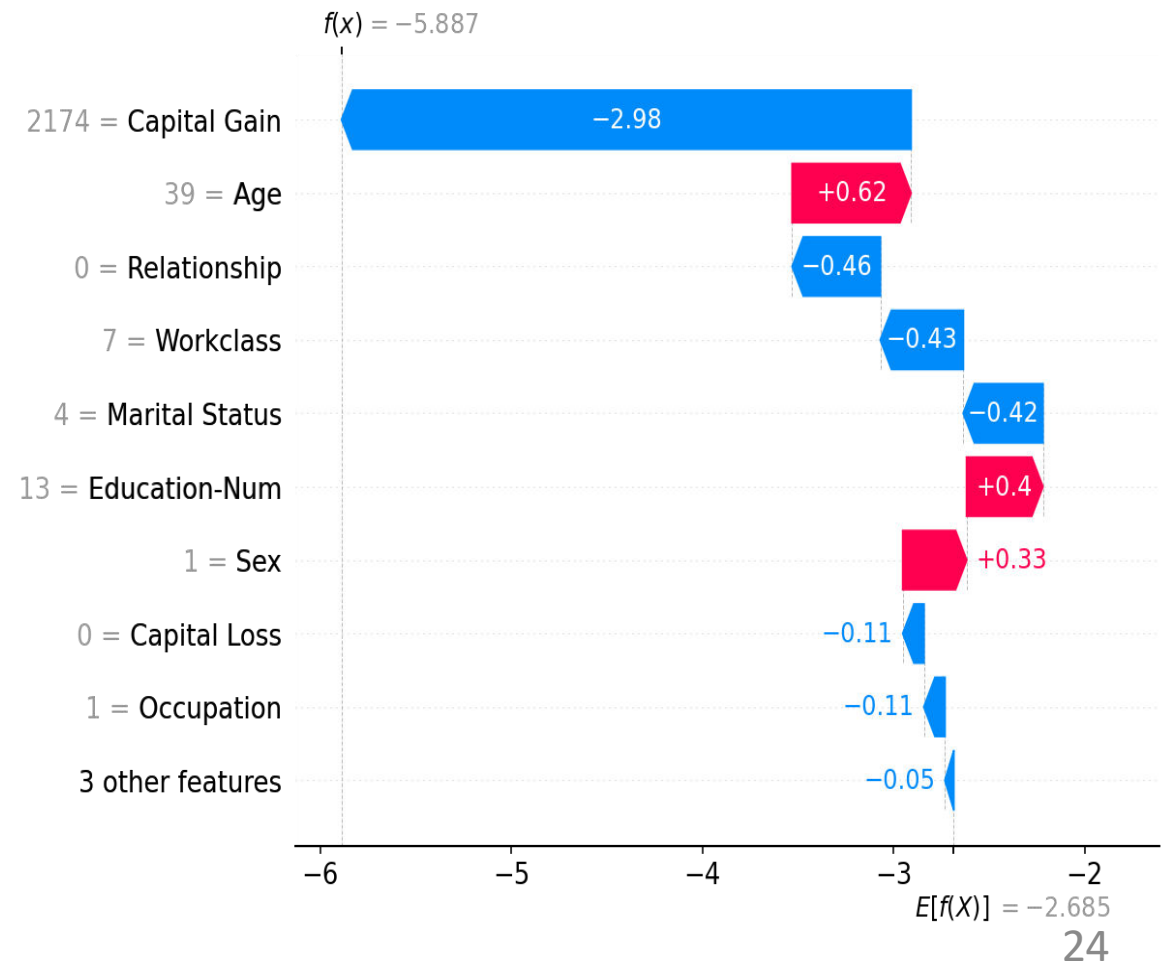


2. Local Interpretation Plot : Waterfall plot

- **(shap.plots.waterfall):** Explains a single prediction by showing how each feature moves the model output from the base value (average) to the final predicted value.
- Waterfall plots are designed to display explanations for individual predictions, so they expect a single row of an Explanation object as input.
- The bottom of a waterfall plot starts as the expected value of the model output, and then each row shows how the positive (red) or negative (blue) contribution of each feature moves the value from the expected model output over the background dataset to the model output for this prediction.

Note that by default SHAP explains XGBoost classifier models in terms of their margin output, before the logistic link function. That means the units on the x-axis are log-odds units, so negative values imply probabilities of less than 0.5 that the person makes over \$50k annually. The gray text before the feature names shows the value of each feature for this sample.

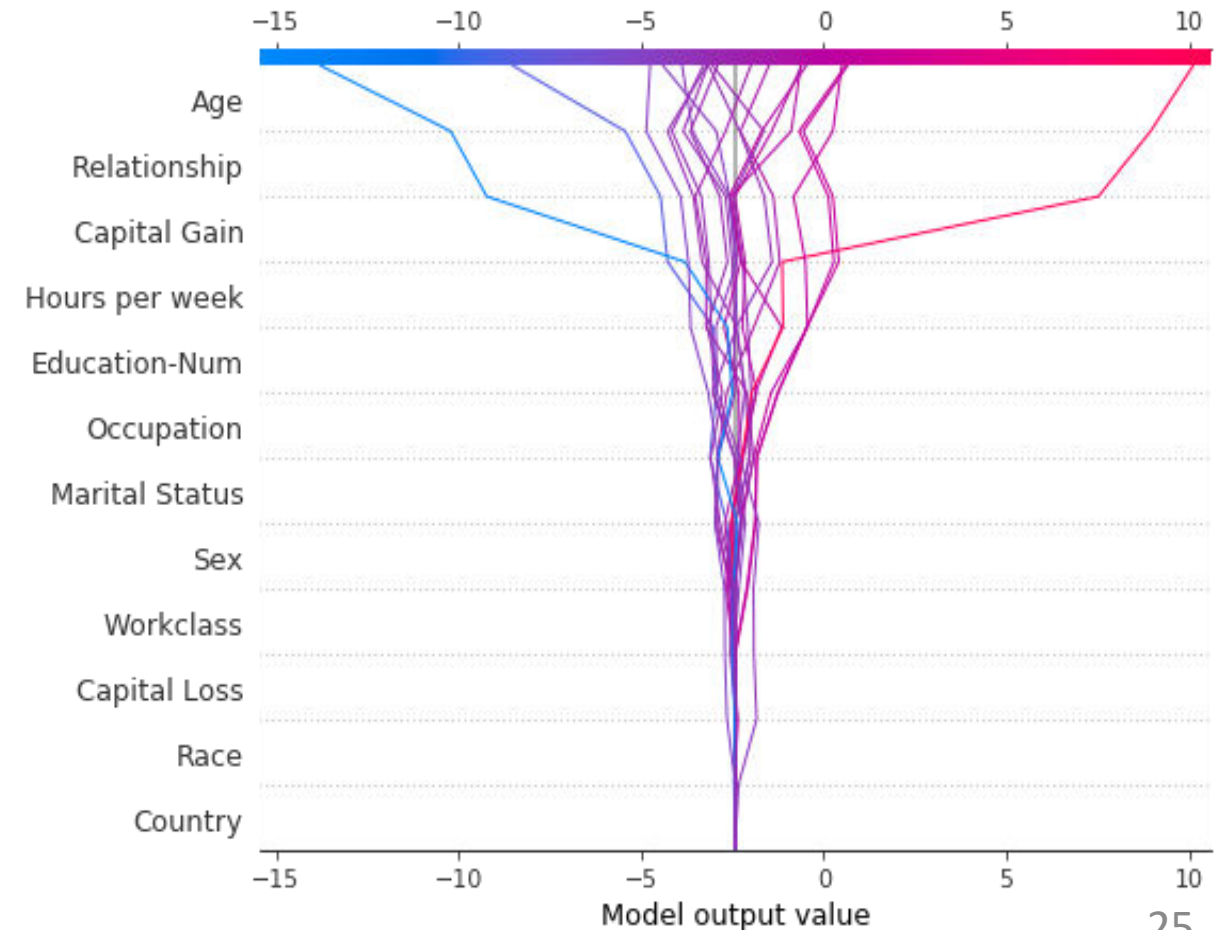
`shap.plots.waterfall(shap_values[0])`



2. Local Interpretation Plot : Decision Plot

- SHAP decision plots show how complex models arrive at their predictions (i.e., how models make decisions). This notebook illustrates decision plot features and use cases with simple
- Each plotted line on the decision plot shows how strongly the individual features contributed to a single model prediction, thus explaining what feature values pushed the prediction.
- Note: This plot isn't informative by itself; we use it only to illustrate the primary concepts. * The x-axis represents the model's output. In this case, the units are log odds. * The plot is centered on the x-axis at explainer.expected_value. All SHAP values are relative to the model's expected value like a linear model's effects are relative to the intercept. * The y-axis lists the model's features.
- Each observation's prediction is represented by a colored line
- At the bottom of the plot, the observations converge at explainer.expected_value.

`shap.decision_plot(expected_value, shap_values, features_display)`





THANK YOU



UE23AM342BA3
Explainable AI

Local model agnostic Methods

Prof. Ayisha Noori VK
Dept. of CSE(AI&ML), PES University,
Bengaluru

Contents:

- Counter Factual Explanations
- Anchors

Counter Factual Explanations

Counterfactual explanations answer:

- “What minimal change in input would change the prediction?”

Simple Example:

- Model says: ✗ Loan Rejected

Counterfactual:

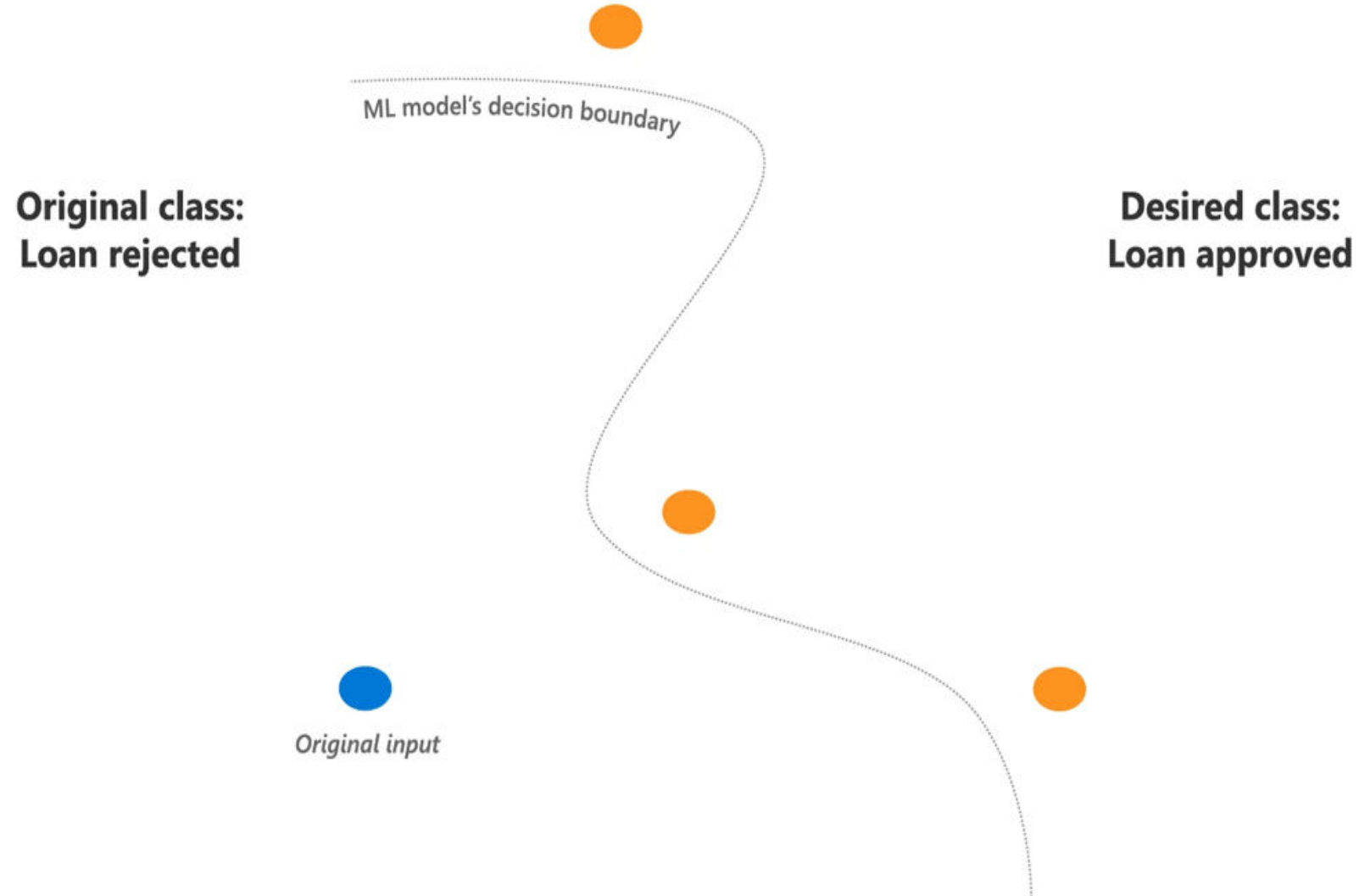
✓ If income increased by ₹10,000 → Loan Approved

Key Properties

- Actionable
- Human understandable
- Local explanation (instance-specific)



Counterfactual Examples



Mathematical Intuition

Counterfactual explanations are usually framed as an optimization problem:

Approach suggested by Wachter et. al (2017)

To minimize the following loss

$$L(x, x', y', \lambda) = \lambda \cdot (\hat{f}(x') - y')^2 + d(x, x')$$

The first term is the quadratic distance between the model prediction for the counterfactual x' and the desired outcome y' , which the user must define in advance.

The second term is the distance d between the instance x to be explained and the counterfactual x' .

The λ parameter balances the distance in prediction (first term) against the distance in feature values

(second term). The loss is solved for a given λ and returns a counterfactual x'

A higher value of λ means that we prefer counterfactuals that come close to the desired outcome y' , a lower value means that we prefer counterfactuals x' that are very similar to x in the feature values

$$|\hat{f}(x') - y'| \leq \epsilon$$

To minimize this loss function, any suitable optimization algorithm can be used, e.g. Nelder-Mead. If you have access to the gradients of the machine learning model, you can use gradient-based methods like ADAM. The instance x to be explained, the desired output y' and the tolerance parameter ϵ must be set in advance. The loss function is minimized for x' and the (locally) optimal counterfactual x' returned while increasing λ until a sufficiently close solution is found (= within the tolerance parameter).

$$\arg \min_{x'} \max_{\lambda} L(x, x', y', \lambda)$$

The function d for measuring the distance between instance x and counterfactual x' is the Manhattan distance weighted feature-wise with the inverse median absolute deviation (MAD).

$$d(x, x') = \sum_{j=1}^p \frac{|x_j - x'_j|}{MAD_j}$$

The total distance is the sum of all p feature-wise distances, that is, the absolute differences of feature values between instance x and counterfactual x' . The feature-wise distances are scaled by the inverse of the median absolute deviation of feature j over the dataset defined as:

$$MAD_j = \text{median}_{i \in \{1, \dots, n\}} (|x_{i,j} - \text{median}_{l \in \{1, \dots, n\}} (x_{l,j})|)$$

How to Produce Counterfactual Explanations?

1. Select an instance x to be explained, the desired outcome y' , a tolerance ϵ and a (low) initial value for λ .
2. Sample a random instance as initial counterfactual.
3. Optimize the loss with the initially sampled counterfactual as starting point.
4. While $|\hat{f}(x') - y'| > \epsilon$:
 - Increase λ .
 - Optimize the loss with the current counterfactual as starting point.
 - Return the counterfactual that minimizes the loss.
5. Repeat steps 2-3 and return the list of counterfactuals or the one that minimizes the loss.

Counterfactual Explanations

Counterfactual explanations are usually framed as an optimization problem:

Mathematical Form:

Objective

- Minimize:
- Distance between original input and counterfactual input

Constraint

- Prediction must change to desired class.

$$\operatorname{argmin}_{x'} \operatorname{distance}(x, x') + \lambda \operatorname{Loss}(f(x'), y_{\text{target}})$$

Where:

- $x \rightarrow$ Original input
- $x' \rightarrow$ Counterfactual input
- $f(x) \rightarrow$ Model prediction
- $y_{\text{target}} \rightarrow$ Desired output

Counterfactual Explanations : Real World Practical Examples

Loan Approval

Original:

- Income = 30K
 - Credit Score = 600
- Prediction = Reject

Counterfactual:

- Income = 40K
- Prediction = Approve

Healthcare

Original:

BMI = 32
Sugar = 180
Prediction = High Risk

Counterfactual:

BMI = 27
Sugar = 140
Prediction = Low Risk

Student Performance

Original:

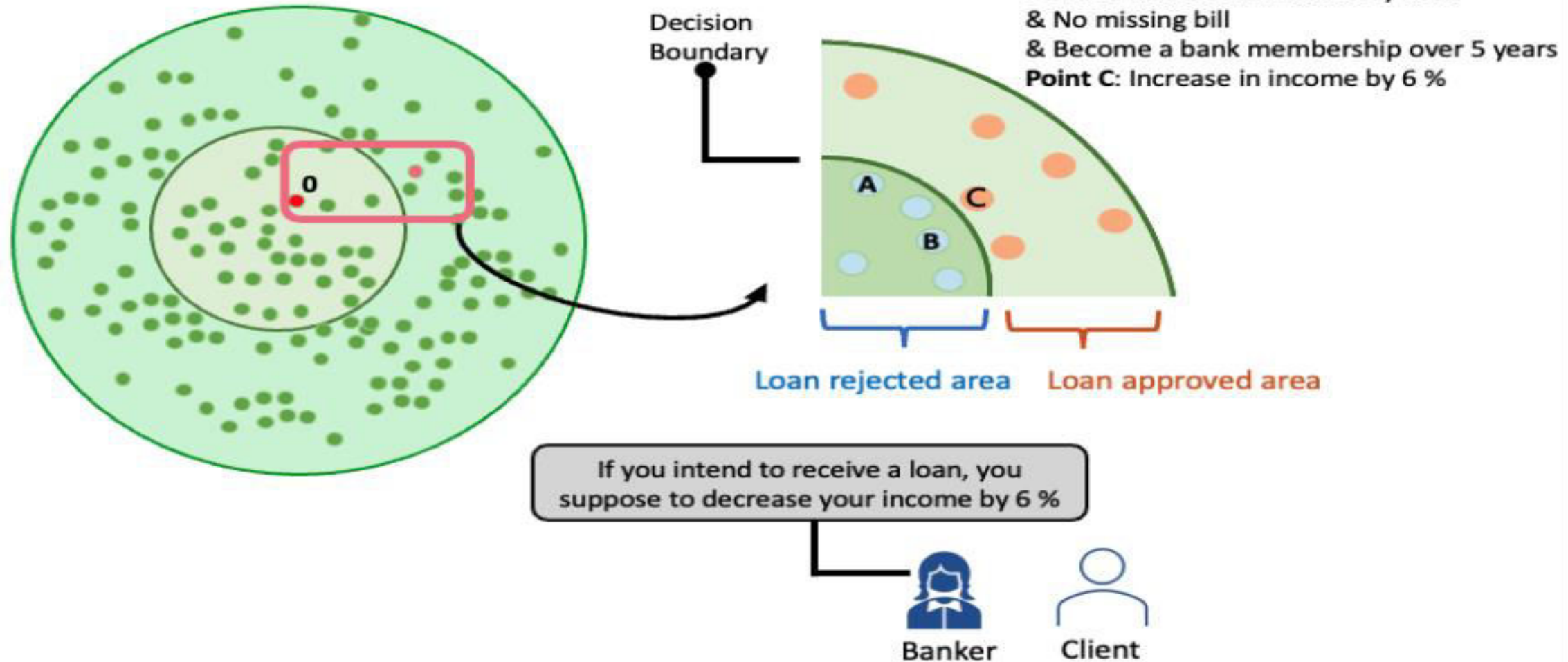
- Study hours = 2/day
- Prediction = Fail

Counterfactual:

- Study hours = 4/day
- Prediction = Pass

Counterfactual generation

| Loan application scenario



Counterfactual generation graph: Each point in the graph contains a different condition of an applicant, including age, credit amount, credit history, etc. A generated counterfactual contains the different conditions from the input of an applicant (point 0) that lead to a loan-approved (orange dots) result. For example, point c shows that the applicant's loan application could be approved if their income was increased by 6%. Points A and B, on the other hand, represent the characteristics of an applicant whose loan application would be rejected.

Counterfactual Explanations : Real World Practical Examples

A three-layer fully-connected neural network is trained to predict a student's average grade of the first year at law school, based on grade point average (GPA) prior to law school, race and law school entrance exam scores. The goal is to find counterfactual explanations for each student that answer the following question: How would the input features need to be changed, to get a predicted score of 0?

Score	GPA	LSAT	Race	GPA x'	LSAT x'	Race x'
0.17	3.1	39.0	0	3.1	34.0	0
0.54	3.7	48.0	0	3.7	32.4	0
-0.77	3.3	28.0	1	3.3	33.5	0
-0.83	2.4	28.5	1	2.4	35.8	0
-0.57	2.7	18.3	0	2.7	34.9	0

- The first column contains the predicted score, the next 3 columns the original feature values and the last 3 columns the counterfactual feature values that result in a score close to 0.
- The first two rows are students with above-average predictions, the other three rows below-average.
- The counterfactuals for the first two rows describe how the student features would have to change to decrease the predicted score and for the other three cases how they would have to change to increase the score to the average.
- The counterfactuals for increasing the score always change the race from black (coded with 1) to white (coded with 0) which shows a racial bias of the model.
- The GPA is not changed in the counterfactuals, but LSAT is.

DiCE – **D**iverse **C**ounterfactual **E**xplanations

- The interpretml/DiCE library (Diverse Counterfactual Explanations) is an open-source Python package used for generating "what-if" explanations for machine learning models. It is part of the broader [InterpretML](#) framework for explainable AI (XAI).
- DiCE is based on [recent research](#) that generates CF explanations for any ML model. The core idea is to setup finding such explanations as an optimization problem, similar to finding adversarial examples.
- The critical difference is that for explanations, we need perturbations that change the output of a machine learning model, but are also diverse and feasible to change. Therefore, DiCE supports generating a set of counterfactual explanations and has tunable parameters for diversity and proximity of the explanations to the original input.
- It also supports simple constraints on features to ensure feasibility of the generated counterfactual examples



With DiCE, generating explanations is a simple three-step process:

1. set up a dataset,
2. train a model,
3. Invoke DiCE to generate counterfactual examples for any input.

DiCE can also work with pre-trained models, with or without their original training data.

Python Implementation (Using DiCE)

#Install Libraries

```
pip install dice-ml
```

```
pip install scikit-learn pandas
```

#Import Libraries

```
import pandas as pd
```

```
import dice_ml
```

```
from dice_ml import Dice
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.ensemble import RandomForestClassifier
```

#Load the dataset

```
#Example: Loan Dataset
```

```
data = pd.read_csv("loan_data.csv")
```

#Train the model

```
X = data.drop("loan_status", axis=1)
```

```
y = data["loan_status"]
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

```
model = RandomForestClassifier()
```

```
model.fit(X_train, y_train)
```

#Create DiCE Data + Model Objects

```
d = dice_ml.Data(  
    dataframe=data,  
    continuous_features=['income', 'credit_score'],  
    outcome_name='loan_status'  
)
```

```
m = dice_ml.Model(model=model, backend="sklearn")
```

```
exp = Dice(d, m)
```

#Generate Counter Factuals

```
query_instance = X_test.iloc[0]
```

```
cf = exp.generate_counterfactuals(  
    query_instance,  
    total_CFs=3,  
    desired_class="opposite"
```

```
)
```

```
cf.visualize_as_dataframe()
```

Example Output (Sample)

Original Input

Feature	Value
Income	30000
Credit Score	620
Prediction → Reject	

Counterfactual Output

Feature	Original	Counterfactual
Income	30000	42000
Credit Score	620	650
Prediction → Approve 		

Counterfactual Generation Failure

Counterfactual generation failure occurs when the optimization/search algorithm is unable to find valid input modifications that change the model prediction while satisfying constraints.

Example error (DiCE library): *“No counterfactuals found for any of the query points”*

Major Reasons for Counterfactual Generation Failure:

- 1. Query Instance Already in Desired Class** : If the input already belongs to the target class, counterfactual cannot be generated.
Example: Desired Class = Loan Approved
Query instance already Approved
- 2. Model Too Confident** (Far from Decision Boundary) : If prediction probability is very high (e.g., 0.99), small changes may not flip prediction.
Reason: Strong decision boundary separation, Overfitted model
- 3. Feature Constraints Too Strict** : If allowed feature ranges are very narrow, search space becomes small.
Example: Income allowed change: only ₹100 But model needs ₹10,000 change
- 4. Incorrect Feature Type Configuration** : If continuous features are wrongly marked as categorical (or vice versa), optimization fails.
- 5. Limited Features Allowed to Vary** : If only few features are allowed to change, valid counterfactual may not exist.
- 6. Complex or Non-linear Model Structure** : Deep neural networks or highly non-linear models may have complex decision surfaces.

Generating counterfactuals for multi-class classification and regression models

- [Generating counterfactuals for multi-class classification and regression models — DiCE 0.12 documentation](#)

ANCHORS(Scoped Rules) :

- **Anchor explanations** are rule-based explanations that tell:
“If these conditions are true, the model will almost always give this prediction.”

Anchors are:

- High precision rules
- Human understandable
- Local explanations

Implementation Examples :

[GitHub - marcotcr/anchor: Code for "High-Precision Model-Agnostic Explanations" paper](#)

ANCHORS : Example Scenario

Proposal Outcome Predictor

Problem Setup

- Suppose we build an AI model to predict: **Will a person say YES to a Valentine proposal?**
- Output:
- 1 → Will say YES 
- 0 → Will Say NO 

This means:

- 👉 As long as these 2 conditions hold,
- 👉 Model will almost always predict **Will say Yes**

Sample Data : Person(with 6 features)



Person Details

- Age → 24
- Relationship Status → In Relationship
- Years known → 3
- Gift Budget → 5000
- Daily chat Hours → 2.4
- Same City → Yes



Model Prediction

Will say yes 

Anchor Rule:

**IF Years_Known > 2 AND
Daily_Chat_Hours > 1.5 THEN
Prediction = YES (Precision = 0.95))**

Python Implementation

```
!pip install alibi scikit-learn pandas numpy matplotlib
```

```
import numpy as np
```

```
import pandas as pd
```

```
np.random.seed(42)
```

```
n = 500
```

```
data = pd.DataFrame({
```

```
    "Age": np.random.randint(18, 35, n),
```

```
    "Years_Known": np.random.randint(0, 10, n),
```

```
    "Daily_Chat_Hours": np.round(np.random.uniform(0, 5, n), 2),
```

```
    "Gift_Budget": np.random.randint(500, 10000, n),
```

```
    "Relationship_Status": np.random.choice([0,1], n), # 0=Friend, 1=Close
```

```
    "Same_City": np.random.choice([0,1], n)
```

```
})
```

```
# Target Logic (Valentine YES/NO)
```

```
data["Will_Say_Yes"] = (
```

```
    (data["Years_Known"] > 2) &
```

```
    (data["Daily_Chat_Hours"] > 1.5) &
```

```
    (data["Gift_Budget"] > 2000)
```

```
).astype(int)
```

```
data.head()
```

#Train Model

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
```

```
X = data.drop("Will_Say_Yes", axis=1)
```

```
y = data["Will_Say_Yes"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
model = RandomForestClassifier()
model.fit(X_train, y_train)
```

```
print("Model Accuracy:", model.score(X_test, y_test))
```

#Anchor Explainer Set

```
from alibi.explainers import AnchorTabular
```

```
feature_names = X.columns.tolist()
```

```
explainer = AnchorTabular(
    predictor=model.predict,
    feature_names=feature_names
)
explainer.fit(X_train.values)
```

#Generate Anchor Explanation

Select one person

```
instance = X_test.iloc[0].values
```

```
explanation = explainer.explain(instance)
```

```
print("Prediction:", model.predict(instance.reshape(1,-1))[0])
```

```
print("\nAnchor Rule:")
```

```
print(" AND ".join(explanation.anchor))
```

```
print("\nPrecision:", explanation.precision)
```

```
print("Coverage:", explanation.coverage)
```

Anchor vs LIME vs SHAP vs Counterfactual

These are four major Explainable AI (XAI) techniques used to understand model predictions.

They answer **different types of questions**

Method	Main Question Answered
Anchor	When will model prediction stay same?
LIME	Which features influenced this prediction locally?
SHAP	How much did each feature contribute fairly?
Counterfactual	What should change to flip prediction?

Use Anchor When:

- ✓ Need simple decision rules
- ✓ Need regulatory explanation
- ✓ Need stable logic

Use LIME When:

- ✓ Quick local explanation needed
- ✓ Model debugging

Use SHAP When:

- ✓ Need consistent explanation
- ✓ Need global feature importance
- ✓ Research / auditing

Use Counterfactual When:

- ✓ Need actionable recommendation
- ✓ Decision support system
- ✓ User guidance system

Detailed Comparison Table

Feature	Anchor	LIME	SHAP	Counterfactual
Type	Rule-based	Surrogate Model	Game Theory Based	Optimization Based
Explanation Style	IF-THEN Rules	Linear Local Approximation	Feature Contribution Values	Minimal Change Scenario
Scope	Local	Local	Local + Global	Local
Human Interpretability	Very High	Medium	Medium	Very High
Actionable Insight	Medium	Low	Low	Very High
Mathematical Complexity	Low	Medium	High	High
Stability	High	Medium	High	Depends on constraints



THANK YOU

sarcastic faculty version